UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

Segundo **Parcial**

NRO GRUPO: 43
MATERIA: INGENIERÍA DE SOFTWARE I
SIGLA: SC
DOCENTE: ING. ROLANDO ANTONIO MARTINEZ CANEDO
INTEGRANTES:

- Calero Suyo Trevor Félix 222153555

SANTA CRUZ - BOLIVIA
2026

Repositorio móvil

Repositorio frontend

Repositorio backend

Drive del proyecto

Contenido

1 Fundamentación Teórica 1

1.1 El Proceso Unificado de Desarrollo de Software (PUDS) 1

1.1.1 Origen y Principios Rectores del Proceso Unificado 1

1.1.2 Características Fundamentales: Dirigido por Casos de Uso, Centrado en la Arquitectura, Iterativo e Incremental 2

1.1.3 Las Cuatro Fases del Ciclo de Vida: Inicio, Elaboración, Construcción y Transición 3

1.1.4 Disciplinas y Flujos de Trabajo en PUDS 3

1.1.5 Artefactos y Entregables en Cada Fase del Proceso 4

1.2 Lenguaje Unificado de Modelado (UML) 2.5 y Superior 5

1.2.1 UML como Estándar ISO/IEC 19501 para la Especificación de Sistemas 5

1.2.2 Clasificación de Diagramas UML: Estructurales vs. de Comportamiento 5

1.2.3 Diagramas de Actividades: Sintaxis y Semántica para la Modelización de Flujos de Trabajo 6

1.2.4 Diagramas de Secuencia y Comunicación: Especificación de Interacciones entre Actores 7

1.2.5 Diagramas de Máquina de Estados: Modelado del Ciclo de Vida de Entidades 8

1.2.6 Perfiles UML (UML Profiles) y Mecanismos de Extensión (Estereotipos, Valores Etiquetados) 9

1.3 Patrones de Control de Flujo en Workflows 10

1.3.1 Fundamentos de los Sistemas de Gestión de Flujos de Trabajo (WfMS) 10

1.3.2 Análisis de Patrones de Control de Flujo (Workflow Patterns - Van Der Aalst) 11

1.3.3 Mecanismos de Enrutamiento Dinámico Basado en Reglas de Negocio 11

1.4 Herramientas de Ingeniería de Software Asistida por Computadora (CASE) 12

1.4.1 Definición y Clasificación de Herramientas CASE 12

1.4.2 Sparx Systems Enterprise Architect como Plataforma de Modelado Integral 13

1.4.3 Interoperabilidad de Modelos mediante el Estándar XMI (XML Metadata Interchange) 14

1.4.4 Integración de Modelos con Entornos de Desarrollo (IDEs) y Repositorios de Código Fuente 14

1.5 Inteligencia Artificial en el Soporte a la Ingeniería de Software..... 15

1.5.1 Procesamiento de Lenguaje Natural (PLN) para la Interpretación de Especificaciones 15

1.5.2 Reconocimiento Óptico de Caracteres (OCR) en la Digitalización de Formularios Físicos 16

1.5.3 Análisis de Rendimiento de Procesos mediante Simulación de Eventos Discretos 16

1.5.4 Identificación de Cuellos de Botella: Fundamentos de Minería de Procesos y Análisis de Tiempos de Ciclo 17

1.6 Arquitectura de Software y Desarrollo Basado en Componentes 17

1.6.1 Principios del Desarrollo Basado en Componentes: Reutilización, Encapsulamiento y Bajo Acoplamiento 17

1.6.2 Arquitectura de Microservicios y Separación de Responsabilidades 18

1.6.3 FastAPI (Python) como Framework para el Desarrollo del Backend 18

1.6.4 Comunicación entre Servicios mediante API RESTful y Mensajería Asíncrona 20

1.7 Persistencia con Bases de Datos NoSQL Orientadas a Documentos . 21

1.7.1 Fundamentos del Modelo de Datos Orientado a Documentos 21

1.7.2 MongoDB: Arquitectura y Características Principales 22

1.7.3 Modelado de Diagramas de Actividad como Estructuras de Grafo en Formato JSON 22

1.7.4 Almacenamiento de Definiciones de Formularios Dinámicos y sus Instancias 23

1.8 Tecnologías para la Capa de Presentación 23

1.8.1 Angular como Framework para Aplicaciones Web de Una Sola Página (SPA) 23

1.8.2 Flutter como Framework para Aplicaciones Móviles Multiplataforma 24

1.8.3 Principios de Diseño de Interfaces para Sistemas de Workflow 24

1.9 Motores de Workflow y Sistemas Basados en Reglas de Negocio 25

1.9.1 Conceptualización de Motores de Reglas de Negocio (Business Rule Engines) 25

1.9.2 Interpretación y Ejecución de Diagramas de Actividad UML 25

1.9.3 Mecanismos de Evaluación de Nodos de Decisión y Asignación Dinámica de Rutas 26

1.9.4 Trazabilidad del Estado de Trámites a lo Largo del Ciclo de Vida del Proceso 27

1.10 Sistemas de Gestión Documental (DMS) 27

1.10.1 Definición y Propósito 27

1.10.2 Componentes Esenciales de un DMS 27

1.10.3 Aplicación en el Proyecto 30

1.11 Deep Learning Aplicado al Proyecto 30

1.11.1 Fundamentos del Deep Learning 30

1.11.2 Técnicas Aplicadas en el Proyecto 31

1.11.3 Integración con FastAPI 33

1.12 Infraestructura Basada en AWS 33

1.12.1 Despliegue de Aplicaciones en la Nube 33

1.12.2 DynamoDB (Base de Datos NoSQL) 35

2 PROCESO DE DESARROLLO 38

2.1 FLUJO DE TRABAJO - CAPTURA DE REQUISITOS 38

2.1.1 Identificar actores y casos de uso 38

2.1.2 Priorización de Casos de Uso 39

2.1.3 Especificar Casos de Uso 40

2.1.4 Estructurar Modelos De Casos De Uso 76

2.2 FLUJO DE TRABAJO – ANÁLISIS 83

2.2.1 Análisis de Arquitectura 83

2.2.2 Análisis de Casos de Uso 95

2.3 FLUJO DE TRABAJO – DISEÑO 100

2.3.1 Diseño de Arquitectura 100

2.3.2 Diseño de Datos 104

2.3.3 Diagramas de secuencia 114

2.4 FLUJO DE TRABAJO – IMPLEMENTACIÓN 116

2.4.1 Elección de Plataforma de Desarrollo de Software 116

2.4.2 Estructura del Proyecto 121

2.4.3 Control de Versiones y CI/CD 129

2.4.4 Configuración de Entornos 131

2.4.5 Despliegue en AWS 131

2.4.6 Convenciones Transversales 132

2.4.7 Implementación de la Arquitectura del Sistema 133

2.4.8 Implementación de la Arquitectura del Subsistema 134

2.5 FLUJO DE TRABAJO – PRUEBAS 140

2.5.1 Unit Testing para DiagramsService 142

2.5.2 Unit Test para el UsersService 144

3 DOCUMENTAFCION PARA EL USO DEL SOFTWARE 148

4 Bibliografía 150

1 Fundamentación Teórica

1.1 El Proceso Unificado de Desarrollo de Software (PUDS)

1.1.1 Origen y Principios Rectores del Proceso Unificado

El Proceso Unificado de Desarrollo de Software, también conocido por sus siglas en inglés RUP (Rational Unified Process) o simplemente PUDS en la literatura hispanohablante, constituye uno de los marcos metodológicos más influyentes en la Ingeniería de Software contemporánea. Su origen se remonta a la década de 1990, cuando Ivar Jacobson, Grady Booch y James Rumbaugh —conocidos colectivamente como "Los Tres Amigos"— emprendieron la tarea de unificar sus respectivas metodologías: el Método Objectory de Jacobson, el Método Booch y la Técnica de Modelado de Objetos (OMT) de Rumbaugh. Esta convergencia dio origen al Lenguaje Unificado de Modelado (UML) y sentó las bases de un proceso integral que integrara las mejores prácticas de la industria.

El PUDS se fundamenta en seis principios rectores. El primero es el desarrollo iterativo e incremental, que organiza el esfuerzo en iteraciones temporales, cada una produciendo un incremento ejecutable del sistema. Frente a los modelos secuenciales como el modelo en cascada, el PUDS reconoce la complejidad e incertidumbre inherentes al software y permite la validación temprana y continua por parte de los interesados. El segundo principio es la gestión de requisitos, que proporciona un marco disciplinado para capturar, documentar, priorizar y gestionar cambios en los requisitos a lo largo del ciclo de vida. El tercer principio es el uso de arquitecturas basadas en componentes, que promueve la reutilización sistemática, la modularidad y el bajo acoplamiento. El cuarto principio es el modelado visual del software mediante UML, que proporciona un lenguaje común para todos los participantes del proyecto. El quinto principio es la verificación continua de la calidad, integrando actividades de

aseguramiento en cada iteración. El sexto principio es el control de cambios, que asegura la trazabilidad entre artefactos y facilita la colaboración.

1.1.2 Características Fundamentales: Dirigido por Casos de Uso, Centrado en la Arquitectura, Iterativo e Incremental

El PUDS se distingue por tres características interdependientes. Es dirigido por casos de uso, lo que significa que el desarrollo se organiza alrededor de las secuencias de interacciones entre actores externos y el sistema que producen un resultado observable de valor. Los casos de uso capturan la funcionalidad desde la perspectiva del usuario y guían todas las disciplinas del proceso: delimitan el alcance en la fase de Inicio, validan la arquitectura en la fase de Elaboración, priorizan el desarrollo en la fase de Construcción y proporcionan la base para las pruebas de aceptación en la fase de Transición.

Es centrado en la arquitectura, lo que implica que las decisiones de diseño más significativas se identifican y validan tempranamente mediante un esqueleto arquitectónico ejecutable. La arquitectura no es un artefacto estático, sino una entidad viva que evoluciona a lo largo del ciclo de vida, refinándose a medida que se implementan los casos de uso. Finalmente, es iterativo e incremental: cada iteración constituye un miniproyecto autónomo que abarca todas las disciplinas —desde la captura de requisitos hasta las pruebas— y produce un incremento ejecutable que añade funcionalidad a los incrementos anteriores. Esta aproximación permite la realimentación temprana de los usuarios, la mitigación progresiva de riesgos y la adaptación a requisitos cambiantes.

1.1.3 Las Cuatro Fases del Ciclo de Vida: Inicio, Elaboración, Construcción y Transición

El PUDS estructura el ciclo de vida en cuatro fases secuenciales, cada una concluida con un hito principal. La fase de Inicio tiene como objetivo establecer el alcance y la viabilidad del proyecto. Se identifican los actores y casos de uso principales, se delimita el alcance, se elabora una estimación preliminar de costes y plazos, y se evalúan los riesgos más significativos. El hito que marca su conclusión es el Hito de los Objetivos del Ciclo de Vida.

La fase de Elaboración es la más crítica desde el punto de vista técnico. Su objetivo es establecer y validar la arquitectura del sistema. Se detallan los casos de uso arquitectónicamente significativos, se diseña e implementa el esqueleto arquitectónico ejecutable, se seleccionan y evalúan las tecnologías, y se elabora un plan detallado para la fase de Construcción. El hito es el Hito de la Arquitectura del Ciclo de Vida.

La fase de Construcción es la de mayor duración y esfuerzo. Durante ella se implementa la totalidad de la funcionalidad del sistema mediante iteraciones sucesivas que producen incrementos ejecutables. Cada iteración incluye análisis, diseño, implementación y pruebas para un subconjunto de casos de uso priorizados. El hito es el Hito de la Capacidad Operacional Inicial.

La fase de Transición tiene como objetivo desplegar el sistema en el entorno de los usuarios finales. Incluye la corrección de defectos, la migración de datos, la formación de usuarios y la elaboración de documentación. El hito es el Hito del Lanzamiento del Producto.

1.1.4 Disciplinas y Flujos de Trabajo en PUDS

El PUDS organiza las actividades en un conjunto de disciplinas, también denominadas flujos de trabajo, que representan las áreas de especialización involucradas

en la construcción de software. A diferencia de las fases, que son secuenciales en el tiempo, las disciplinas se ejecutan de manera concurrente a lo largo de todo el ciclo de vida, aunque con diferente intensidad en cada fase.

Las disciplinas principales son: Modelado de Negocio, que busca comprender la estructura y dinámica de la organización donde se desplegará el sistema; Requisitos, centrada en la captura y documentación de los requisitos mediante el modelo de casos de uso; Análisis y Diseño, que traduce los requisitos en una especificación técnica; Implementación, que abarca la codificación, pruebas unitarias e integración; Prueba, que verifica la calidad del sistema; Despliegue, que pone el sistema a disposición de los usuarios; Gestión de Proyecto, que coordina la planificación, seguimiento y control; Gestión de Configuración y Cambios, que controla la evolución de los artefactos; y Entorno, que configura y mantiene las herramientas de desarrollo.

1.1.5 Artefactos y Entregables en Cada Fase del Proceso

El PUDS define un conjunto comprehensivo de artefactos que documentan los diferentes aspectos del sistema. En la fase de Inicio se producen el Documento de Visión, el Modelo de Casos de Uso de Alto Nivel, el Glosario del Dominio, el Plan de Proyecto Preliminar y el Caso de Negocio. En la fase de Elaboración se generan el Documento de Arquitectura de Software, el Modelo de Casos de Uso Detallado, el Modelo de Análisis y Diseño parcial, el Prototipo Arquitectónico Ejecutable y el Plan de Proyecto Refinado. En la fase de Construcción se completan el Modelo de Casos de Uso, el Modelo de Análisis y Diseño, el Código Fuente, los Casos de Prueba y el Manual de Usuario Preliminar. En la fase de Transición se entregan el Producto Software Desplegable, el Manual de Usuario Definitivo, los Materiales de Formación, el Plan de Despliegue y el Informe de Lecciones Aprendidas.

1.2 Lenguaje Unificado de Modelado (UML) 2.5 y Superior

1.2.1 UML como Estándar ISO/IEC 19501 para la Especificación de Sistemas

El Lenguaje Unificado de Modelado (UML) representa la culminación de décadas de evolución en el campo del modelado de sistemas de software. Su estandarización formal como ISO/IEC 19501 constituye un hito fundamental, ya que establece una notación común, semánticamente precisa y universalmente reconocida para la especificación, visualización, construcción y documentación de los artefactos de un sistema.

La génesis de UML se sitúa en la convergencia de las tres metodologías de modelado orientado a objetos predominantes en la década de 1990. La primera versión pública, UML 1.0, fue presentada en 1997 y adoptada por el Object Management Group (OMG) como estándar. Las versiones subsiguientes, culminando en UML 2.5, han expandido significativamente el alcance y la precisión semántica del lenguaje. UML no es meramente una notación gráfica, sino un lenguaje formal dotado de una sintaxis abstracta —definida mediante un metamodelo—, una sintaxis concreta —la notación gráfica— y una semántica que especifica el significado preciso de cada elemento. Esta fundamentación formal permite que los modelos UML puedan ser interpretados y procesados por herramientas automatizadas para generar código fuente, documentación y otros artefactos.

1.2.2 Clasificación de Diagramas UML: Estructurales vs. de Comportamiento

UML 2.5 define catorce tipos de diagramas organizados en dos grandes categorías. Los diagramas estructurales representan la organización estática de los elementos del sistema, independientemente del tiempo. Incluyen el Diagrama de Clases, que constituye el diagrama estructural por excelencia; el Diagrama de Objetos, que muestra una instantánea del sistema en un momento concreto; el Diagrama de Paquetes,

que organiza los elementos en agrupaciones lógicas; el Diagrama de Componentes, que representa la descomposición del sistema en unidades modulares reemplazables; el Diagrama de Estructura Compuesta, que modela la estructura interna de un clasificador; y el Diagrama de Despliegue, que representa la disposición física de los artefactos sobre los nodos hardware.

Los diagramas de comportamiento representan los aspectos dinámicos del sistema. Incluyen el Diagrama de Casos de Uso, que captura la funcionalidad desde la perspectiva de los actores; el Diagrama de Actividades, que modela flujos de control y datos; el Diagrama de Secuencia, que enfatiza el orden cronológico de los mensajes; el Diagrama de Comunicación, que enfatiza las relaciones estructurales entre objetos; el Diagrama de Máquina de Estados, que modela el comportamiento de una entidad a lo largo de su ciclo de vida; el Diagrama de Tiempos, que representa el cambio en el estado o valor de elementos a lo largo del tiempo; y el Diagrama de Vista de Interacción, que combina fragmentos de otros diagramas con construcciones de control de flujo.

1.2.3 Diagramas de Actividades: Sintaxis y Semántica para la Modelización de Flujos de Trabajo

El diagrama de actividades constituye uno de los diagramas de comportamiento más versátiles y expresivos de UML, particularmente adecuado para la modelización de procesos de negocio y flujos de trabajo. Su semántica se fundamenta en las redes de Petri y en los diagramas de flujo tradicionales, enriquecida con capacidades avanzadas para capturar la concurrencia, la sincronización y la asignación de responsabilidades.

Los nodos de control determinan el flujo de ejecución. El Nodo Inicial marca el punto de comienzo de la actividad. El Nodo Final de Actividad señala la terminación de la actividad en su totalidad; cuando un flujo de control lo alcanza, todas las acciones y

flujos concurrentes cesan inmediatamente. El Nodo Final de Flujo indica la terminación de un flujo específico sin afectar a otros flujos concurrentes. El Nodo de Decisión implementa una bifurcación condicional: cuando el flujo de control lo alcanza, las condiciones de guarda asociadas a los flujos salientes se evalúan y el flujo continúa por el primer flujo cuya condición sea verdadera. El Nodo de Bifurcación (Fork) divide un flujo único en múltiples flujos concurrentes que se ejecutan en paralelo. El Nodo de Unión (Join) sincroniza múltiples flujos concurrentes, combinándolos en un único flujo que continúa solo cuando todos los flujos entrantes han alcanzado el nodo.

Las particiones (swimlanes) organizan las acciones de acuerdo con la entidad responsable de su ejecución. Gráficamente se representan como bandas que dividen el diagrama en regiones adyacentes. Su uso clarifica la división de responsabilidades, pone de manifiesto las interacciones entre participantes y proporciona una base natural para la implementación de los flujos de trabajo.

Las regiones de expansión encapsulan un conjunto de acciones que se ejecutan múltiples veces sobre colecciones de elementos. UML 2.5 define tres modos: iterativo (las acciones se ejecutan secuencialmente para cada elemento), paralelo (las acciones se ejecutan concurrentemente para todos los elementos) y de flujo (los elementos se procesan a medida que están disponibles).

1.2.4 Diagramas de Secuencia y Comunicación: Especificación de Interacciones entre Actores

Los diagramas de interacción constituyen una familia de diagramas UML dedicados a modelar cómo los objetos o roles colaboran para realizar una funcionalidad específica mediante el intercambio de mensajes. El diagrama de secuencia representa una interacción como una secuencia cronológica de mensajes intercambiados entre líneas de vida. Cada línea de vida representa un participante y los mensajes se

representan mediante flechas horizontales, donde la posición vertical indica el orden cronológico. UML 2.x introdujo los fragmentos combinados, que permiten representar estructuras de control complejas como alternativas (alt), opciones (opt), bucles (loop), paralelismo (par) e interrupciones (break).

El diagrama de comunicación representa una interacción enfatizando la organización estructural de los objetos participantes. La dimensión temporal se indica mediante numeración secuencial de los mensajes, que puede ser jerárquica para representar anidamiento de llamadas. La elección entre ambos tipos de diagrama depende del aspecto que se desee enfatizar: si el foco es el orden cronológico detallado, se prefiere el diagrama de secuencia; si se desea resaltar las conexiones estructurales, se prefiere el diagrama de comunicación.

1.2.5 Diagramas de Máquina de Estados: Modelado del Ciclo de Vida de Entidades

El diagrama de máquina de estados constituye el instrumento fundamental en UML para modelar el comportamiento dinámico de una entidad a lo largo de su ciclo de vida en respuesta a eventos discretos. Un estado representa una situación en la vida de una entidad durante la cual se satisface alguna condición, se realiza alguna actividad o se espera la ocurrencia de algún evento. UML distingue tres tipos de acciones asociadas a un estado: entry (se ejecuta al entrar), exit (se ejecuta al salir) y do (se ejecuta de manera continua mientras se permanece en el estado).

Una transición representa el cambio de un estado origen a un estado destino, desencadenado por la ocurrencia de un evento, y puede incluir una condición de guarda que debe evaluarse como verdadera para que la transición se dispare, así como una acción que se ejecuta como consecuencia. UML soporta estados compuestos, que contienen una máquina de estados anidada, ya sea secuencial (un único estado activo en cada momento) u ortogonal (múltiples regiones concurrentes). Los pseudostados —

Inicial, de Elección, de Unión, de Bifurcación, de Historia y de Terminación— representan puntos de control transitorios.

1.2.6 Perfiles UML (UML Profiles) y Mecanismos de Extensión (Estereotipos, Valores Etiquetados)

UML fue diseñado como un lenguaje de propósito general, pero la práctica de la Ingeniería de Software a menudo requiere adaptarlo para expresar conceptos específicos de un dominio particular. Para ello, UML proporciona un mecanismo de extensión ligero denominado Perfil UML, que consiste en un conjunto de estereotipos, valores etiquetados y restricciones que especializan el metamodelo sin modificar su semántica fundamental.

Un estereotipo es una subclasificación de un elemento del metamodelo que le confiere un significado adicional específico del dominio. Gráficamente se representa mediante el nombre del estereotipo entre comillas francesas (ej. <<controlador>>, <<entidad>>). Un valor etiquetado es una propiedad adicional definida para un estereotipo que permite adjuntar información específica del dominio. Una restricción es una condición semántica que debe cumplirse para que el modelo sea considerado válido, expresable en lenguaje natural o en Object Constraint Language (OCL). La definición de Perfiles UML permite adaptar el lenguaje de modelado estándar a las necesidades específicas del dominio, capturando no solo la estructura y el comportamiento, sino también metadatos adicionales que pueden ser interpretados por herramientas automatizadas.

1.3 Patrones de Control de Flujo en Workflows

1.3.1 Fundamentos de los Sistemas de Gestión de Flujos de Trabajo (WfMS)

Los Sistemas de Gestión de Flujos de Trabajo (Workflow Management Systems, WfMS) constituyen una categoría especializada de sistemas de software diseñados para definir, ejecutar y monitorizar procesos de negocio estructurados como secuencias coordinadas de tareas. La Workflow Management Coalition (WfMC) define un sistema de workflow como "un sistema que define, crea y gestiona la ejecución de workflows mediante el uso de software, que es capaz de interpretar la definición del proceso, interactuar con los participantes del workflow y, cuando sea necesario, invocar el uso de herramientas y aplicaciones informáticas".

La evolución histórica de los WfMS refleja la creciente sofisticación en la automatización de procesos empresariales. Las primeras generaciones, surgidas en las décadas de 1980 y 1990, se centraban en procedimientos administrativos altamente estructurados, con definiciones de procesos embebidas en el código de la aplicación. La evolución hacia sistemas más flexibles ha sido impulsada por la creciente volatilidad de los entornos de negocio, la proliferación de arquitecturas orientadas a servicios y los avances en estándares de modelado de procesos como BPMN 2.0 y los diagramas de actividades UML

Un WfMS moderno se articula en torno a tres componentes arquitectónicos fundamentales: el Editor de Definiciones de Procesos, que proporciona las herramientas visuales para que los analistas definan la estructura y las reglas de los workflows; el Motor de Workflow, núcleo ejecutivo responsable de interpretar las definiciones, instanciar casos, enrutar tareas a los participantes y gestionar el estado de cada instancia; y la Interfaz de Usuario de Worklist, que presenta a cada participante las tareas pendientes y recopila la información necesaria para completarlas.

1.3.2 Análisis de Patrones de Control de Flujo (Workflow Patterns - Van Der Aalst)

El estudio sistemático de los patrones de control de flujo en workflows, iniciado por Wil van der Aalst y sus colaboradores a finales de la década de 1990, representa una de las contribuciones más significativas al campo de los sistemas de gestión de flujos de trabajo. Los patrones de workflow proporcionan un lenguaje común y un marco de referencia para describir, analizar y comparar las capacidades expresivas de diferentes sistemas y lenguajes de definición de procesos.

El patrón de secuencia constituye el mecanismo más fundamental: una actividad B solo puede comenzar después de que una actividad A haya completado su ejecución. Los patrones de divergencia y convergencia exclusiva (XOR-split y XOR-join) implementan la capacidad de seleccionar una ruta entre múltiples alternativas mutuamente excluyentes basándose en condiciones de guarda, y posteriormente reunificar los caminos alternativos. Los patrones de divergencia y convergencia paralela (AND-split y AND-join) permiten ejecutar múltiples actividades de manera concurrente y sincronizar sus flujos antes de continuar. Los patrones de repetición e iteración capturan la capacidad de ejecutar actividades repetidamente, ya sea un número predeterminado de veces, mientras se cumpla una condición, o hasta que se alcance un criterio de terminación.

1.3.3 Mecanismos de Enrutamiento Dinámico Basado en Reglas de Negocio

El enrutamiento dinámico constituye una capacidad avanzada de los sistemas de workflow que permite determinar el destinatario o la ruta de una tarea en tiempo de ejecución, basándose en reglas de negocio modificables sin alterar el código de la aplicación. Los mecanismos de enrutamiento dinámico pueden clasificarse en varias categorías: enrutamiento basado en atributos del proceso, que utiliza los valores de los

atributos asociados a la instancia (importe, tipo de producto, ubicación); enrutamiento basado en carga de trabajo y disponibilidad de recursos, que optimiza la utilización de los recursos humanos; enrutamiento basado en separación de deberes, que implementa controles de seguridad y cumplimiento normativo; y enrutamiento basado en escalado temporal, que gestiona situaciones donde una tarea no es completada dentro del plazo establecido.

Para implementar mecanismos sofisticados, los sistemas de workflow modernos a menudo integran motores de reglas de negocio especializados. La separación entre el motor de workflow —responsable de la orquestación del flujo de control— y el motor de reglas —responsable de la toma de decisiones basadas en políticas de negocio— constituye un ejemplo paradigmático del principio de diseño de separación de preocupaciones, permitiendo que las políticas de negocio sean modificadas por administradores sin intervención de desarrolladores.

1.4 Herramientas de Ingeniería de Software Asistida por Computadora (CASE)

1.4.1 Definición y Clasificación de Herramientas CASE

Las Herramientas de Ingeniería de Software Asistida por Computadora, conocidas por su acrónimo inglés CASE (Computer-Aided Software Engineering), constituyen un conjunto de aplicaciones informáticas diseñadas para automatizar, apoyar y optimizar las actividades del ciclo de vida del desarrollo de software. La premisa fundamental es que el desarrollo de software puede beneficiarse de la automatización de tareas repetitivas, la aplicación consistente de estándares metodológicos, la verificación automatizada de la corrección de modelos y la trazabilidad entre artefactos.

La clasificación tradicional distingue tres categorías: Upper-CASE, que proporciona soporte para las fases iniciales del ciclo de vida (planificación, análisis de requisitos, diseño de alto nivel); Lower-CASE, centrada en las fases posteriores (implementación, pruebas, mantenimiento); e Integrated-CASE (I-CASE) , que representa la convergencia de ambas en un entorno unificado con un repositorio centralizado de información.

1.4.2 Sparx Systems Enterprise Architect como Plataforma de Modelado Integral

Sparx Systems Enterprise Architect se ha consolidado como una de las plataformas de modelado visual y diseño de sistemas más comprehensivas del mercado. Su filosofía de diseño se fundamenta en el principio de que el modelado no debe ser una actividad aislada relegada a las fases iniciales, sino una práctica continua que permea todas las disciplinas del desarrollo. Para ello, proporciona un repositorio centralizado —basado en sistemas de gestión de bases de datos relacionales como SQL Server, Oracle o MySQL— que almacena todos los elementos del modelo y garantiza su accesibilidad simultánea por equipos distribuidos.

Enterprise Architect implementa exhaustivamente los estándares UML 2.5 y BPMN 2.0. Su soporte para diagramas de actividades UML incluye todos los constructores avanzados: nodos de bifurcación y unión para concurrencia, nodos de decisión para enrutamiento condicional, regiones de expansión para procesamiento iterativo y paralelo, y particiones para asignación de responsabilidades. Sus capacidades de modelado colaborativo incluyen un repositorio centralizado con bloqueos a nivel de elemento, control de versiones integrado con sistemas como Subversion y Git, y funcionalidades de discusiones y revisiones formales. El motor de simulación permite ejecutar y validar dinámicamente diagramas de actividades y de máquina de estados antes de que exista una línea de código, alineándose con la filosofía iterativa del PUDS.

El generador de documentación automatiza la producción de documentos en formatos RTF, PDF y HTML a partir del contenido del repositorio. Los mecanismos de extensibilidad —MDG Technologies, scripting y Perfiles UML personalizados— permiten adaptar la herramienta a las necesidades específicas de cada organización.

1.4.3 Interoperabilidad de Modelos mediante el Estándar XMI (XML Metadata Interchange)

El estándar XMI (XML Metadata Interchange), promulgado por el OMG, define un formato de intercambio basado en XML para modelos conformes al Meta-Object Facility (MOF), el metamodelo sobre el que se construye UML. XMI especifica cómo los elementos de un modelo UML deben serializarse como elementos XML y cómo las relaciones entre ellos deben representarse mediante referencias cruzadas. Enterprise Architect implementa XMI en varias versiones, permitiendo la exportación e importación de modelos entre diferentes herramientas.

1.4.4 Integración de Modelos con Entornos de Desarrollo (IDEs) y Repositorios de Código Fuente

Enterprise Architect facilita la transición del modelado a la implementación mediante integración con IDEs y sistemas de control de versiones. En la dirección "forward engineering", genera esqueletos de código fuente a partir de los modelos de diseño, adaptables mediante plantillas a las convenciones de frameworks como Spring Boot. En la dirección inversa, "reverse engineering", analiza código existente y genera modelos UML. La integración con sistemas de control de versiones permite gestionar los modelos con las mismas herramientas utilizadas para el código fuente.

1.5 Inteligencia Artificial en el Soporte a la Ingeniería de Software

1.5.1 Procesamiento de Lenguaje Natural (PLN) para la Interpretación de Especificaciones

El Procesamiento de Lenguaje Natural (PLN) constituye una subdisciplina de la Inteligencia Artificial que se ocupa de la interacción entre las computadoras y el lenguaje humano. Su aplicación al dominio de la Ingeniería de Software representa una de las fronteras más prometedoras de investigación.

Los Modelos de Lenguaje de Gran Escala (LLMs) representan un avance paradigmático en el PLN. Estos modelos, entrenados sobre corpus masivos de texto y basados en la arquitectura Transformer con mecanismos de atención, han demostrado capacidades notables en la comprensión de especificaciones técnicas y la generación de artefactos de diseño. En la generación asistida de diagramas UML, el ingeniero proporciona una especificación informal en lenguaje natural y el LLM genera una representación formal del diagrama. El proceso involucra la identificación de entidades, actores, acciones y relaciones, la abstracción para determinar el tipo de diagrama más apropiado y la generación de una representación en formato interpretable por herramientas de modelado.

El Reconocimiento Automático del Habla (ASR) permite transcribir el habla humana a texto escrito. Los sistemas ASR modernos, impulsados por arquitecturas de aprendizaje profundo, han alcanzado niveles de precisión que los hacen viables para la captura de requisitos de software. La integración de ASR con sistemas de modelado permite un flujo de trabajo más fluido, donde el analista puede verbalizar descripciones de procesos y el sistema genera los artefactos de modelado correspondientes.

1.5.2 Reconocimiento Óptico de Caracteres (OCR) en la Digitalización de Formularios Físicos

El Reconocimiento Óptico de Caracteres (OCR) permite convertir imágenes de texto en texto digital editable y procesable. Los avances recientes en aprendizaje profundo —redes neuronales convolucionales y redes neuronales recurrentes con mecanismos de atención— han mejorado dramáticamente su precisión y robustez. En el contexto de los sistemas de workflow, el OCR desempeña un papel crucial como puente entre los procesos basados en papel y los flujos de trabajo digitales, eliminando la necesidad de transcripción manual.

Un sistema OCR moderno orientado al procesamiento de formularios aborda la detección de la estructura del formulario, la extracción de pares clave-valor, la interpretación del contexto semántico y la integración con flujos de trabajo para inyectar la información extraída en el sistema.

1.5.3 Análisis de Rendimiento de Procesos mediante Simulación de Eventos Discretos

La simulación de eventos discretos es una técnica de modelado computacional que permite estudiar el comportamiento dinámico de un sistema a lo largo del tiempo mediante la representación de los cambios de estado que ocurren en instantes discretos como consecuencia de eventos. Aplicada al dominio de los procesos de negocio, proporciona un laboratorio virtual donde experimentar con diferentes configuraciones, evaluar el rendimiento bajo diversas condiciones de carga y predecir el impacto de cambios propuestos.

Los componentes fundamentales de un modelo de simulación incluyen: entidades (los elementos que fluyen a través del proceso), actividades (las tareas que consumen tiempo y recursos), recursos (los medios necesarios para ejecutar las

actividades), colas (los puntos de acumulación de entidades en espera) y rutas (el flujo a través de la red de actividades). El motor de simulación avanza el tiempo de evento en evento, recopilando estadísticas sobre métricas de rendimiento clave como tiempos de ciclo, tiempos de espera, utilización de recursos y rendimiento del proceso.

1.5.4 Identificación de Cuellos de Botella: Fundamentos de Minería de Procesos y Análisis de Tiempos de Ciclo

La minería de procesos se sitúa en la intersección de la ciencia de datos y la gestión de procesos de negocio. Su materia prima son los registros de eventos generados por los sistemas de información durante la ejecución de los procesos. Los tres tipos fundamentales de análisis son: descubrimiento de procesos, que infiere automáticamente un modelo de proceso a partir de los registros; verificación de conformidad, que identifica y cuantifica las desviaciones entre el comportamiento prescrito y el observado; y mejora de procesos, que utiliza la información extraída para identificar oportunidades de optimización.

El análisis de cuellos de botella es una aplicación específica de la minería de procesos. Un cuello de botella se define como una actividad o recurso cuya capacidad limitada restringe el flujo global del proceso. Los indicadores cuantitativos incluyen tiempo medio de espera elevado, alta utilización del recurso, longitud media de la cola significativamente superior y efecto de propagación de la congestión.

1.6 Arquitectura de Software y Desarrollo Basado en Componentes

1.6.1 Principios del Desarrollo Basado en Componentes: Reutilización, Encapsulamiento y Bajo Acoplamiento

El Desarrollo Basado en Componentes (DBC) constituye un paradigma de construcción de software que enfatiza el ensamblaje de sistemas a partir de unidades

software preexistentes, autocontenidas y reemplazables denominadas componentes. Se fundamenta en tres principios arquitectónicos interdependientes: la reutilización, que persigue maximizar el aprovechamiento de activos de software existentes; el encapsulamiento, que oculta los detalles internos de implementación tras interfaces bien definidas; y el bajo acoplamiento, que minimiza el grado de interdependencia entre componentes para facilitar la evolución independiente.

1.6.2 Arquitectura de Microservicios y Separación de Responsabilidades

La arquitectura de microservicios propugna la construcción de sistemas como un conjunto de servicios pequeños, autónomos y débilmente acoplados, cada uno implementando una capacidad de negocio específica y pudiendo ser desarrollado, desplegado y escalado independientemente. Sus características definitorias incluyen: autonomía (cada microservicio gestiona su propio ciclo de vida y almacenamiento de datos), especialización (cada uno se centra en una única capacidad de negocio), despliegue independiente (sin coordinación con otros servicios), comunicación a través de la red (exclusivamente mediante protocolos de red) y gestión de datos descentralizada (sin compartir bases de datos).

En el contexto de un sistema de gestión de políticas de negocio y workflow, una arquitectura de microservicios permite separar responsabilidades en servicios especializados: un servicio de definición de políticas, un servicio de motor de workflow, un servicio de formularios dinámicos, un servicio de IA, un servicio de gestión documental y un servicio de notificaciones, cada uno desarrollado con el framework más adecuado.

1.6.3 FastAPI (Python) como Framework para el Desarrollo del Backend

FastAPI es un framework web moderno para construir APIs con Python que ha experimentado una adopción meteórica desde su lanzamiento. Su propuesta de valor

combina un rendimiento excepcional —comparable al de frameworks Node.js o Go— con una facilidad de uso y productividad de desarrollo que lo han convertido en la opción predilecta para servicios de backend en el ecosistema Python, particularmente en dominios relacionados con la ciencia de datos y la inteligencia artificial.

FastAPI se construye sobre Starlette, un framework ligero de bajo nivel para servicios web asíncronos, y utiliza el protocolo ASGI (Asynchronous Server Gateway Interface) como interfaz entre el servidor web y la aplicación Python. ASGI es la evolución asíncrona del tradicional WSGI, diseñado para soportar operaciones de entrada/salida no bloqueantes, WebSockets y otros protocolos de larga duración.

La naturaleza asíncrona de FastAPI se alinea perfectamente con los patrones de trabajo típicos en servicios de inteligencia artificial y sistemas de workflow. Las operaciones de inferencia de modelos de machine learning, aunque computacionalmente intensivas, a menudo implican tiempos de espera significativos mientras el modelo procesa la entrada y genera la salida. De manera similar, las llamadas a APIs externas —como servicios OCR en la nube, APIs de modelos de lenguaje de gran escala, o servicios de almacenamiento como Amazon S3— están dominadas por la latencia de red. FastAPI permite que estas operaciones sean definidas como funciones asíncronas utilizando las palabras clave async y await de Python, liberando el hilo de ejecución para atender otras peticiones concurrentes.

FastAPI integra Pydantic para la validación automática de datos mediante las anotaciones de tipo de Python. Los modelos de datos se definen como clases de Pydantic que heredan de BaseModel. Cuando una petición HTTP llega a un endpoint, el framework automáticamente analiza el cuerpo de la petición, valida que los campos requeridos estén presentes y que sus tipos coincidan con los declarados, convierte los valores a los tipos Python apropiados y, si la validación falla, genera una respuesta de

error HTTP 422 con detalles precisos sobre qué campos fallaron y por qué. Este mecanismo de validación automática elimina una cantidad sustancial de código repetitivo de validación manual, reduce la superficie de errores y garantiza que el código de negocio opera siempre sobre datos validados y correctamente tipados.

Otra característica destacada de FastAPI es la generación automática de documentación interactiva de la API en formato OpenAPI (anteriormente conocido como Swagger). Esta documentación está disponible a través de dos interfaces web integradas: Swagger UI, que proporciona una interfaz interactiva para explorar y probar los endpoints, y ReDoc, que ofrece una vista de documentación más orientada a la lectura. La generación automática de documentación OpenAPI es particularmente valiosa en el contexto de una arquitectura de microservicios, donde múltiples servicios desarrollados por diferentes equipos necesitan interoperar. La disponibilidad de especificaciones OpenAPI precisas y siempre actualizadas para cada servicio facilita la generación automática de clientes de API en diversos lenguajes, la configuración de puertas de enlace API y la validación de contratos entre servicios.

Para la interacción con bases de datos MongoDB, FastAPI se integra de manera natural con Motor, un driver asíncrono para MongoDB, o con ODMappers como Beanie, que proporciona una capa de abstracción adicional con validación de esquemas integrada mediante Pydantic. Esta integración permite que las operaciones de base de datos se ejecuten de manera no bloqueante, maximizando el rendimiento del servidor.

1.6.4 Comunicación entre Servicios mediante API RESTful y Mensajería Asíncrona

La comunicación entre los componentes de un sistema distribuido condiciona profundamente sus propiedades no funcionales. El estilo arquitectónico REST (Representational State Transfer) se ha convertido en el estándar de facto para APIs sobre HTTP, basándose en la identificación de recursos mediante URIs, una interfaz

uniforme basada en métodos HTTP, la ausencia de estado en el servidor y la negociación de representaciones múltiples.

La mensajería asíncrona, por su parte, desacopla temporalmente a los servicios emisores y receptores mediante un intermediario (broker de mensajes como RabbitMQ, Apache Kafka o Amazon SQS), proporcionando resiliencia mejorada, mejor utilización de recursos y escalabilidad elástica. En un sistema de workflow, ambos estilos coexisten: las interacciones iniciadas por el usuario se modelan como operaciones RESTful síncronas, mientras que las interacciones entre servicios internos —como la notificación al motor de workflow de que una tarea de IA ha completado el procesamiento— se benefician de la mensajería asíncrona.

1.7 Persistencia con Bases de Datos NoSQL Orientadas a Documentos

1.7.1 Fundamentos del Modelo de Datos Orientado a Documentos

El modelo de datos orientado a documentos organiza los datos como colecciones de documentos autocontenidos, típicamente en formatos como JSON o BSON. A diferencia del modelo relacional, que dispersa la información relacionada a través de múltiples tablas, un documento encapsula toda la información relevante de una entidad en un único objeto jerárquico. Este enfoque se rige por el principio de localidad de los datos: la información accedida conjuntamente se almacena conjuntamente, eliminando la necesidad de costosas operaciones de unión en tiempo de consulta.

El modelo documental abraza el principio de esquema flexible o "schema-on-read": la base de datos no impone un esquema uniforme a todos los documentos de una colección, trasladando la responsabilidad de interpretar la estructura a la aplicación. Esta flexibilidad facilita la evolución ágil del modelo de datos sin migraciones de esquema y permite modelar variantes de entidades con diferentes conjuntos de campos.

1.7.2 MongoDB: Arquitectura y Características Principales

MongoDB se ha erigido como la base de datos orientada a documentos más ampliamente adoptada. Su arquitectura se articula en torno a tres componentes: mongod, el demonio primario responsable de gestionar peticiones de datos; mongos, un enrutador de consultas en despliegues con sharding; y los Servidores de Configuración, que almacenan los metadatos del clúster. MongoDB proporciona dos mecanismos fundamentales para garantizar disponibilidad y durabilidad: la replicación mediante Replica Sets (un grupo de instancias que mantienen copias redundantes con elección automática de primario) y el sharding para escalabilidad horizontal (distribución de datos a través de múltiples servidores).

Los documentos se almacenan en formato BSON (Binary JSON), que extiende JSON con tipos adicionales como fechas, datos binarios y tipos numéricos de precisión arbitraria. Las colecciones agrupan documentos sin imponer un esquema uniforme. MongoDB soporta una amplia variedad de tipos de índices: sobre campos individuales, compuestos, multikey (para arrays), geoespaciales, de texto y TTL (Time-To-Live).

1.7.3 Modelado de Diagramas de Actividad como Estructuras de Grafo en Formato JSON

Los diagramas de actividad UML son inherentemente estructuras de grafo compuestas por nodos (acciones, decisiones, bifurcaciones, uniones) y aristas dirigidas (flujos de control). La estrategia de modelado más directa en MongoDB consiste en almacenar el diagrama completo como un único documento BSON, donde los nodos se representan como elementos de un array nodos y las aristas como elementos de un array flujos. Este enfoque de documento único es óptimo cuando las operaciones predominantes son la lectura y escritura del diagrama completo, ya que una única operación de base de datos recupera toda la definición del proceso.

1.7.4 Almacenamiento de Definiciones de Formularios Dinámicos y sus Instancias

La gestión de formularios dinámicos —cuya estructura puede ser definida y modificada por administradores sin intervención de desarrolladores— constituye un caso de uso paradigmático para MongoDB. Las definiciones de formularios especifican arrays de campos con tipos, etiquetas, reglas de validación y valores por defecto. Las instancias de formularios referencian a la definición que les da estructura y contienen los valores proporcionados por el usuario. Esta separación permite que múltiples versiones de un mismo formulario coexistan en el tiempo.

1.8 Tecnologías para la Capa de Presentación

1.8.1 Angular como Framework para Aplicaciones Web de Una Sola Página (SPA)

Angular es un framework de desarrollo de aplicaciones web mantenido por Google, construido sobre TypeScript. Su pilar arquitectónico fundamental es el componente, una clase TypeScript que encapsula la lógica de presentación, el estado local y la plantilla HTML asociada para una porción reutilizable de la interfaz de usuario. Los componentes se organizan en una jerarquía de árbol, creando una estructura modular y composable.

Angular implementa un sofisticado sistema de enlace de datos que sincroniza automáticamente el estado del modelo con la vista. Soporta interpolación, enlace de propiedades, enlace de eventos y enlace bidireccional. La reactividad se fundamenta en RxJS (Reactive Extensions for JavaScript), que implementa el paradigma de programación reactiva mediante Observables. Para formularios complejos, Angular proporciona formularios reactivos, que se construyen programáticamente en la clase del componente, permitiendo validación dinámica, dependencias entre campos y renderizado a partir de definiciones JSON recibidas del backend. Esta capacidad es

particularmente relevante para formularios dinámicos cuya estructura no se conoce en tiempo de compilación.

1.8.2 Flutter como Framework para Aplicaciones Móviles Multiplataforma

Flutter es un framework de desarrollo de aplicaciones multiplataforma creado por Google. Su arquitectura difiere radicalmente de otros frameworks: incluye su propio motor de renderizado de alto rendimiento, Skia, que dibuja cada píxel de la interfaz directamente sobre el lienzo proporcionado por el sistema operativo, garantizando una apariencia y comportamiento idénticos en todos los dispositivos.

La filosofía central de Flutter es "Everything is a Widget": un Widget es una descripción inmutable de una parte de la interfaz de usuario. Los Widgets se organizan en una jerarquía de composición, promoviendo la reutilización y la legibilidad. Para la gestión del estado, el patrón BLoC (Business Logic Component) implementa una variante de Flux/Redux que establece una separación estricta entre la capa de presentación y la capa de lógica de negocio, con un flujo unidireccional de datos. Flutter proporciona bibliotecas para comunicación HTTP (http, dio) y WebSockets (web_socket_channel), permitiendo comunicación síncrona y bidireccional en tiempo real con el backend.

1.8.3 Principios de Diseño de Interfaces para Sistemas de Workflow

El diseño de interfaces para sistemas de workflow presenta desafíos específicos. Un principio fundamental es la visualización clara del estado y el progreso: el usuario debe comprender inmediatamente en qué punto del proceso se encuentra cada trámite. La reducción de la carga cognitiva en formularios complejos se logra mediante la descomposición en pasos lógicos, la agrupación visual de campos relacionados y la validación en tiempo real. La adaptación a múltiples dispositivos mediante diseño

responsivo es esencial para funcionarios en oficina (escritorio) y clientes en movilidad (móvil). Las notificaciones y alertas contextuales mantienen a los usuarios informados sobre tareas pendientes, plazos próximos y eventos relevantes.

1.9 Motores de Workflow y Sistemas Basados en Reglas de Negocio

1.9.1 Conceptualización de Motores de Reglas de Negocio (Business Rule Engines)

Un Motor de Reglas de Negocio (Business Rule Engine, BRE) es un componente de software diseñado para ejecutar reglas de negocio en un entorno de producción. Su propósito fundamental es externalizar la lógica de decisión del código fuente de la aplicación, alojándola en un repositorio gestionable por analistas de negocio. La arquitectura conceptual se articula en tres componentes: el Repositorio de Reglas, que almacena las definiciones en formatos expresivos; el Motor de Inferencia, que aplica las reglas a los hechos proporcionados mediante algoritmos de emparejamiento de patrones como el algoritmo Rete; y la Memoria de Trabajo, que contiene los hechos que representan el estado actual de la decisión.

El ciclo de ejecución típico sigue el patrón "Match-Resolve-Act": en la fase Match, se evalúan las condiciones de todas las reglas contra los hechos; en la fase Resolve, se selecciona una regla mediante estrategias de resolución de conflictos; en la fase Act, se ejecutan las acciones de la regla seleccionada. La externalización de las reglas confiere agilidad de negocio, transparencia, consistencia y auditabilidad.

1.9.2 Interpretación y Ejecución de Diagramas de Actividad UML

La ejecución automatizada de diagramas de actividad UML por un motor de workflow requiere un proceso de interpretación que traduzca la representación gráfica en comportamiento dinámico. Cada tipo de nodo tiene una semántica de ejecución bien definida: el Nodo Inicial coloca un token de control que se transfiere inmediatamente; la

Acción representa una tarea que puede ser automática o requerir intervención humana; la Decisión evalúa condiciones de guarda y selecciona una rama; la Bifurcación genera tokens independientes para cada flujo saliente; la Unión sincroniza todos los flujos entrantes antes de continuar; el Nodo Final de Actividad termina la instancia en su totalidad; y el Nodo Final de Flujo termina un hilo específico sin afectar a otros.

El motor de workflow debe gestionar el contexto de datos de la instancia —variables del proceso y datos de formularios—, el manejo de excepciones y errores mediante flujos de excepción predefinidos, y la persistencia del estado de ejecución para permitir que instancias de larga duración sobrevivan a reinicios del servidor.

1.9.3 Mecanismos de Evaluación de Nodos de Decisión y Asignación Dinámica de Rutas

Las condiciones de guarda en nodos de decisión se expresan como expresiones booleanas que referencian los datos del contexto de la instancia. El motor de workflow debe proporcionar un evaluador de expresiones que interprete estas condiciones en tiempo de ejecución, sustituyendo las referencias a variables por sus valores actuales. Para decisiones complejas, el nodo de decisión puede delegar la evaluación en un motor de reglas externo, desacoplando completamente la lógica de decisión del motor de workflow.

La asignación dinámica de destinatarios para tareas humanas puede basarse en roles organizacionales, asignación directa a usuarios específicos, reglas de carga de trabajo o restricciones de separación de deberes, todas ellas configurables por los administradores como parte de la definición del proceso.

1.9.4 Trazabilidad del Estado de Trámites a lo Largo del Ciclo de Vida del Proceso

La trazabilidad —la capacidad de reconstruir la historia completa de una instancia de proceso— es un requisito fundamental por consideraciones operativas, normativas y de auditoría. El motor de workflow debe registrar, para cada instancia, una secuencia cronológica de eventos que documenten su evolución, incluyendo marca de tiempo, tipo de evento, identificador de la instancia, usuario ejecutor y contexto relevante. Este registro histórico constituye la materia prima para el seguimiento en tiempo real, el análisis de rendimiento de procesos, la minería de procesos y la auditoría de cumplimiento normativo.

1.10 Sistemas de Gestión Documental (DMS)

1.10.1 Definición y Propósito

Un Sistema de Gestión Documental (Document Management System, DMS) es una solución de software diseñada para almacenar, administrar, controlar y rastrear documentos electrónicos a lo largo de su ciclo de vida completo. Su propósito fundamental es transformar la gestión de documentos físicos en un flujo de trabajo digital, proporcionando un repositorio centralizado que garantice la integridad, disponibilidad, confidencialidad y trazabilidad de la información institucional.

1.10.2 Componentes Esenciales de un DMS

Ciclo de Vida Documental: El DMS gestiona las fases por las que transita un documento desde su creación o captura hasta su disposición final:

Creación o Captura: Ingreso del documento al sistema, ya sea mediante creación directa (formularios electrónicos), digitalización (escaneo con OCR) o importación desde fuentes externas (correo electrónico, otras aplicaciones).

- **Almacenamiento:** Depósito del archivo físico en un repositorio seguro (sistema de archivos, almacenamiento en la nube como Amazon S3) y registro de sus metadatos asociados en una base de datos (MongoDB).
- **Clasificación e Indexación:** Asignación de metadatos (tipo documental, autor, fecha, trámite asociado, palabras clave) y organización en estructuras lógicas (carpetas, categorías) para facilitar su recuperación.
- **Acceso y Distribución:** Mecanismos controlados de consulta, visualización, descarga y compartición, gobernados por permisos granulares basados en roles y perfiles de usuario.
- **Colaboración y Revisión:** Funcionalidades que permiten a múltiples usuarios trabajar simultáneamente sobre un documento, registrar comentarios, realizar anotaciones y gestionar flujos de aprobación.
- **Control de Versiones:** Registro automático de cada modificación realizada sobre un documento, preservando el historial completo de cambios. Cada versión se identifica unívocamente, permitiendo la consulta, comparación y restauración de versiones anteriores.
- **Retención y Disposición:** Aplicación de políticas de retención documental que determinan cuánto tiempo debe conservarse un documento antes de ser archivado o eliminado, en cumplimiento de normativas legales y fiscales.
- **Auditoría y Trazabilidad:** Registro inmutable de cada acción realizada sobre un documento (creación, visualización, modificación, eliminación, firma), incluyendo usuario, fecha, hora y tipo de acción.

Metadatos: Son datos estructurados que describen el contexto, contenido y estructura de un documento. Constituyen el núcleo de la capacidad de búsqueda y organización de un DMS. Se clasifican en:

- **Metadatos Descriptivos:** Título, autor, tipo documental, palabras clave, resumen.
- **Metadatos Administrativos:** Fecha de creación, fecha de modificación, permisos de acceso, política de retención.
- **Metadatos Estructurales:** Relación con otros documentos, posición en una jerarquía de carpetas, asociación con un trámite o actividad específica.

Control de Versiones: Mecanismo que permite gestionar las sucesivas modificaciones de un documento. Cada vez que se edita y guarda un archivo, el sistema crea una nueva versión, manteniendo las anteriores intactas. Las funcionalidades asociadas incluyen:

- **Versionado Automático:** El sistema incrementa el número de versión (1.0, 1.1, 2.0) con cada cambio.
- **Historial de Versiones:** Visualización cronológica de todas las versiones con sus metadatos (autor, fecha, comentario).
- **Comparación de Versiones:** Funcionalidad para visualizar las diferencias entre dos versiones.
- **Restauración:** Capacidad de revertir el documento a una versión anterior.
- **Bloqueo (Check-out / Check-in):** Mecanismo para evitar ediciones simultáneas conflictivas.

Firma Electrónica: Técnica criptográfica que permite validar la autenticidad e integridad de un documento digital. Se basa en el uso de certificados digitales y funciones hash. Los aspectos fundamentales incluyen:

- **Hash del Documento:** Se genera un resumen criptográfico (SHA-256) del contenido del archivo. Cualquier modificación posterior alterará el hash, invalidando la firma.

- **Firma Digital:** El hash se cifra con la clave privada del firmante, generando la firma. La verificación se realiza descifrando la firma con la clave pública y comparando el hash resultante con el hash actual del documento.
- **No Repudio:** La firma electrónica vincula de forma única al firmante con el documento, impidiendo que niegue haberlo firmado.
- **Sello de Tiempo:** Asocia una fecha y hora cierta a la firma, proporcionada por una autoridad de sellado de tiempo (TSA).

1.10.3 Aplicación en el Proyecto

En el contexto del sistema integral, la gestión documental se implementa mediante:

- **FastAPI** como backend para la lógica de negocio y la API REST.
- **MongoDB** para almacenar los metadatos de documentos, versiones, firmas y registros de auditoría.
- **Amazon S3** como repositorio físico de los archivos, proporcionando almacenamiento duradero, escalable y seguro.
- **TensorFlow** para el procesamiento OCR de documentos escaneados (extracción de texto) y la clasificación automática de tipos documentales.
- **WebSockets** para la colaboración en tiempo real sobre documentos compartidos.

1.11 Deep Learning Aplicado al Proyecto

1.11.1 Fundamentos del Deep Learning

El Deep Learning (aprendizaje profundo) es una rama del Machine Learning basada en redes neuronales artificiales con múltiples capas (profundas) que aprenden

representaciones jerárquicas de los datos. Cada capa sucesiva aprende características cada vez más abstractas, permitiendo modelar relaciones complejas no lineales.

1.11.2 Técnicas Aplicadas en el Proyecto

Análisis de Texto y NLP (Procesamiento de Lenguaje Natural):

- **Tokenización y Vectorización:** Conversión del texto de las solicitudes en secuencias de tokens y posteriormente en vectores numéricos (embeddings) que capturan el significado semántico. Técnicas como Word2Vec, GloVe o embeddings contextuales (BERT) son utilizadas.
- **Modelos de Clasificación de Texto:** Redes neuronales recurrentes (LSTM, GRU) o Transformers entrenados para asignar una solicitud de trámite a una categoría predefinida (la política de negocio correspondiente). Capturan la intención del usuario a partir de la descripción textual.
- **Reconocimiento de Entidades Nombradas (NER):** Modelos que identifican y extraen información estructurada del texto libre: nombres de personas, fechas, direcciones, montos, tipos de trámite. Esta información se utiliza para pre-rellenar campos de formularios.
- **ASR (Reconocimiento Automático del Habla):** Modelos de Deep Learning (basados en arquitecturas Transformer como Whisper) que transcriben el audio de las solicitudes habladas a texto, para su posterior procesamiento por el pipeline de NLP.

TensorFlow como Framework:

TensorFlow es un framework de código abierto desarrollado por Google para construir, entrenar y desplegar modelos de Machine Learning y Deep Learning. Se caracteriza por:

- **Cálculo basado en Grafos:** Define un grafo computacional donde los nodos representan operaciones matemáticas y las aristas los tensores (arrays multidimensionales) que fluyen entre ellas.
- **Keras API:** Interfaz de alto nivel integrada en TensorFlow que simplifica la definición, compilación, entrenamiento y evaluación de modelos.
- **TensorFlow Serving:** Sistema de despliegue para modelos entrenados, permitiendo exponerlos como servicios de inferencia de alta performance vía gRPC o REST.
- **TensorFlow Lite:** Para el despliegue de modelos en dispositivos móviles (Flutter) o edge computing.

Aplicaciones Específicas:

- **Clasificación de Solicitudes (CU-29):** Modelo de clasificación de texto (LSTM o Transformer) entrenado con datos históricos de solicitudes etiquetadas con la política de negocio correspondiente. Recibe el texto de la solicitud y predice la política más probable.
- **Llenado Automático de Formularios (CU-30):** Modelo NER que identifica entidades en el texto y las mapea a los campos del formulario dinámico.
- **Predicción de Riesgo de Demora (CU-33):** Modelo de series temporales o de supervivencia entrenado con datos históricos de trámites (tiempos de ejecución, carga de funcionarios, documentos pendientes). Predice la probabilidad de que un trámite exceda su plazo.
- **Recomendación de Ruta Óptima (CU-34):** Modelo de recomendación basado en aprendizaje por refuerzo o en filtrado colaborativo, que sugiere la asignación de tareas a los funcionarios con mejor desempeño histórico para minimizar tiempos de ciclo.

- **Detección de Anomalías (CU-35):** Modelos no supervisados (Autoencoders, Isolation Forest) que aprenden el comportamiento "normal" de un trámite y detectan desviaciones significativas (tiempos excesivos, demasiadas versiones de documentos, devoluciones inusuales).

1.11.3 Integración con FastAPI

Los modelos de TensorFlow se entrenan offline y se serializan en formato SavedModel. En el backend FastAPI, se cargan en memoria al iniciar la aplicación y se exponen endpoints de inferencia. La comunicación entre FastAPI y TensorFlow puede ser directa (inferencia en el mismo proceso) o mediante TensorFlow Serving como microservicio independiente para mayor escalabilidad.

1.12 Infraestructura Basada en AWS

1.12.1 Despliegue de Aplicaciones en la Nube

Para el despliegue del sistema se utilizarán los siguientes servicios de AWS:

Opción 1: Amazon EC2 + Docker

- **EC2 (Elastic Compute Cloud):** Instancias de servidores virtuales escalables. Se despliegan contenedores Docker con el backend FastAPI y el frontend Angular.
- **ECS (Elastic Container Service) o EKS (Elastic Kubernetes Service):** Alternativas gestionadas para orquestación de contenedores, recomendables para entornos de producción con múltiples microservicios.
- **ECR (Elastic Container Registry):** Repositorio privado para almacenar las imágenes Docker.
- **ALB (Application Load Balancer):** Distribuye el tráfico entrante entre múltiples instancias, proporcionando alta disponibilidad.

Opción 2: AWS Elastic Beanstalk (PaaS)

- Plataforma como servicio que abstrae la gestión de infraestructura. Permite desplegar aplicaciones web (FastAPI) con un simple comando, gestionando automáticamente el balanceo de carga, escalado y monitorización.

Opción 3: Railway (Recomendada para MVP)

- Plataforma de despliegue moderna con integración directa con GitHub. Soporta despliegues automáticos, gestión de variables de entorno, y conexión sencilla con bases de datos y almacenamiento de objetos.

- **1.12.2. Amazon S3 (Simple Storage Service)**

Características Fundamentales:

- **Almacenamiento de Objetos:** Almacena datos como objetos dentro de buckets. Cada objeto se identifica por una clave única.
- **Durabilidad del 99.999999999% (11 nueves):** Los datos se replican automáticamente en múltiples zonas de disponibilidad.
- **Escalabilidad Ilimitada:** Soporta volúmenes de datos y solicitudes masivas sin degradación del rendimiento.
- **Clases de Almacenamiento:** S3 Standard (acceso frecuente), S3 Intelligent-Tiering (optimización automática), S3 Standard-IA (acceso infrecuente), S3 Glacier (archivo a largo plazo).
- **Control de Acceso:** Políticas IAM, ACLs y URLs pre-firmadas para acceso temporal y seguro.
- **Versionado:** Funcionalidad que mantiene múltiples variantes de un objeto en el mismo bucket. Al sobrescribir o eliminar un objeto, se conservan las versiones anteriores.

- **Ciclo de Vida:** Políticas que automatizan la transición de objetos entre clases de almacenamiento o su eliminación tras un período definido.
- **Cifrado:** Soporta cifrado en reposo (SSE-S3, SSE-KMS) y en tránsito (TLS).

Aplicación en el Proyecto:

- Los archivos físicos de los documentos se almacenan en un bucket S3 con versionado habilitado.
- Se generan URLs pre-firmadas (con tiempo de expiración) para que el frontend Angular pueda visualizar o descargar documentos de forma segura sin exponer el bucket públicamente.
- Las versiones de documentos se mapean directamente al versionado nativo de S3: cada nueva versión de un documento en el sistema corresponde a una nueva versión del objeto en S3.
- Políticas de ciclo de vida archivarán automáticamente documentos antiguos a S3 Glacier para reducir costos.

1.12.2 DynamoDB (Base de Datos NoSQL)

Características Fundamentales:

- **Totalmente Gestionada:** Sin servidores que administrar. Escalado automático en función del tráfico.
- **Modelo Clave-Valor y Documentos:** Almacena datos como items con una clave primaria (clave de partición y opcionalmente clave de ordenación).
- **Alta Disponibilidad y Durabilidad:** Datos replicados automáticamente en múltiples zonas de disponibilidad dentro de una región. Tablas globales para replicación multi-región.
- **Rendimiento Consistente:** Latencia de respuesta en milisegundos, incluso bajo cargas masivas.

- **Modelo de Consistencia Configurable:** Lecturas eventualmente consistentes (por defecto, menor latencia) o fuertemente consistentes.
- **Capacidad bajo Demanda o Provisionada:** Modo on-demand (pago por uso) o provisioned (especificar lecturas/escrituras por segundo).
- **Streams:** Capturan cambios en los items (inserciones, actualizaciones, eliminaciones) para integración con otros servicios (Lambda, Kinesis).

Comparación y Complementariedad con MongoDB:

En este proyecto, **MongoDB** sigue siendo la base de datos principal debido a su esquema flexible orientado a documentos JSON/BSON, que es ideal para las definiciones dinámicas de diagramas de actividades y formularios. Sin embargo, **DynamoDB** se propone como una alternativa o complemento para componentes específicos que requieren:

Característica	MongoDB (Principal)	DynamoDB (Complementario)
Modelo de Datos	Orientado a documentos (JSON/BSON)	Clave-valor y documentos (JSON)
Consultas	Lenguaje de consulta rico (MQL), agregaciones complejas	Operaciones simples por clave. Query por índice secundario
Escalado	Manual (sharding) o gestionado (Atlas)	Automático y transparente
Rendimiento	Muy bueno con índices adecuados	Latencia de milisegundos predecible a cualquier escala
Casos de Uso en el Proyecto	Políticas, workflow, metadatos documentales, auditoría	Notificaciones en tiempo real, sesiones de usuario, colas de mensajería, tracking de eventos de workflow

Uso Propuesto de DynamoDB en el Proyecto:

- **Notificaciones en Tiempo Real:** Almacenar notificaciones push pendientes de envío, con TTL para expiración automática.
- **Sesiones de Usuario y Tokens JWT:** Almacenar tokens de refresco o listas negras de tokens, aprovechando la baja latencia.

- **Eventos de Workflow (Streaming):** Registrar eventos de avance de trámites para alimentar dashboards en tiempo real o funciones Lambda reactivas.
- **Bloqueos Distribuidos:** Para la edición colaborativa de diagramas, utilizando la consistencia fuerte de DynamoDB.

2 PROCESO DE DESARROLLO

2.1 FLUJO DE TRABAJO - CAPTURA DE REQUISITOS

2.1.1 Identificar actores y casos de uso

2.1.1.1 Actores

ID	Actor	Descripción
A1	Administrador	Gestiona usuarios, roles, permisos, departamentos, políticas de negocio, formularios, reglas, reportes y configuración general del sistema.
A2	Diseñador de Procesos	Crea y modifica diagramas de actividades, define responsables, configura formularios y valida reglas de negocio.
A3	Funcionario	Atiende tareas asignadas, llena formularios, carga documentos, revisa información y deriva trámites.
A4	Supervisor	Aprueba actividades, revisa documentos, firma electrónicamente y controla el avance del workflow.
A5	Auditor	Consulta historial de cambios, versiones documentales, acciones de usuarios, firmas y trazabilidad del trámite.
A6	Cliente o Solicitante	Inicia solicitudes, carga documentos, consulta el estado del trámite y recibe notificaciones.
A7	Sistema de IA	Analiza texto o audio, clasifica solicitudes, llena formularios, genera reportes, predice demoras y recomienda rutas.
A8	Servicio S3	Almacena documentos físicos como PDF, Word, imágenes y archivos firmados.

2.1.1.2 Casos de Uso

ID	Nombre	Entorno
CU-01	Iniciar sesión	WEB, MÓVIL
CU-02	Cerrar sesión	WEB, MÓVIL
CU-03	Gestionar usuarios	WEB
CU-04	Gestionar roles y permisos	WEB
CU-05	Gestionar departamentos	WEB
CU-06	Gestionar políticas de negocio	WEB
CU-07	Diseñar diagrama de actividades	WEB
CU-08	Gestionar formularios dinámicos	WEB
CU-09	Crear trámite	WEB, MÓVIL
CU-10	Visualizar bandeja de tareas	WEB
CU-11	Reclamar tarea	WEB
CU-12	Llenar formulario de trámite	WEB
CU-13	Derivar trámite	WEB
CU-14	Consultar estado del trámite	WEB, MÓVIL

CU-15	Generar KPI básico	WEB
CU-16	Crear repositorio documental por trámite	WEB
CU-17	Subir documento al trámite	WEB, MÓVIL
CU-18	Subir documento a una actividad específica	WEB
CU-19	Visualizar documento	WEB, MÓVIL
CU-20	Crear nueva versión del documento	WEB
CU-21	Consultar historial de versiones	WEB
CU-22	Comparar versiones	WEB
CU-23	Firmar electrónicamente documento	WEB
CU-24	Validar firma electrónica	WEB
CU-25	Asignar permisos documentales	WEB
CU-26	Comentar documento	WEB
CU-27	Editar documento colaborativamente	WEB
CU-28	Registrar auditoría documental	WEB
CU-29	Clasificar solicitud mediante IA	WEB, MÓVIL
CU-30	Llenar formulario mediante IA	WEB
CU-31	Generar reporte por texto	WEB
CU-32	Generar reporte por audio	WEB
CU-33	Predecir riesgo de demora	WEB
CU-34	Recomendar ruta óptima	WEB
CU-35	Detectar anomalías	WEB

2.1.2 Priorización de Casos de Uso

El proyecto se organiza en dos ciclos iterativos dentro de la fase de Construcción de PUDS. El **Ciclo 1** entrega la base funcional del sistema, mientras que el **Ciclo 2** incorpora las funcionalidades avanzadas solicitadas en el segundo parcial.

2.1.2.1 Ciclo 1: Base funcional del sistema

ID	Nombre	Actor(es)	Prioridad
CU-01	Iniciar sesión	A1, A2, A3, A4, A5, A6	ALTA
CU-02	Cerrar sesión	Todos los usuarios	ALTA
CU-03	Gestionar usuarios	A1	ALTA
CU-04	Gestionar roles y permisos	A1	ALTA
CU-05	Gestionar departamentos	A1	ALTA
CU-06	Gestionar políticas de negocio	A1, A2	ALTA
CU-07	Diseñar diagrama de actividades	A2	ALTA
CU-08	Gestionar formularios dinámicos	A1, A2	ALTA
CU-09	Crear trámite	A6	ALTA

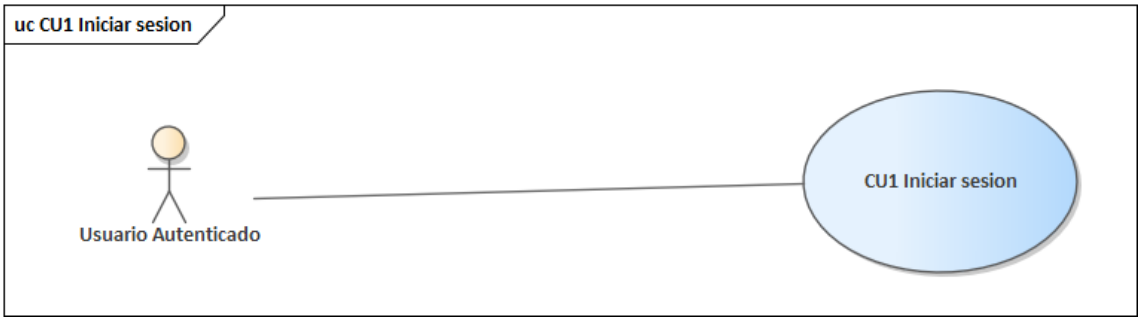
CU-10	Visualizar bandeja de tareas	A3	ALTA
CU-11	Reclamar tarea	A3	ALTA
CU-12	Llenar formulario de trámite	A3	ALTA
CU-13	Derivar trámite	A3	ALTA
CU-14	Consultar estado del trámite	A3, A6	ALTA
CU-15	Generar KPI básico	A1, A4	MEDIA

2.1.2.2 Ciclo 2: Evolución con gestión documental e IA

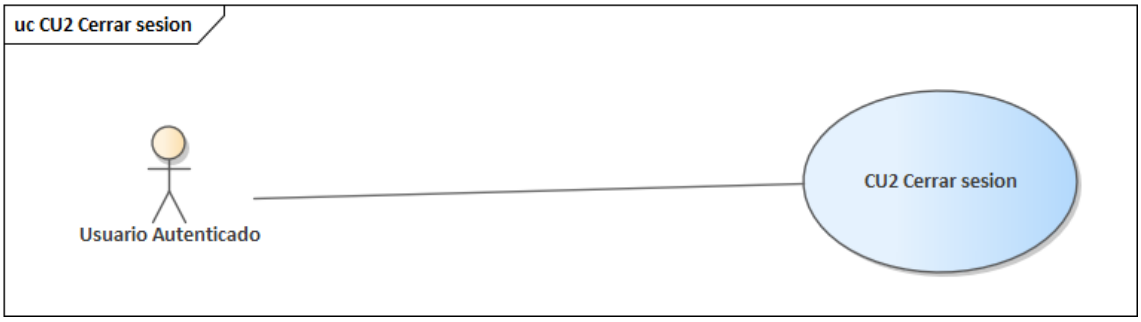
ID	Nombre	Actor(es)	Prioridad
CU-16	Crear repositorio documental por trámite	Sistema, A3, A6	ALTA
CU-17	Subir documento al trámite	A3, A6	ALTA
CU-18	Subir documento a una actividad específica	A3	ALTA
CU-19	Visualizar documento	A3, A4, A5, A6	ALTA
CU-20	Crear nueva versión del documento	A3, A4	ALTA
CU-21	Consultar historial de versiones	A4, A5	MEDIA
CU-22	Comparar versiones	A4, A5	BAJA
CU-23	Firmar electrónicamente documento	A4	ALTA
CU-24	Validar firma electrónica	A5, Sistema	ALTA
CU-25	Asignar permisos documentales	A1, A4	MEDIA
CU-26	Comentar documento	A3, A4	BAJA
CU-27	Editar documento colaborativamente	A2, A3, A4	MEDIA
CU-28	Registrar auditoría documental	Sistema, A5	ALTA
CU-29	Clasificar solicitud mediante IA	A6, A3, A7	ALTA
CU-30	Llenar formulario mediante IA	A3, A7	MEDIA
CU-31	Generar reporte por texto	A1, A4, A7	ALTA
CU-32	Generar reporte por audio	A1, A4, A7	MEDIA
CU-33	Predecir riesgo de demora	A7, A4	ALTA
CU-34	Recomendar ruta óptima	A7, A3	MEDIA
CU-35	Detectar anomalías	A7, A4	MEDIA

2.1.3 Especificar Casos de Uso

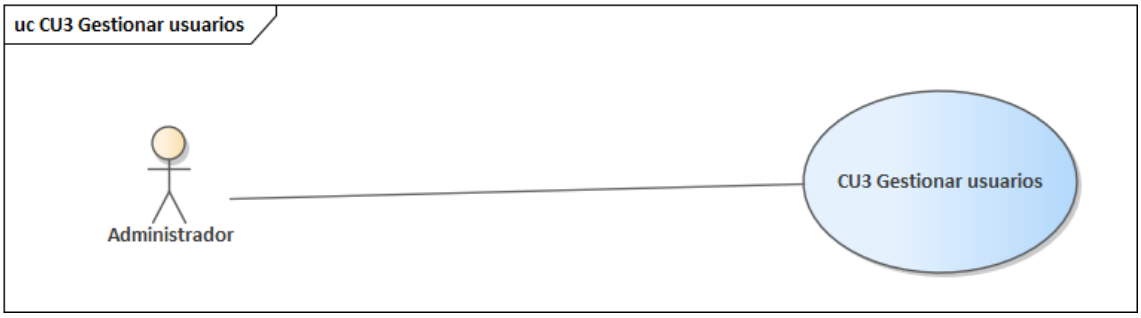
2.1.3.1 CICLO 1



Nombre de CU	CU-01. Iniciar Sesión
Propósito	Permitir que los usuarios registrados accedan al sistema mediante credenciales válidas.
Actores	Administrador (A1), Diseñador de Procesos (A2), Funcionario (A3), Supervisor (A4), Auditor (A5), Cliente (A6)
Actor iniciador	Usuario no autenticado
Precondición	CU-03. Gestionar Usuarios (El usuario debe estar registrado y activo en el sistema).
Flujo principal	**Acceso al formulario:** • El usuario ingresa a la página principal (Web o App Móvil). • Visualiza el botón "Iniciar Sesión" y accede al formulario. • Ingresa su correo electrónico y contraseña. • Presiona "Ingresar". **Validación y autenticación:** • El sistema valida el formato del correo y que los campos no estén vacíos. • El backend (FastAPI) verifica las credenciales contra MongoDB. • Si son correctas, genera un token JWT con tiempo de expiración. • Registra la fecha y hora de ingreso en el perfil del usuario. • Redirige al panel correspondiente según el rol del usuario.
Postcondición	El usuario queda autenticado con un token JWT activo. Se registra el inicio de sesión.
Excepción	• Correo no registrado. • Contraseña incorrecta. • Usuario inactivo o deshabilitado. • Error de conexión con el servidor. • Token de sesión previa aún vigente (según configuración).

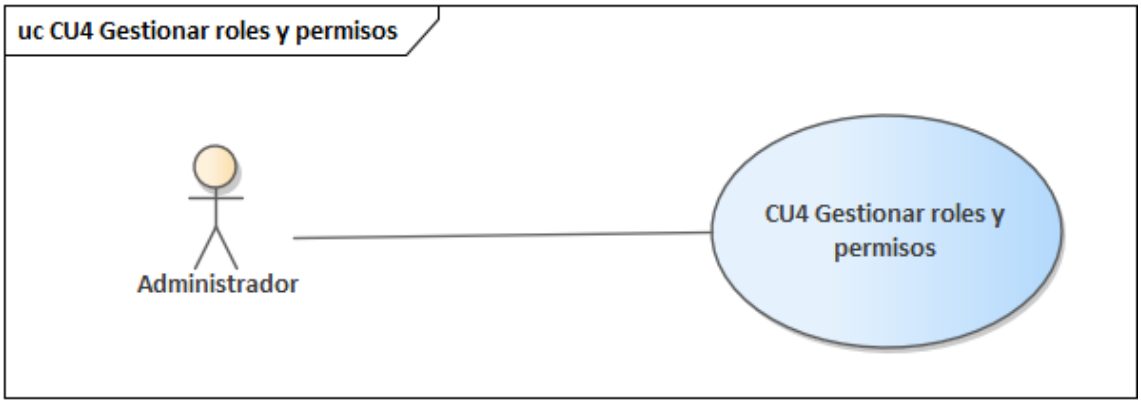


Nombre de CU	CU-02. Cerrar Sesión
Propósito	Permitir que el usuario finalice su sesión de manera segura, invalidando el token de autenticación.
Actores	Todos los usuarios autenticados
Actor iniciador	Usuario autenticado
Precondición	CU-01. Iniciar Sesión (El usuario debe tener una sesión activa).
Flujo principal	• El usuario selecciona la opción "Cerrar Sesión" desde el menú de perfil. • El sistema invalida el token JWT (lo agrega a una lista de denegación o simplemente el cliente lo descarta). • El sistema registra la fecha y hora de salida en el historial de sesiones. • El sistema redirige al usuario a la página de inicio de sesión.
Postcondición	El token de autenticación queda invalidado. El usuario no puede acceder a secciones protegidas sin volver a autenticarse.
Excepción	• Token ya expirado al momento de cerrar sesión. • Error de conexión con el servidor. • Sesión ya finalizada por inactividad.

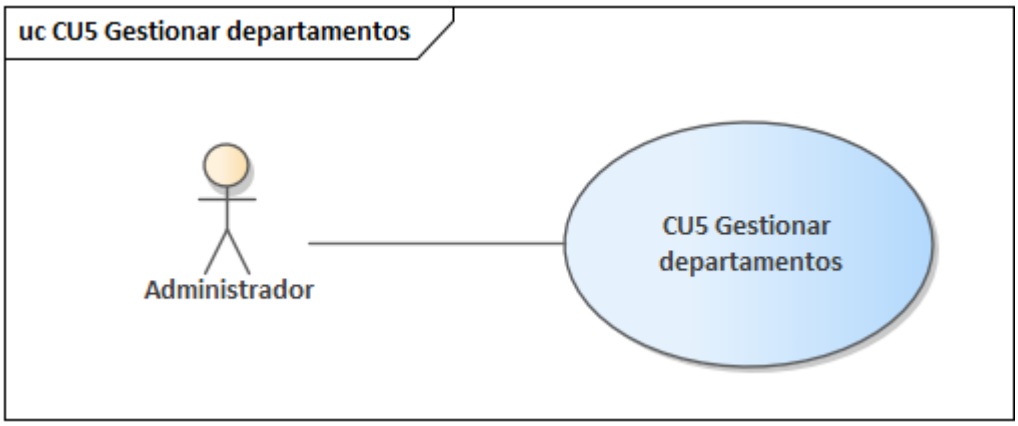


Nombre de CU	CU-03. Gestionar Usuarios
Propósito	Registrar, modificar, habilitar y deshabilitar las cuentas de los usuarios que acceden al sistema.
Actores	Administrador (A1)
Actor iniciador	Administrador

Precondición	CU-04. Gestionar Roles y Permisos (Deben existir roles definidos). CU-05. Gestionar Departamentos (Deben existir departamentos para asignar a funcionarios).
Flujo principal	**Obtener usuarios:** • Listar todos los usuarios registrados con filtros por rol, departamento y estado. • Visualizar detalle de un usuario específico. **Registrar usuario:** • Ingresar datos: nombre, apellido, correo, teléfono. • Asignar un rol (Administrador, Diseñador, Funcionario, Supervisor, Auditor). • Si el rol es Funcionario o Supervisor, asignar un departamento. • El sistema genera una contraseña temporal y envía un correo de bienvenida. • Guardar registro. **Modificar usuario:** • Buscar usuario por nombre, correo o DNI. • Obtener datos actuales. • Actualizar información (teléfono, departamento, rol). • Guardar cambios. **Deshabilitar usuario:** • Buscar usuario activo. • Cambiar estado a "Inactivo". • El usuario no podrá iniciar sesión hasta ser reactivado.
Postcondición	El usuario queda registrado, modificado o deshabilitado según la acción ejecutada.
Excepción	• Correo ya registrado. • Formato de correo inválido. • Campos obligatorios incompletos. • Error al enviar correo de bienvenida.

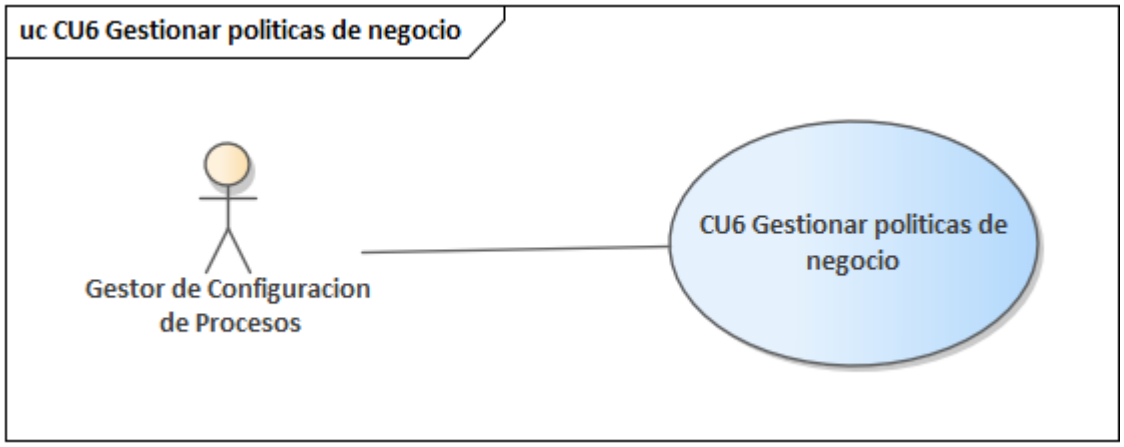


Nombre de CU	CU-04. Gestionar Roles y Permisos
Propósito	Definir y mantener los roles del sistema y los permisos asociados para controlar el acceso a las funcionalidades.
Actores	Administrador (A1)
Actor iniciador	Administrador
Precondición	Ninguna.
Flujo principal	**Obtener roles:** • Listar todos los roles registrados. • Visualizar detalle de un rol con sus permisos. **Registrar rol:** • Ingresar nombre y descripción. • Seleccionar permisos: Crear Política, Diseñar Diagrama, Gestionar Formularios, Ejecutar Tareas, Aprobar, Firmar, Auditar, Ver Reportes, Configurar Sistema. • Guardar. **Modificar rol:** • Buscar rol existente. • Actualizar nombre, descripción o permisos. • Guardar cambios. **Eliminar rol:** • Buscar rol. • Verificar que no esté asignado a ningún usuario activo. • Eliminar.
Postcondición	El rol queda registrado, modificado o eliminado. Los permisos determinan el acceso a las funcionalidades.
Excepción	• Nombre de rol duplicado. • Rol asignado a usuarios activos no puede eliminarse. • Campos obligatorios incompletos.



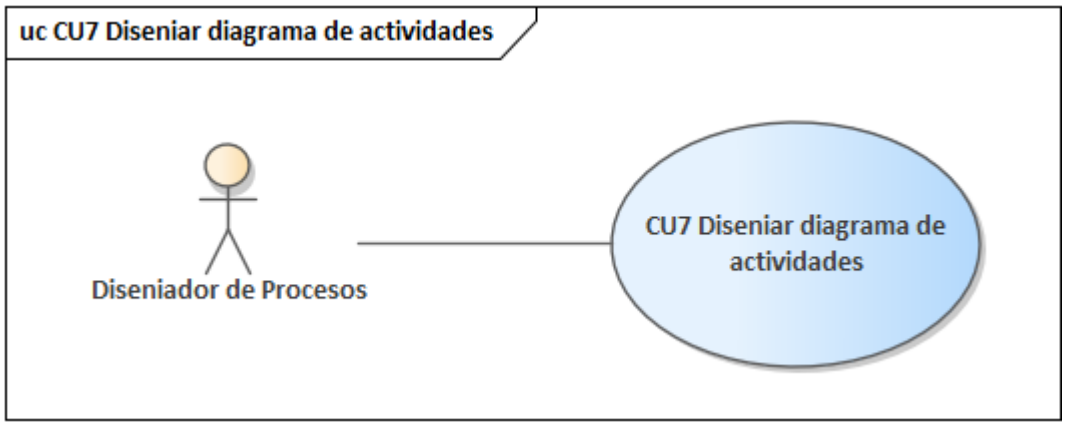
Nombre de CU	CU-05. Gestionar Departamentos
Propósito	Registrar, modificar y eliminar los departamentos que conforman la estructura organizacional a la que se asignan las tareas del workflow.
Actores	Administrador (A1)
Actor iniciador	Administrador
Precondición	Ninguna.
Flujo principal	**Obtener departamentos:** • Listar todos los departamentos. • Visualizar detalle: nombre, código, descripción, funcionarios asignados. **Registrar departamento:** • Ingresar nombre (ej: "Caja", "Préstamos Vehículo", "Gerencia de Riesgos"). • Ingresar código interno y descripción. • Guardar. **Modificar departamento:** • Buscar departamento existente. • Actualizar nombre, código o descripción. **Eliminar departamento:** • Verificar que no esté asignado a políticas activas ni a funcionarios. • Deshabilitar o eliminar.
Postcondición	El departamento queda disponible para ser asignado en diagramas de actividades (swimlanes) y en tareas del workflow.

Excepción	• Nombre o código duplicado. • Departamento asignado a políticas activas o funcionarios. • Campos obligatorios incompletos.



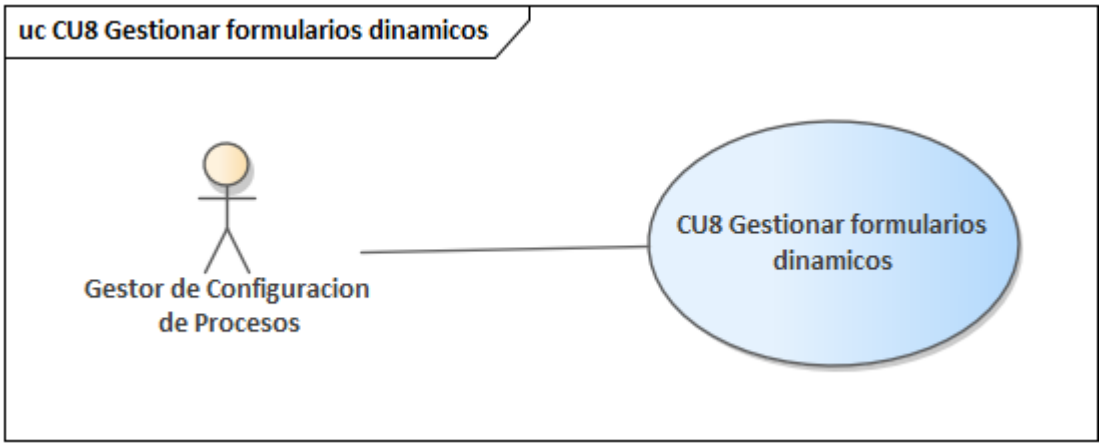
Nombre de CU	CU-06. Gestionar Políticas de Negocio
Propósito	Listar, crear, clonar, versionar, publicar y archivar las políticas de negocio que definen los flujos de trabajo del sistema.
Actores	Administrador (A1), Diseñador de Procesos (A2)
Actor iniciador	Administrador o Diseñador de Procesos
Precondición	CU-05. Gestionar Departamentos (Deben existir departamentos).
Flujo principal	**Obtener políticas:** • Listar todas las políticas con nombre, versión, estado, fecha. • Filtrar por estado (Borrador, Publicado, Archivado). • Visualizar detalle de una política. **Crear nueva política:** • Ingresar nombre y descripción. • Seleccionar categoría o tipo de trámite. • Guardar (estado inicial: Borrador). • El sistema redirige al diseñador de diagramas (CU-07). **Clonar política:** • Seleccionar una política existente. • Crear una copia exacta con nuevo nombre y versión 1. **Versionar política:**

	• Publicar una nueva versión. • Las instancias en curso mantienen la versión anterior; los nuevos trámites usan la última publicada. **Archivar política:** • Cambiar estado a "Archivado". • No se pueden iniciar nuevos trámites con esta política.
Postcondición	La política queda registrada, clonada, versionada o archivada según la acción.
Excepción	• Nombre de política duplicado. • No se puede archivar una política con trámites activos. • Campos obligatorios incompletos.



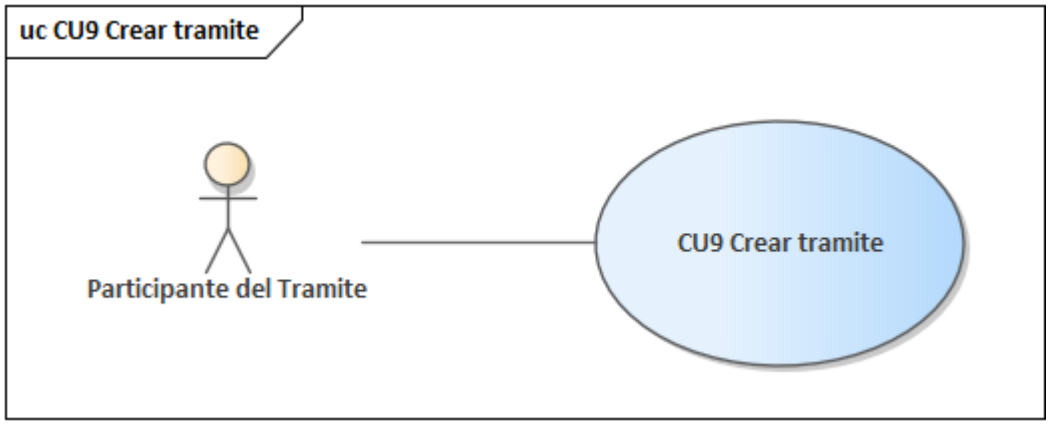
Nombre de CU	CU-07. Diseñar Diagrama de Actividades
Propósito	Permitir al Diseñador de Procesos modelar visualmente una política de negocio utilizando nodos UML (Inicio, Acción, Decisión, Bifurcación, Unión, Fin) y calles (swimlanes) para asignar responsables.
Actores	Diseñador de Procesos (A2)
Actor iniciador	Diseñador de Procesos
Precondición	CU-06. Gestionar Políticas de Negocio (política creada y seleccionada). CU-05. Gestionar Departamentos (departamentos configurados). CU-08. Gestionar Formularios Dinámicos (formularios creados para asociar).
Flujo principal	**Agregar nodos al lienzo:** • El usuario dispone de una paleta con: Inicial, Acción, Decisión, Bifurcación, Unión, Final de Actividad. • Arrastra los nodos necesarios al lienzo.

	Conectar nodos (flujos de control): • Define los flujos de control entre nodos. • En flujos que salen de una Decisión, configura condiciones de guarda (ej: [monto > 10000], [else]). **Configurar acciones:** • Asigna nombre a cada Acción. • Asigna el departamento responsable (calle / swimlane). • Asocia un formulario dinámico (CU-08). • Para bifurcaciones paralelas, inserta nodos Fork y Join. **Guardar diagrama:** • El sistema convierte el diagrama a una estructura de grafo JSON. • Almacena la definición en la colección definiciones_workflow en MongoDB.
Postcondición	El diagrama queda almacenado como grafo JSON. El motor de workflow queda listo para interpretarlo cuando la política sea publicada.
Excepción	• Flujo sin Nodo Final de Actividad (el sistema advierte). • Nodo de Decisión sin condiciones en sus salidas. • Nodo de Acción sin departamento asignado. • Error de conexión al guardar.

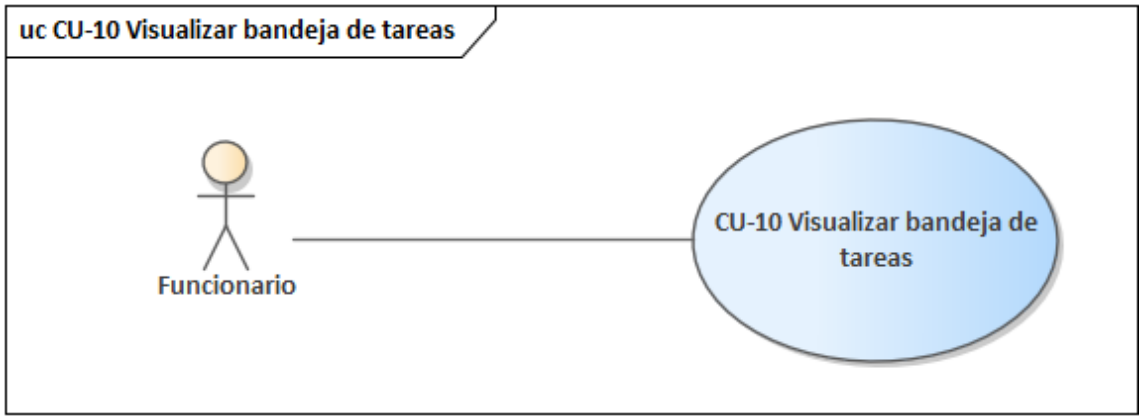


Nombre de CU	CU-08. Gestionar Formularios Dinámicos
Propósito	Crear, editar, versionar y eliminar definiciones de formularios dinámicos que los funcionarios cumplimentarán en las distintas actividades del workflow.

Actores	Administrador (A1), Diseñador de Procesos (A2)
Actor iniciador	Administrador o Diseñador de Procesos
Precondición	CU-06. Gestionar Políticas de Negocio (política existente para asociar el formulario).
Flujo principal	**Obtener formularios:** • Listar formularios con nombre, versión, estado. **Crear formulario:** • Ingresar nombre y descripción. • Agregar campos: tipo (texto, número, fecha, selección, selección múltiple, archivo, área de texto), etiqueta, validaciones (obligatorio, min, max, regex, formatos permitidos). • Ordenar campos. • Guardar: la definición se almacena como JSON en MongoDB. **Modificar formulario:** • Agregar, modificar o eliminar campos. • El sistema genera una nueva versión; trámites en curso mantienen la versión anterior. **Eliminar formulario:** • Verificar que no esté asociado a una política activa. • Deshabilitar o eliminar.
Postcondición	La definición del formulario queda disponible para ser asociada a nodos de acción del workflow.
Excepción	• Nombre de formulario duplicado. • Formulario sin campos definidos. • Formulario asociado a política activa no puede eliminarse.

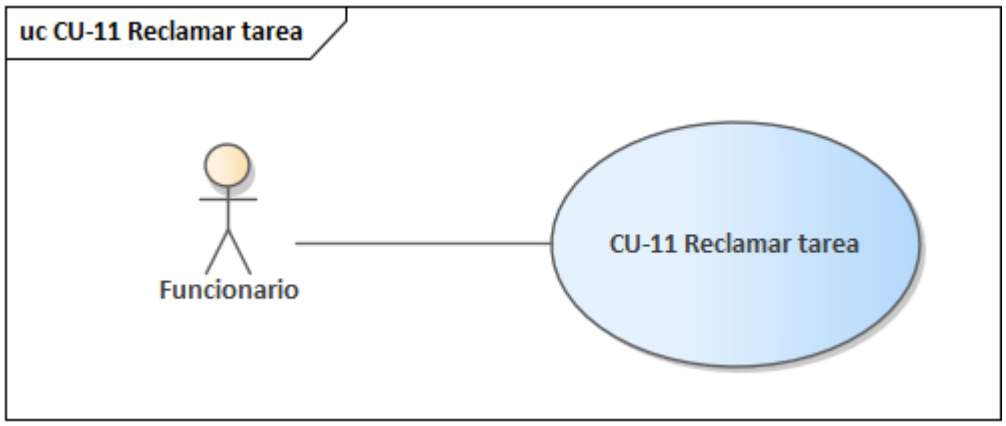


Nombre de CU	CU-09. Crear Trámite
Propósito	Permitir al Cliente o a un Funcionario iniciar una nueva instancia de una política de negocio publicada, dando origen a un trámite.
Actores	Cliente (A6), Funcionario (A3)
Actor iniciador	Cliente o Funcionario
Precondición	CU-01. Iniciar Sesión (usuario autenticado). Debe existir al menos una política de negocio en estado "Publicado".
Flujo principal	**Selección de política:** • El usuario accede a "Solicitar Nuevo Trámite". • El sistema muestra la lista de políticas publicadas disponibles. • El usuario selecciona la política deseada. **Inicio del trámite:** • El sistema crea una nueva instancia en la colección instancias_proceso. • Asigna el cliente_id del solicitante. • Inicializa el token_actual en el Nodo Inicial del diagrama. • El motor de workflow avanza automáticamente al primer nodo de acción. • Crea la primera tarea en tareas para el departamento correspondiente. • Crea el repositorio documental del trámite (CU-16 en Ciclo 2, o estructura básica desde el Ciclo 1). **Confirmación:** • El sistema muestra el ID del trámite generado y confirma la creación.
Postcondición	Una nueva instancia de workflow queda registrada. Se crea la primera tarea en la bandeja del departamento inicial.
Excepción	• No hay políticas publicadas. • Error al crear la instancia en la base de datos. • Error de conexión con el servidor.



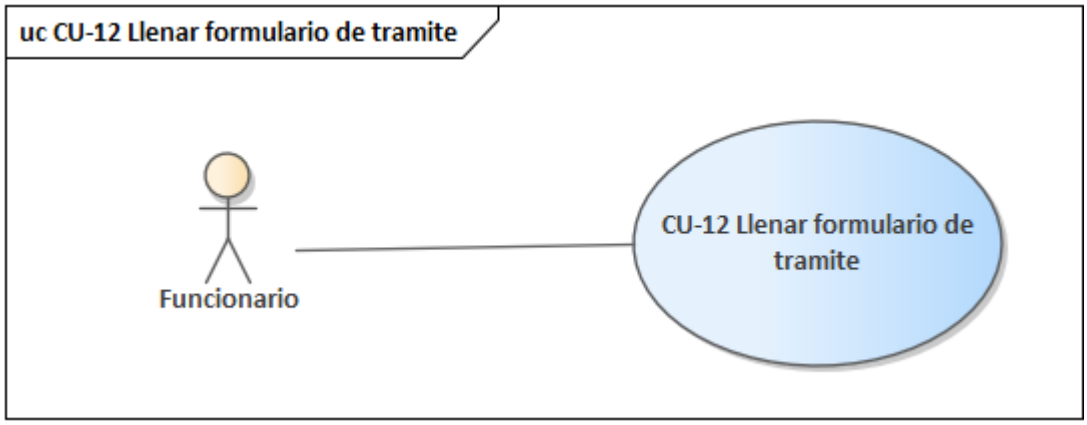
Nombre de CU	CU-10. Visualizar Bandeja de Tareas
Propósito	Permitir al Funcionario ver la lista de tareas pendientes y en progreso asignadas a su departamento, funcionando como su centro de trabajo principal.
Actores	Funcionario (A3)
Actor iniciador	Funcionario
Precondición	CU-01. Iniciar Sesión (Funcionario autenticado). Deben existir tareas asignadas a su departamento.
Flujo principal	**Acceso a la bandeja:** • El Funcionario inicia sesión y es redirigido a su Bandeja de Tareas (Dashboard principal). **Visualización de tareas:** • El sistema muestra una tabla con las tareas del departamento. • Columnas: ID Trámite, Cliente (Nombre, DNI), Tarea, Tipo de Trámite, Fecha, Prioridad, Estado. **Filtros y búsqueda:** • Filtrar por Estado (Pendiente, En Proceso). • Filtrar por Tipo de Trámite. • Filtrar por Prioridad. • Buscar por nombre de cliente o ID de trámite. **Actualización en tiempo real:** • La bandeja se actualiza automáticamente cuando llega una nueva tarea (WebSocket).
Postcondición	El Funcionario visualiza todas las tareas disponibles para su departamento.

Excepción	• Bandeja vacía (sin tareas pendientes). • Error de conexión con el servidor. • Error en la conexión WebSocket (requiere refresco manual).



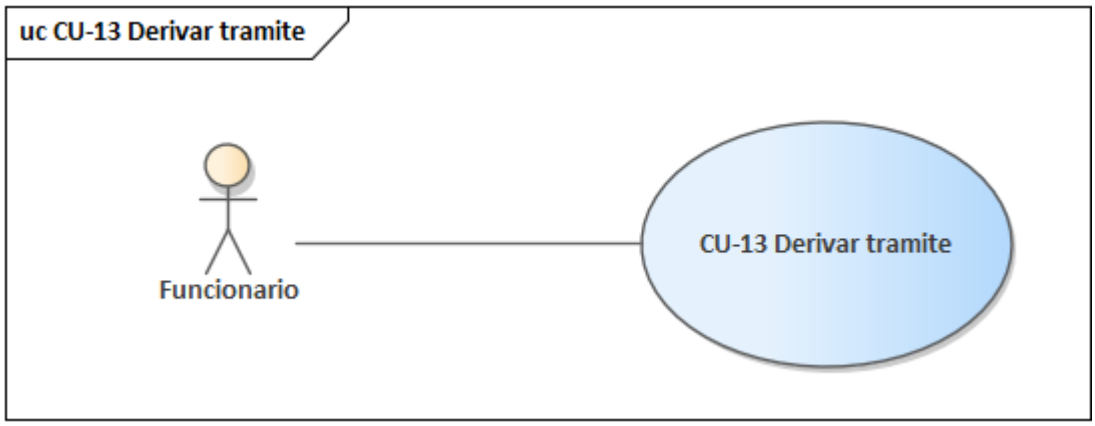
Nombre de CU	CU-11. Reclamar Tarea
Propósito	Permitir al Funcionario tomar posesión de una tarea pendiente de su bandeja, bloqueándola para que otros funcionarios del mismo departamento no puedan trabajar en ella simultáneamente.
Actores	Funcionario (A3)
Actor iniciador	Funcionario
Precondición	CU-10. Visualizar Bandeja de Tareas (Tarea visible con estado "Pendiente").
Flujo principal	**Selección y reclamación:** • El Funcionario identifica una tarea de interés. • Hace clic en "Tomar Tarea" o "Reclamar". **Bloqueo de la tarea:** • El sistema cambia el estado de "PENDIENTE" a "EN_PROCESO". • Asigna el usuario_id del funcionario a la tarea. • La tarea desaparece de la bandeja de otros funcionarios (o se muestra como "Asignada a: [Nombre]"). **Redirección:** • El sistema redirige al Funcionario a la pantalla de ejecución de la tarea (CU-12).
Postcondición	La tarea queda bloqueada y asignada exclusivamente al funcionario que la reclamó.
Excepción	• La tarea ya fue reclamada por otro funcionario en el mismo instante (condición de carrera).

	• La tarea ya no existe (fue cancelada). • Error de conexión al intentar reclamar.



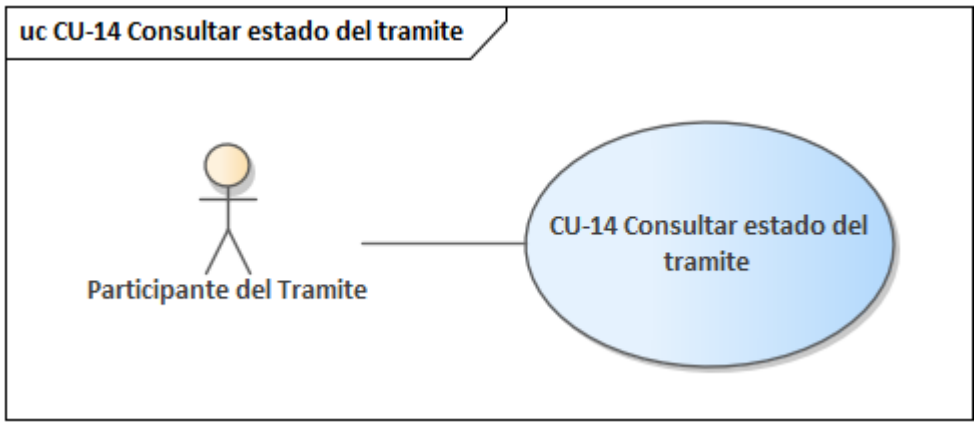
Nombre de CU	CU-12. Llenar Formulario de Trámite
Propósito	Permitir al Funcionario visualizar y completar el formulario dinámico asociado a la etapa actual del trámite, ingresando los datos del cliente.
Actores	Funcionario (A3)
Actor iniciador	Funcionario
Precondición	CU-11. Reclamar Tarea (Tarea asignada al funcionario, estado "EN_PROCESO"). CU-07. Diseñar Diagrama de Actividades (El nodo debe tener un formulario asociado).
Flujo principal	**Carga de la tarea:** • El sistema carga los datos de la tarea reclamada. • Consulta la definición del formulario asociado al nodo desde MongoDB. **Renderizado del formulario:** • El sistema (Angular) interpreta el JSON de definición. • Renderiza dinámicamente los campos: texto, número, fecha, selector, archivo, etc. • Aplica las validaciones configuradas (obligatorio, min, max, regex). **Visualización de datos previos:** • Muestra en modo solo lectura los datos ingresados en pasos anteriores del workflow. **Ingreso de datos:** • El Funcionario completa los campos.

	• El sistema valida en tiempo real. • Muestra mensajes de error si los datos no son válidos. **Guardar borrador (opcional):** • El Funcionario puede guardar el progreso sin derivar.
Postcondición	Los datos del formulario quedan validados en el cliente y listos para ser enviados al derivar (CU-13).
Excepción	• Campos obligatorios vacíos. • Datos con formato inválido. • No hay definición de formulario asociada (el sistema permite continuar sin formulario).



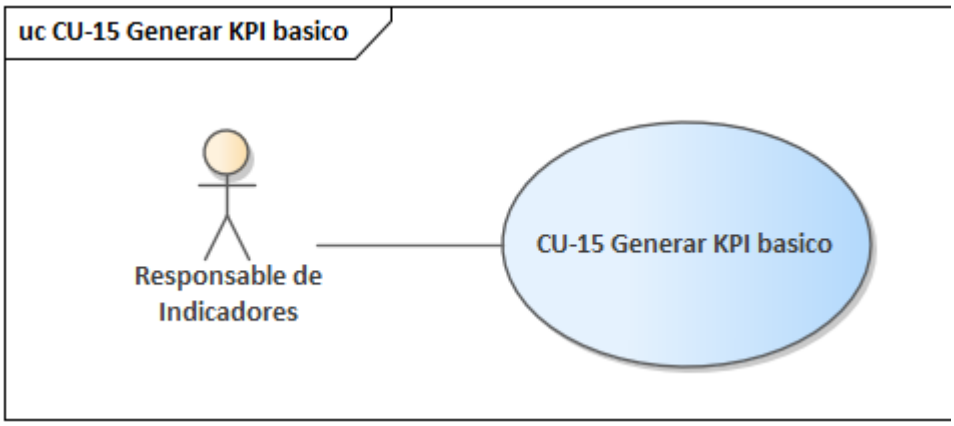
Nombre de CU	CU-13. Derivar Trámite
Propósito	Finalizar la tarea actual del funcionario, ejecutar el motor de reglas y enrutar el trámite al siguiente paso según el diagrama de actividades.
Actores	Funcionario (A3)
Actor iniciador	Funcionario
Precondición	CU-12. Llenar Formulario de Trámite (Formulario completado y validado).
Flujo principal	**Confirmación de derivación:** • El Funcionario presiona "Completar y Derivar". • El sistema realiza una validación final. **Guardado de datos:** • Los valores del formulario se guardan en el contexto_datos de la instancia del trámite en MongoDB. • Se registra el formulario completado en formularios_completados.

	Ejecución del motor de reglas: • El motor ubica el nodo actual en el grafo. • Evalúa los flujos salientes: si es Decisión, evalúa condiciones de guarda; si es Fork, crea múltiples tareas; si es Join, verifica sincronización. • Avanza el token al siguiente nodo. **Creación de nueva tarea:** • Si el nuevo nodo es Acción, crea una entrada en tareas para el departamento correspondiente. • Si es Fin, marca la instancia como COMPLETADO. **Notificación:** • La bandeja del departamento destino se actualiza vía WebSocket. • Se envía notificación push al cliente si corresponde.
Postcondición	La tarea del funcionario queda completada. El trámite avanza al siguiente departamento o finaliza.
Excepción	• Error al evaluar condiciones de guarda (variables no definidas). • Error al guardar los datos del formulario. • Nodo destino sin departamento asignado.



Nombre de CU	CU-14. Consultar Estado del Trámite
Propósito	Permitir al Funcionario o al Cliente consultar el estado actual de un trámite, visualizando el progreso en el workflow.
Actores	Funcionario (A3), Cliente (A6)
Actor iniciador	Funcionario o Cliente

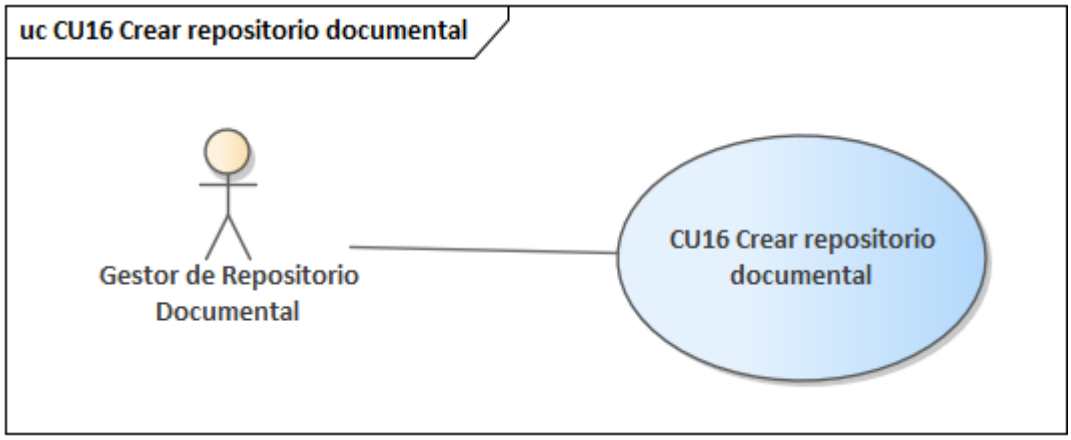
Precondición	CU-01. Iniciar Sesión (Usuario autenticado). Debe existir al menos un trámite asociado al usuario (como ejecutor o como solicitante).
Flujo principal	**Acceso al trámite:** • El usuario selecciona un trámite de su lista. **Visualización del progreso:** • El sistema muestra una representación visual del flujo (barra de progreso o stepper). • Nodos completados: (verde) con fecha. • Nodo actual: (azul pulsante) con departamento responsable. • Nodos futuros: (gris bloqueado). **Información adicional:** • El sistema muestra el ID del trámite, la política utilizada, fechas y departamento actual.
Postcondición	El usuario comprende en qué etapa se encuentra el trámite y los pasos restantes.
Excepción	• Trámite no encontrado. • Usuario sin permisos para ver el trámite.



Nombre de CU	CU-15. Generar KPI Básico
Propósito	Mostrar indicadores clave de desempeño sobre los trámites, como totales, tiempos promedio y distribución por departamento.
Actores	Administrador (A1), Supervisor (A4)
Actor iniciador	Administrador o Supervisor
Precondición	Deben existir trámites registrados en el sistema.

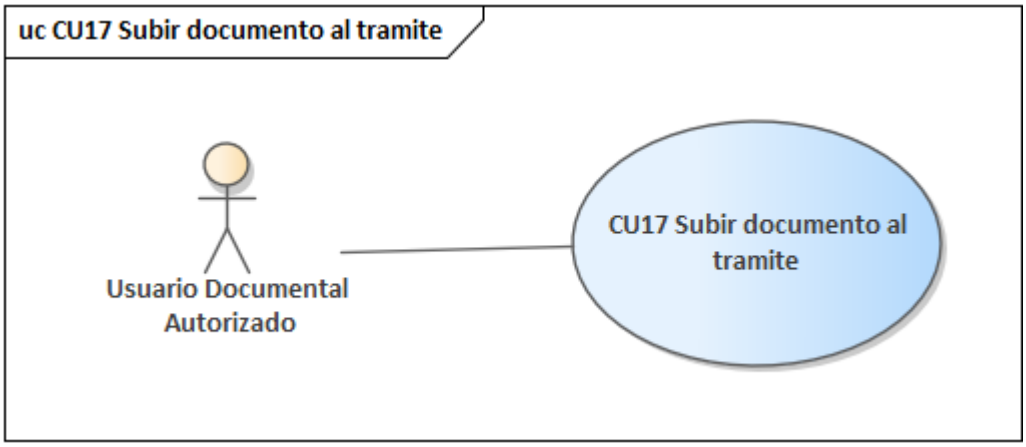
Flujo principal	**Acceso al dashboard:** • El usuario accede a la sección "Dashboard" o "KPI". **Visualización de indicadores:** • Total de trámites creados, en proceso, completados y cancelados. • Tiempo promedio de atención por tipo de trámite. • Trámites por departamento (gráfico de barras). • Trámites por funcionario. • Trámites pendientes por fecha. **Filtros:** • El usuario puede filtrar por rango de fechas, departamento y tipo de trámite.
Postcondición	El usuario visualiza los indicadores básicos de desempeño del sistema.
Excepción	• No hay datos suficientes para mostrar los indicadores. • Error de conexión con la base de datos.

2.1.3.2 CICLO 2



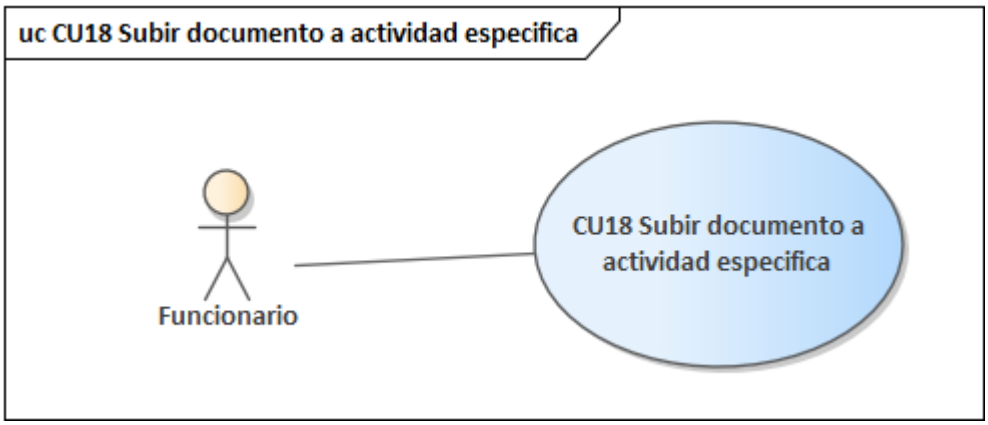
Nombre de CU	CU-16. Crear Repositorio Documental por Trámite
Propósito	Crear automáticamente un espacio documental independiente para cada trámite, donde se almacenarán todos los documentos relacionados.
Actores	Sistema (iniciador automático), Funcionario (A3), Cliente (A6)
Actor iniciador	Sistema
Precondición	CU-09. Crear Trámite (Instancia de trámite creada).

Flujo principal	1. El sistema detecta la creación de un nuevo trámite. 2. Crea un documento en la colección repositorios_documentales asociado al tramite_id. 3. Establece una estructura básica de categorías: "Solicitud", "Documentos del Cliente", "Informes Internos", "Firmas". 4. Asigna permisos según el rol del usuario. 5. Registra la creación del repositorio en la auditoría. 6. El repositorio queda disponible para la carga de documentos.
Postcondición	El trámite cuenta con un repositorio documental propio listo para recibir archivos.
Excepción	• Error al crear el repositorio en MongoDB. • Trámite no encontrado.



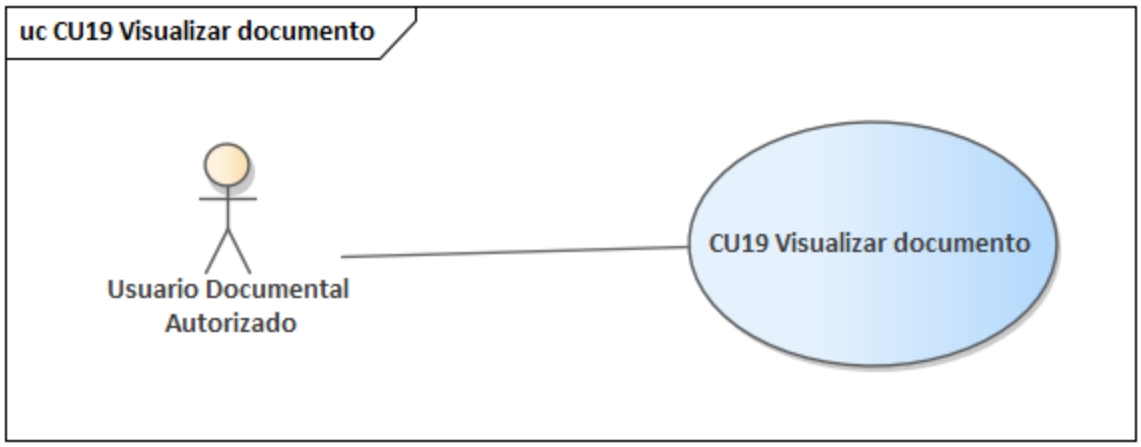
Nombre de CU	CU-17. Subir Documento al Trámite
Propósito	Permitir que un usuario autorizado cargue documentos al repositorio de un trámite específico.
Actores	Funcionario (A3), Cliente (A6), Supervisor (A4)
Actor iniciador	Usuario autorizado
Precondición	CU-16. Crear Repositorio Documental por Trámite (El repositorio debe existir). El usuario debe tener permiso de carga.
Flujo principal	1. El usuario ingresa al detalle del trámite. 2. Accede al repositorio documental. 3. Selecciona "Subir Documento". 4. El sistema solicita: archivo, tipo documental (ej: DNI, Factura, Informe) y descripción opcional. 5. El usuario adjunta el archivo (PDF, Word, JPG, PNG). 6. El sistema valida formato y tamaño. 7. El backend (FastAPI) envía el archivo a Amazon S3.

	8. Guarda los metadatos en MongoDB: documento, versión 1, ruta S3, hash, autor, fecha. 9. Registra la acción en auditoría. 10. El documento aparece en el repositorio del trámite.
Postcondición	El documento queda almacenado en S3 y vinculado al trámite. MongoDB guarda los metadatos.
Excepción	• Formato de archivo no permitido. • Archivo excede el tamaño máximo. • Error de carga en S3. • Usuario sin permisos. • Trámite cerrado o finalizado.

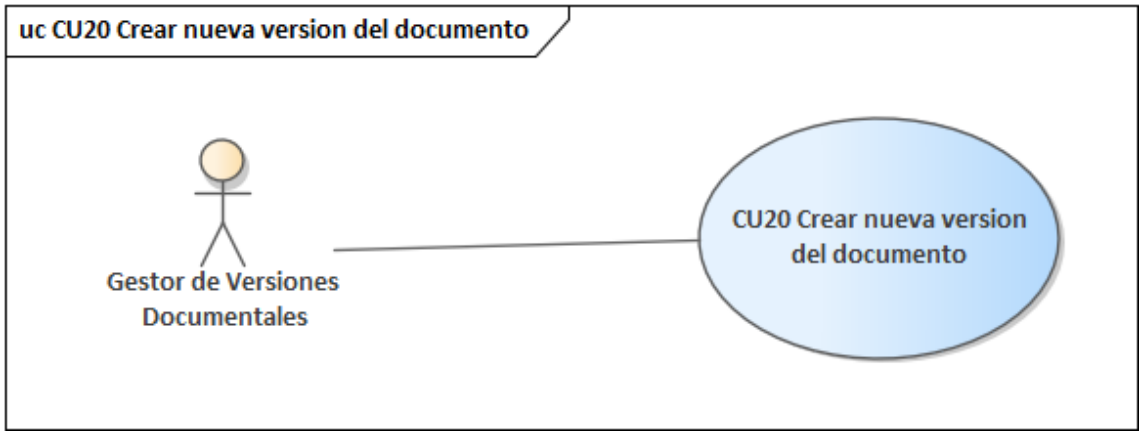


Nombre de CU	CU-18. Subir Documento a una Actividad Específica
Propósito	Permitir al Funcionario asociar un documento no solo al trámite general, sino a una actividad concreta del workflow.
Actores	Funcionario (A3)
Actor iniciador	Funcionario
Precondición	CU-11. Reclamar Tarea (Tarea asignada al funcionario). El repositorio del trámite debe existir.
Flujo principal	1. Durante la ejecución de una tarea, el Funcionario selecciona "Adjuntar Documento a esta Actividad". 2. Sube el archivo siguiendo el flujo de CU-17. 3. El sistema asocia el documento al actividad_id actual, además del tramite_id. 4. El documento queda etiquetado con la actividad específica, facilitando la trazabilidad.
Postcondición	El documento queda vinculado al trámite y a la actividad concreta dentro del workflow.

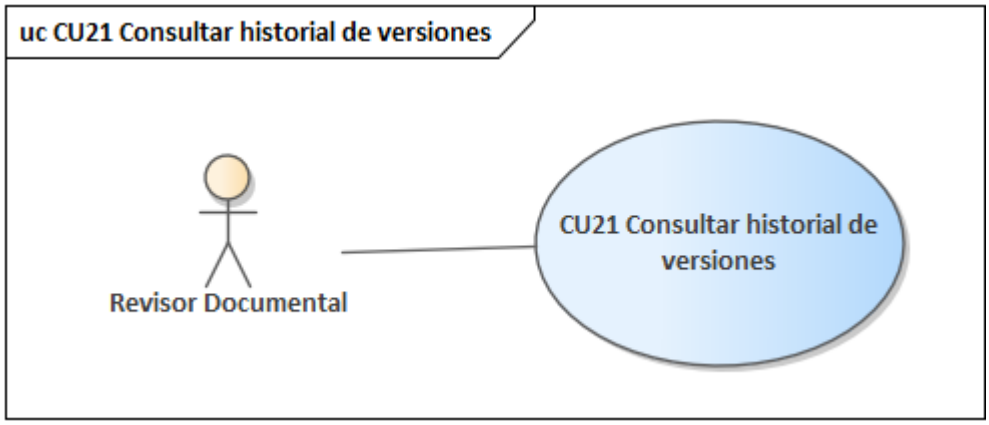
Excepción	• Las mismas que CU-17. • La tarea no está en estado "EN_PROCESO".



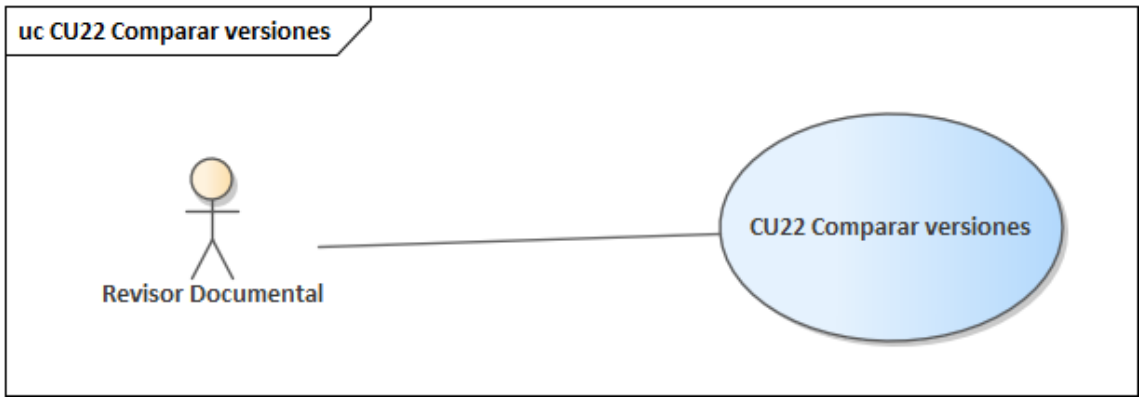
Nombre de CU	CU-19. Visualizar Documento
Propósito	Permitir a los usuarios autorizados previsualizar documentos almacenados en el repositorio del trámite sin necesidad de descargarlos.
Actores	Funcionario (A3), Supervisor (A4), Auditor (A5), Cliente (A6)
Actor iniciador	Usuario autorizado
Precondición	El documento debe existir en el repositorio. El usuario debe tener permiso de lectura sobre el documento.
Flujo principal	1. El usuario accede al repositorio documental del trámite. 2. Selecciona un documento de la lista. 3. El sistema solicita a S3 una URL temporal firmada para el archivo. 4. El sistema muestra el documento en un visor integrado (PDF, imágenes) o permite su previsualización. 5. Se registra la acción de visualización en la auditoría.
Postcondición	El usuario visualiza el contenido del documento.
Excepción	• Documento no encontrado en S3. • Usuario sin permisos de lectura. • Formato no soportado para previsualización (se ofrece descarga).



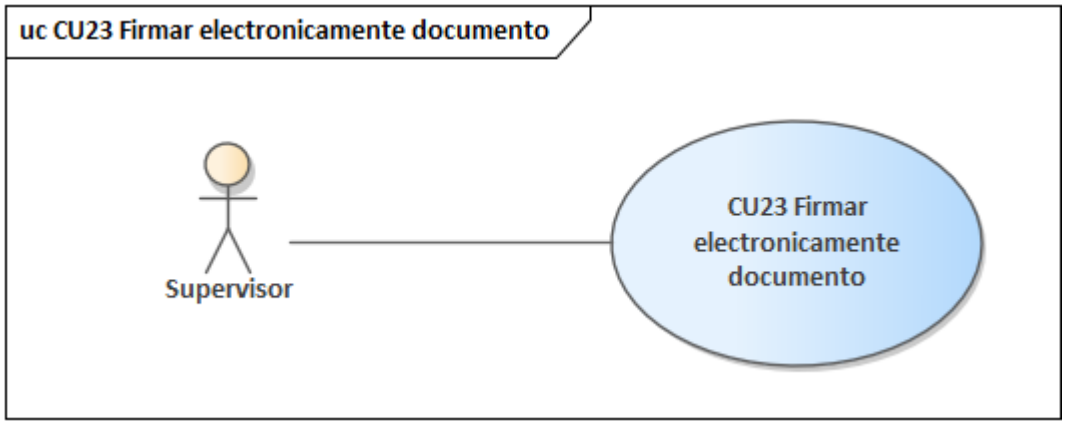
Nombre de CU	CU-20. Crear Nueva Versión del Documento
Propósito	Registrar una nueva versión de un documento preservando el historial completo y la versión anterior intacta.
Actores	Funcionario (A3), Supervisor (A4), Administrador (A1)
Actor iniciador	Usuario autorizado
Precondición	Debe existir un documento previamente cargado en el repositorio.
Flujo principal	1. El usuario selecciona un documento en el repositorio. 2. El sistema muestra sus versiones y la versión actual. 3. El usuario selecciona "Cargar Nueva Versión". 4. Adjunta el nuevo archivo. 5. El sistema valida permisos, formato y tamaño. 6. Incrementa el número de versión (ej: de v1 a v2). 7. Sube el nuevo archivo a S3. 8. Crea un nuevo documento en versiones_documento con hash, ruta, autor, fecha y comentario. 9. La versión anterior se preserva intacta. 10. Se registra en auditoría y se notifica a los interesados.
Postcondición	El documento tiene una nueva versión vigente. El historial de versiones se amplía.
Excepción	• Documento bloqueado (ej: en proceso de firma). • Usuario sin permisos de modificación. • Archivo inválido o corrupto.



Nombre de CU	CU-21. Consultar Historial de Versiones
Propósito	Permitir al usuario visualizar el historial completo de versiones de un documento, incluyendo autor, fecha y comentarios de cada cambio.
Actores	Supervisor (A4), Auditor (A5), Funcionario (A3)
Actor iniciador	Usuario autorizado
Precondición	El documento debe existir y tener al menos una versión registrada.
Flujo principal	1. El usuario selecciona un documento y accede a "Historial de Versiones". 2. El sistema muestra una lista cronológica de versiones: número de versión, fecha, autor, comentario, tamaño, hash. 3. Para cada versión, el usuario puede: visualizar el archivo, descargarlo o consultar sus metadatos. 4. La versión vigente se resalta.
Postcondición	El usuario visualiza el historial completo de cambios del documento.
Excepción	• Documento sin versiones registradas (estado inicial). • Error al cargar el historial desde MongoDB.

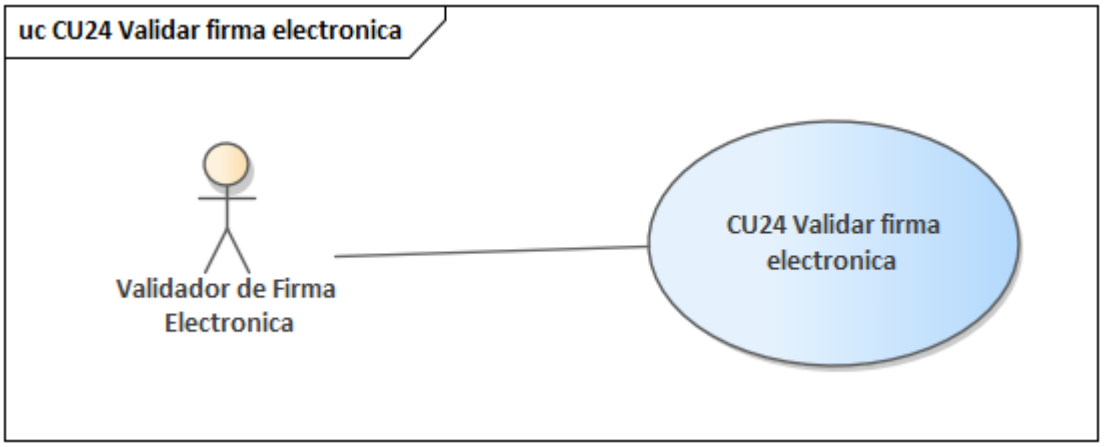


Nombre de CU	CU-22. Comparar Versiones
Propósito	Permitir al usuario comparar dos versiones de un documento para identificar diferencias en contenido o metadatos.
Actores	Supervisor (A4), Auditor (A5)
Actor iniciador	Supervisor o Auditor
Precondición	El documento debe tener al menos dos versiones registradas.
Flujo principal	1. El usuario accede al historial de versiones (CU-21). 2. Selecciona dos versiones para comparar (ej: v1 y v3). 3. El sistema recupera ambos archivos desde S3. 4. Para documentos de texto o PDF, muestra una vista lado a lado con las diferencias resaltadas. 5. Para imágenes, las muestra superpuestas o lado a lado. 6. Muestra una tabla comparativa de metadatos (autor, fecha, tamaño, hash).
Postcondición	El usuario visualiza las diferencias entre las dos versiones seleccionadas.
Excepción	• Formatos de archivo incompatibles para comparación. • Una de las versiones no está disponible en S3.



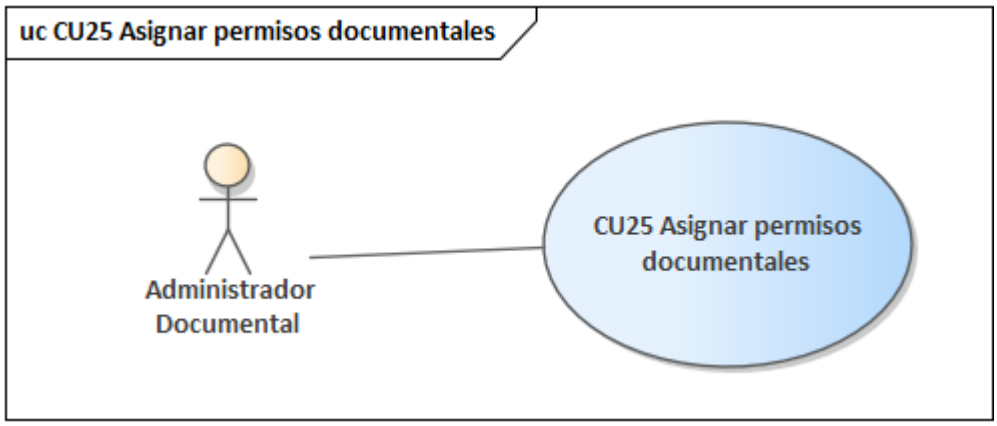
Nombre de CU	CU-23. Firmar Electrónicamente Documento
Propósito	Permitir a un supervisor o funcionario autorizado firmar digitalmente una versión específica de un documento, validando su integridad y autoría.
Actores	Supervisor (A4), Funcionario autorizado
Actor iniciador	Supervisor o Funcionario autorizado

Precondición	El documento debe tener una versión vigente. El usuario debe tener permiso de firma.
Flujo principal	1. El usuario abre el documento desde el visor (CU-19). 2. Selecciona "Firmar Documento". 3. El sistema muestra un resumen de la versión (nombre, hash, fecha) y solicita confirmación. 4. El usuario confirma (posiblemente con autenticación adicional, como contraseña o código OTP). 5. El sistema genera un registro en firmas_electronicas asociando: version_id, firmado_por, fecha_firma, hash_firma, tipo_firma. 6. El documento pasa a estado "Firmado". 7. Se registra la acción en auditoría.
Postcondición	La versión del documento queda firmada electrónicamente. Cualquier modificación posterior requerirá una nueva versión y una nueva firma.
Excepción	• Documento ya firmado (requiere nueva versión para modificaciones). • Usuario sin permiso de firma. • Error en el servicio de firma. • El hash del archivo en S3 no coincide con el registrado.



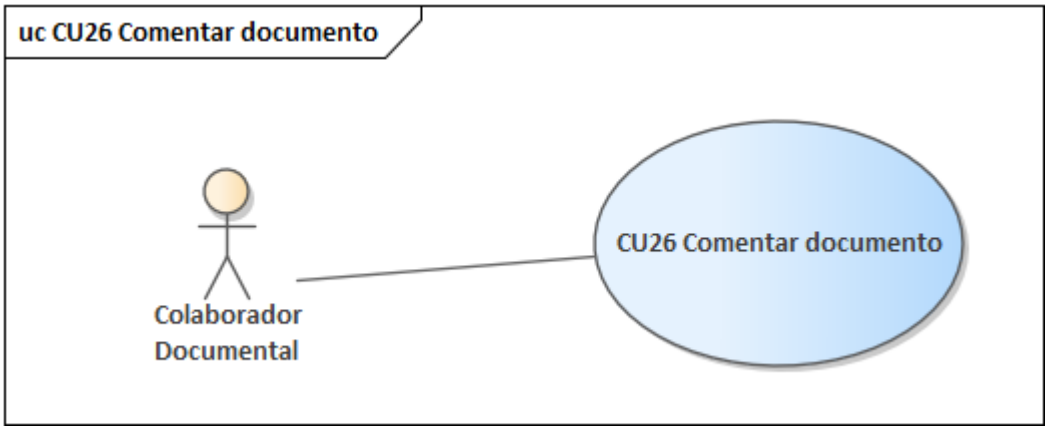
Nombre de CU	CU-24. Validar Firma Electrónica
Propósito	Verificar la autenticidad y vigencia de una firma electrónica asociada a un documento.
Actores	Auditor (A5), Sistema
Actor iniciador	Auditor
Precondición	El documento debe tener al menos una firma registrada.
Flujo principal	1. El auditor accede al detalle del documento firmado. 2. Selecciona "Validar Firma". 3. El sistema verifica: integridad del archivo en S3 (hash), vigencia del

	certificado (si aplica), identidad del firmante. 4. Muestra el resultado: "Firma Válida" (con detalles del firmante y fecha) o "Firma Inválida" (con el motivo). 5. Registra la validación en auditoría.
Postcondición	El estado de la firma queda verificado.
Excepción	• Firma no encontrada. • Documento modificado después de la firma (hash no coincide). • Certificado del firmante expirado.

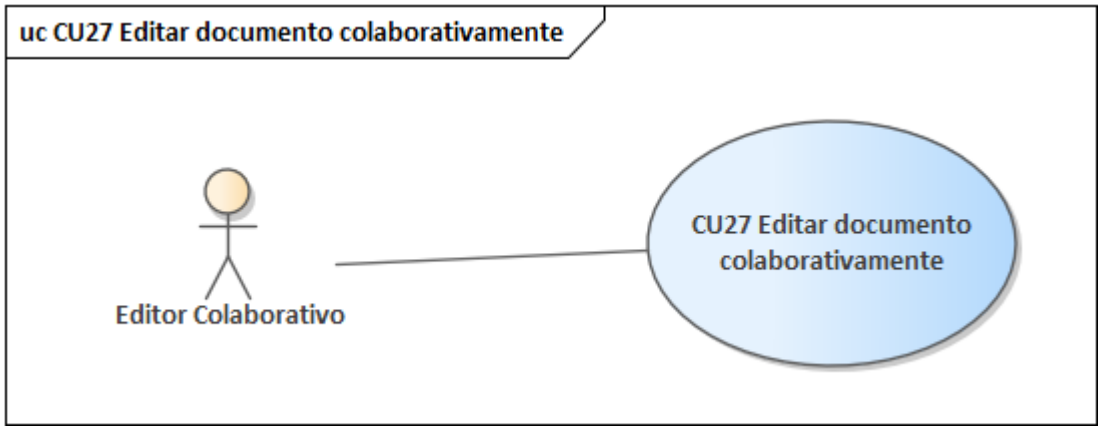


Nombre de CU	CU-25. Asignar Permisos Documentales
Propósito	Permitir al Administrador o Supervisor definir qué usuarios o roles pueden leer, modificar, versionar o firmar cada documento dentro de un repositorio.
Actores	Administrador (A1), Supervisor (A4)
Actor iniciador	Administrador o Supervisor
Precondición	El repositorio documental y los documentos deben existir.
Flujo principal	1. El usuario accede a la gestión de permisos del repositorio o de un documento específico. 2. Visualiza la lista de permisos actuales. 3. Agrega un nuevo permiso: selecciona usuario o rol, y el nivel de acceso (Lectura, Escritura, Versionado, Firma, Administración). 4. Modifica o revoca permisos existentes. 5. Guarda los cambios. 6. El sistema aplica los nuevos permisos inmediatamente y registra la acción en auditoría.
Postcondición	Los permisos documentales quedan actualizados. Los usuarios ven restringido o permitido su acceso según la configuración.

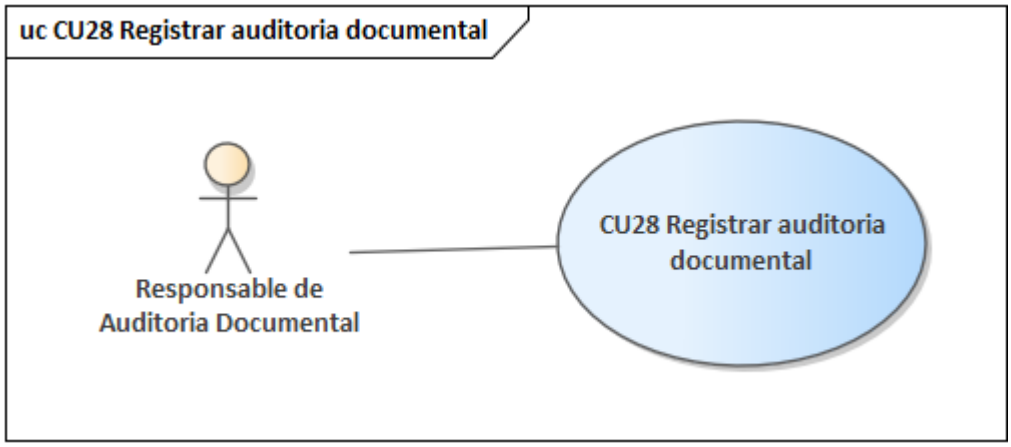
Excepción	• Usuario o rol no encontrado. • No se puede revocar el permiso del propietario del trámite.



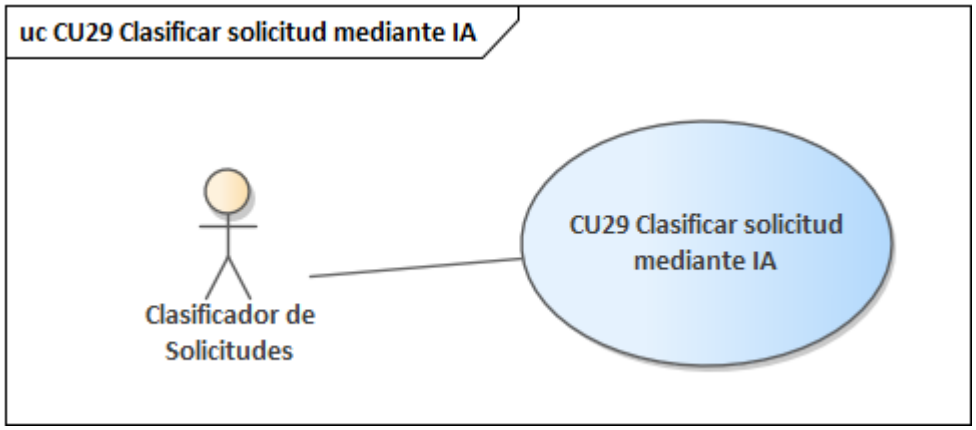
Nombre de CU	CU-26. Comentar Documento
Propósito	Permitir a los usuarios dejar comentarios asociados a un documento o a una versión específica, facilitando la comunicación y la revisión.
Actores	Funcionario (A3), Supervisor (A4)
Actor iniciador	Usuario autorizado
Precondición	El documento debe existir en el repositorio.
Flujo principal	1. El usuario abre el documento o su detalle. 2. Accede a la sección "Comentarios". 3. Escribe un comentario (texto enriquecido). 4. Puede mencionar a otros usuarios (ej: "@Supervisor revisar"). 5. Publica el comentario. 6. El sistema guarda el comentario asociado al documento y notifica a los usuarios mencionados.
Postcondición	El comentario queda publicado y visible para los usuarios con acceso al documento.
Excepción	• Documento no encontrado. • Usuario sin permisos para comentar.



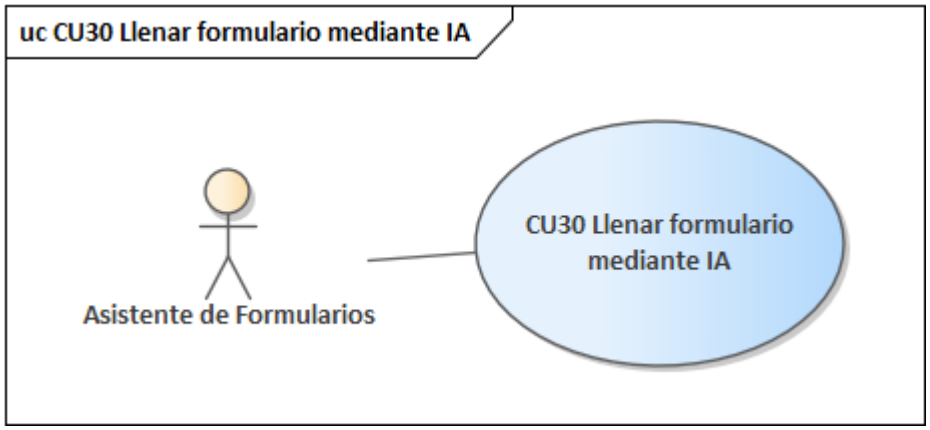
Nombre de CU	CU-27. Editar Documento Colaborativamente
Propósito	Permitir que múltiples usuarios visualicen y, si tienen permisos, modifiquen el contenido de un documento o un diagrama en tiempo real.
Actores	Diseñador de Procesos (A2), Funcionario (A3), Supervisor (A4)
Actor iniciador	Usuario autorizado
Precondición	El documento o diagrama debe estar en un formato editable y el usuario debe tener permiso de escritura.
Flujo principal	1. El usuario abre un diagrama de actividades o un documento de texto desde el repositorio. 2. El sistema establece una conexión WebSocket para la sala de colaboración. 3. Los usuarios conectados ven los cambios en tiempo real: cursores de colores, selecciones, ediciones. 4. Las modificaciones se sincronizan automáticamente. 5. Al guardar, se genera una nueva versión del documento (CU-20).
Postcondición	Los cambios colaborativos quedan guardados y sincronizados entre todos los participantes.
Excepción	• Pérdida de conexión WebSocket (reconexión automática). • Conflicto de edición (el sistema notifica a los usuarios).



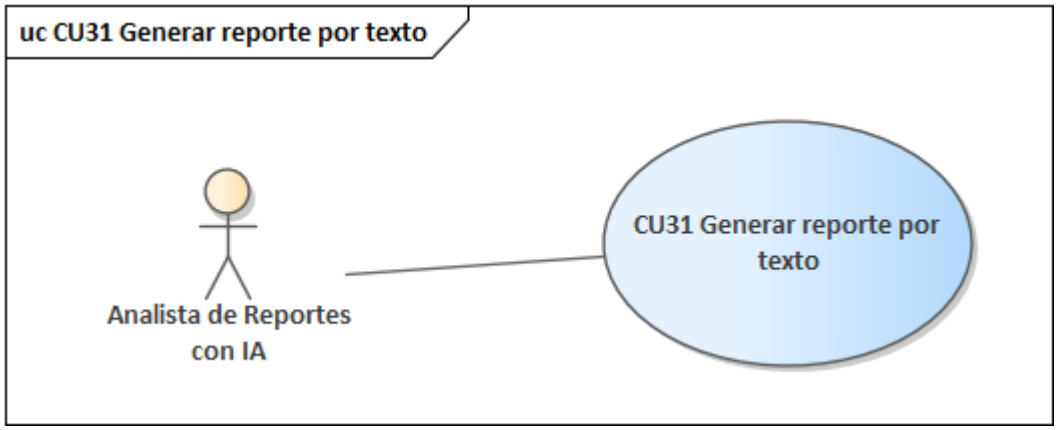
Nombre de CU	CU-28. Registrar Auditoría Documental
Propósito	Registrar de forma automática e inmutable cada acción relevante sobre los documentos, permitiendo trazabilidad completa para fines de auditoría.
Actores	Sistema (automático), Auditor (A5)
Actor iniciador	Sistema
Precondición	El módulo de auditoría debe estar activo.
Flujo principal	1. Cada vez que ocurre una acción documental (crear repositorio, subir documento, versionar, firmar, validar, modificar permisos, comentar, visualizar), el sistema genera un registro en la colección auditoria. 2. El registro incluye: usuario_id, accion, entidad (documento, versión, firma), entidad_id, fecha_hora, detalle, ip. 3. El auditor (A5) puede consultar estos registros mediante filtros por fecha, usuario, trámite o tipo de acción.
Postcondición	Cada acción documental queda registrada de forma trazable e inmutable.
Excepción	• Error de escritura en la colección de auditoría (el sistema lo notifica como incidente).



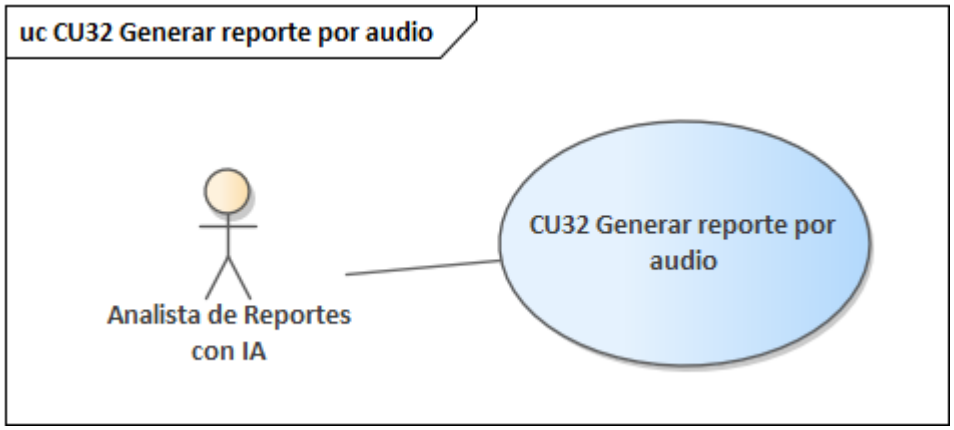
Nombre de CU	CU-29. Clasificar Solicitud mediante IA
Propósito	Analizar una solicitud ingresada por texto o audio para identificar automáticamente la política de negocio correspondiente y extraer datos iniciales.
Actores	Cliente (A6), Funcionario (A3), Sistema de IA (A7)
Actor iniciador	Cliente o Funcionario
Precondición	Debe existir al menos una política de negocio activa. El módulo de IA debe estar operativo.
Flujo principal	1. El usuario ingresa una solicitud por texto o audio. 2. Si es audio, el sistema (ASR) lo convierte a texto. 3. El texto se envía al módulo de IA (TensorFlow). 4. La IA analiza intención, palabras clave y contexto. 5. Identifica la política más probable y asigna un nivel de confianza. 6. Extrae datos relevantes: nombres, fechas, tipo de trámite, montos. 7. El sistema muestra la propuesta de clasificación y los datos precargados. 8. El funcionario valida o corrige. 9. Se crea el trámite (CU-09) con la política identificada y los datos iniciales.
Postcondición	La solicitud queda clasificada y se inicia un trámite asociado a la política identificada.
Excepción	• Texto o audio ambiguo. • Audio ilegible. • Ninguna política alcanza el umbral mínimo de confianza (se deriva a clasificación manual). • Error en el servicio de IA.



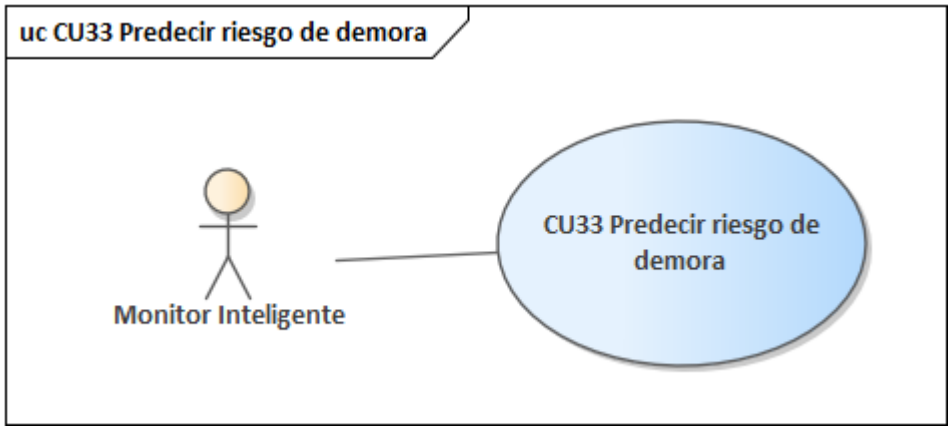
Nombre de CU	CU-30. Llenar Formulario mediante IA
Propósito	Permitir que la IA analice el texto de una solicitud o un documento cargado y auto-rellene los campos del formulario de trámite.
Actores	Funcionario (A3), Sistema de IA (A7)
Actor iniciador	Funcionario
Precondición	CU-12. Llenar Formulario de Trámite (formulario abierto). Debe existir texto de solicitud o documento cargado.
Flujo principal	1. Durante el llenado del formulario, el Funcionario selecciona "Auto-rellenar con IA". 2. El sistema envía el texto de la solicitud o el contenido extraído de un documento (OCR) al módulo de IA. 3. La IA identifica entidades: nombres, fechas, direcciones, montos, tipos. 4. Mapea las entidades a los campos del formulario. 5. El sistema precarga los campos con los datos extraídos. 6. El Funcionario revisa y corrige si es necesario antes de derivar.
Postcondición	Los campos del formulario quedan precargados con los datos extraídos por la IA.
Excepción	• Datos insuficientes en el texto para rellenar los campos obligatorios. • Baja confianza en los datos extraídos (se resaltan para revisión). • Error en el servicio de IA.



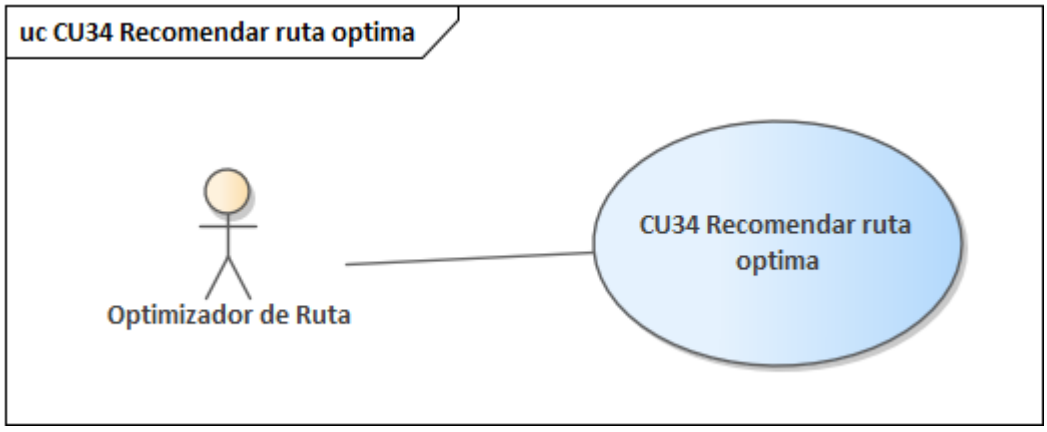
Nombre de CU	CU-31. Generar Reporte por Texto
Propósito	Permitir al usuario generar reportes mediante consultas expresadas en lenguaje natural escrito.
Actores	Administrador (A1), Supervisor (A4), Sistema de IA (A7)
Actor iniciador	Administrador o Supervisor
Precondición	Deben existir trámites registrados. El módulo de IA debe estar operativo.
Flujo principal	1. El usuario accede al generador de reportes. 2. Ingresa una consulta en texto: "Necesito un reporte de trámites atendidos por el departamento técnico desde el 1 de mayo hasta el 30 de mayo". 3. La IA interpreta: tipo de reporte (trámites atendidos), filtro departamento, filtro fechas. 4. El backend traduce la interpretación a una consulta de MongoDB. 5. Obtiene los datos y construye una tabla y gráficos. 6. El usuario visualiza el reporte y puede exportarlo a PDF o Excel.
Postcondición	El reporte se genera y queda disponible para su consulta o exportación.
Excepción	• Consulta ambigua (la IA solicita aclaración). • Filtros inválidos. • Sin datos para el período solicitado.



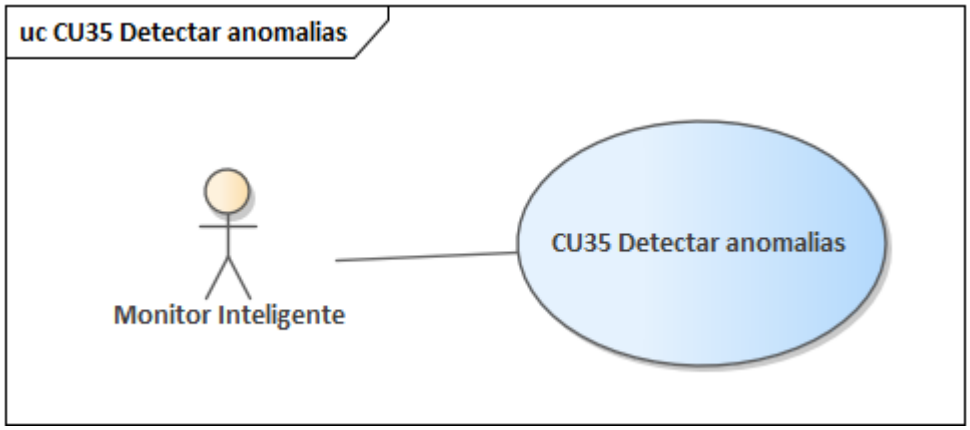
Nombre de CU	CU-32. Generar Reporte por Audio
Propósito	Permitir al usuario generar reportes mediante consultas expresadas por voz.
Actores	Administrador (A1), Supervisor (A4), Sistema de IA (A7)
Actor iniciador	Administrador o Supervisor
Precondición	Deben existir trámites registrados. El módulo de IA (ASR + PLN) debe estar operativo.
Flujo principal	1. El usuario accede al generador de reportes y selecciona el modo de audio. 2. Graba su consulta: "Muéstrame los trámites con riesgo de demora alto en el último mes". 3. El sistema transcribe el audio a texto (ASR). 4. La IA interpreta el texto (PLN) y extrae los filtros y métricas. 5. El backend ejecuta la consulta en MongoDB. 6. Se genera el reporte con tabla y gráficos. 7. El usuario puede exportarlo a PDF o Excel.
Postcondición	El reporte se genera a partir de la consulta por voz.
Excepción	• Audio ilegible o con ruido de fondo. • Consulta no interpretable. • Las mismas que CU-31.



Nombre de CU	CU-33. Predecir Riesgo de Demora
Propósito	Utilizar IA para predecir la probabilidad de que un trámite exceda los plazos establecidos, basándose en datos históricos y el estado actual.
Actores	Sistema de IA (A7), Supervisor (A4)
Actor iniciador	Sistema (automático, por lote) o Supervisor (bajo demanda)
Precondición	Deben existir datos históricos suficientes de trámites completados. El modelo de IA debe estar entrenado.
Flujo principal	1. El sistema (o el supervisor) solicita la evaluación de riesgo de trámites activos. 2. El módulo de IA analiza variables: tiempo transcurrido, actividad actual, carga del funcionario, cantidad de documentos pendientes, desviaciones históricas. 3. La IA asigna un puntaje de riesgo (bajo, medio, alto) a cada trámite. 4. Los resultados se muestran en el dashboard de KPI y en la bandeja de tareas. 5. Los trámites con riesgo alto se resaltan para priorizar su atención.
Postcondición	Los trámites activos quedan clasificados según su riesgo de demora.
Excepción	• Datos históricos insuficientes (el modelo no tiene suficiente precisión). • Error en la inferencia del modelo de IA.



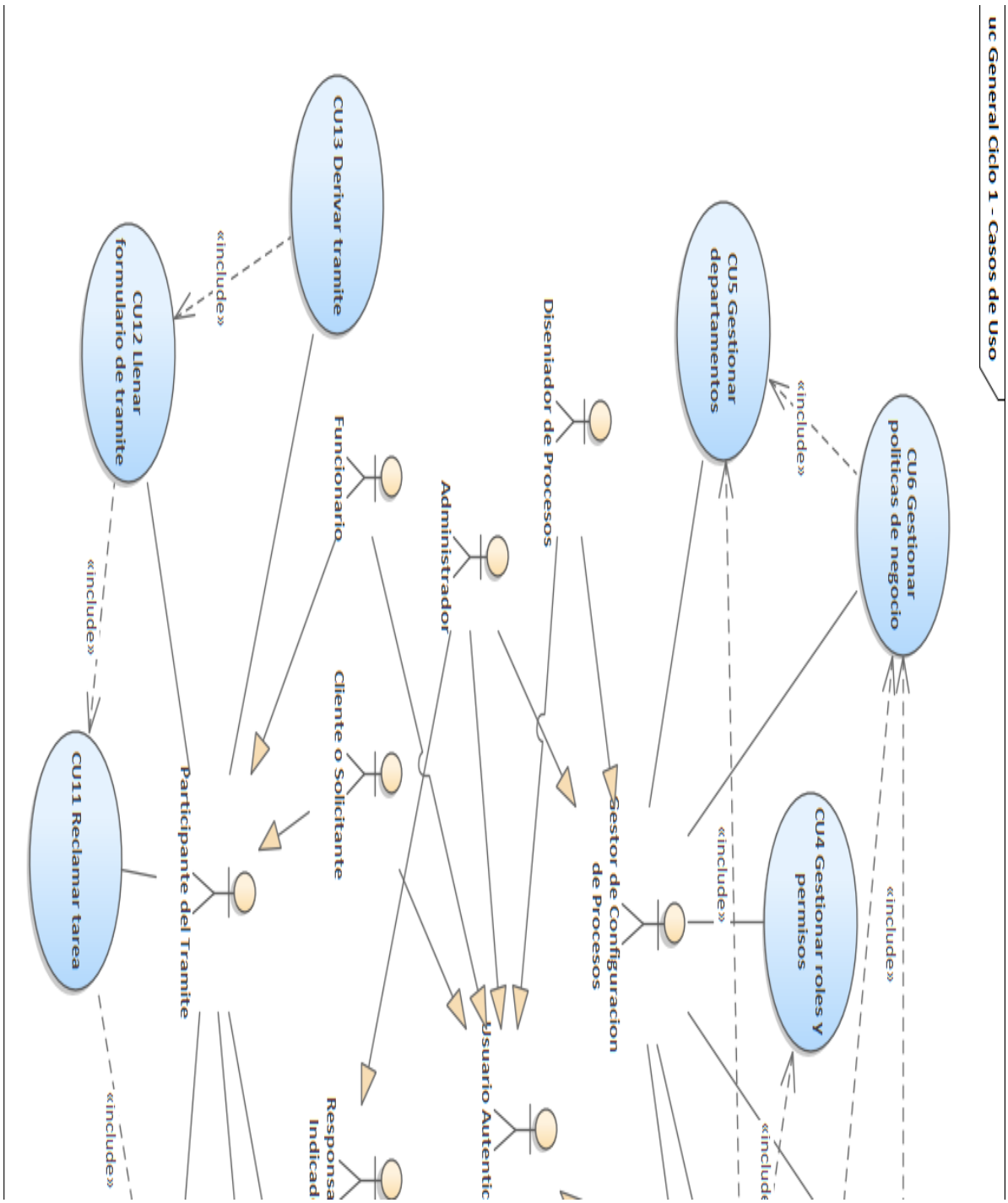
Nombre de CU	CU-34. Recomendar Ruta Óptima
Propósito	Utilizar IA para recomendar la mejor ruta de asignación para un trámite, considerando carga de trabajo, tiempos históricos y prioridad.
Actores	Sistema de IA (A7), Funcionario (A3), Supervisor (A4)
Actor iniciador	Sistema (al derivar) o Supervisor (bajo demanda)
Precondición	Deben existir datos históricos de asignaciones y tiempos de resolución.
Flujo principal	1. Al momento de derivar un trámite (CU-13), si el nodo destino es una Decisión sin condición determinista, o si el supervisor solicita una recomendación, el sistema invoca a la IA. 2. La IA analiza: carga actual de los funcionarios del departamento destino, tiempos históricos de resolución para trámites similares, prioridad del trámite, documentos pendientes. 3. La IA recomienda: el funcionario más adecuado, o la rama del flujo que minimiza el tiempo total estimado. 4. El sistema muestra la recomendación al funcionario o supervisor, quien puede aceptarla o elegir manualmente.
Postcondición	Se sugiere una ruta óptima que puede reducir el tiempo de ciclo del trámite.
Excepción	• Datos insuficientes para una recomendación confiable. • El usuario decide no seguir la recomendación.

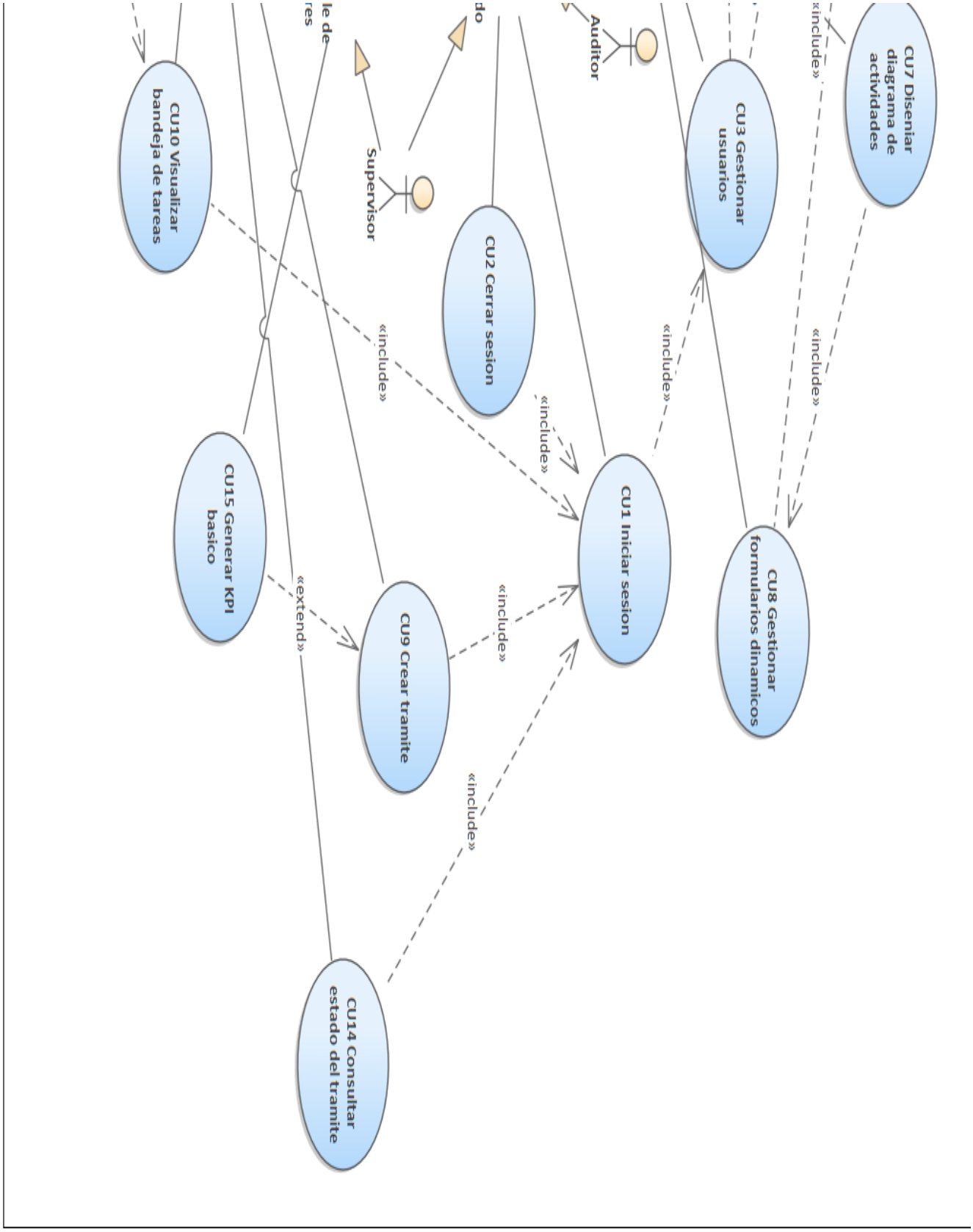


Nombre de CU	CU-35. Detectar Anomalías
Propósito	Utilizar IA para identificar comportamientos anómalos en los trámites, como detenciones prolongadas, modificaciones excesivas de documentos o patrones fuera de lo normal.
Actores	Sistema de IA (A7), Supervisor (A4)
Actor iniciador	Sistema (automático, periódico)
Precondición	Deben existir datos históricos de trámites para establecer patrones normales.
Flujo principal	1. El sistema ejecuta periódicamente un análisis de anomalías sobre los trámites activos. 2. La IA compara el comportamiento actual con patrones históricos: tiempo en cada actividad, cantidad de versiones de documentos, cantidad de devoluciones, frecuencia de modificaciones. 3. Detecta desviaciones significativas: trámite detenido más del doble del promedio, documento con más de 5 versiones en un día, trámite que retrocede más de 2 veces. 4. Genera alertas en el dashboard de KPI y notifica al supervisor. 5. El supervisor revisa las anomalías y decide las acciones correctivas.
Postcondición	Las anomalías detectadas quedan registradas y visibles para el supervisor.
Excepción	• Falsos positivos (el supervisor puede descartar la alerta). • Error en el modelo de detección de anomalías.

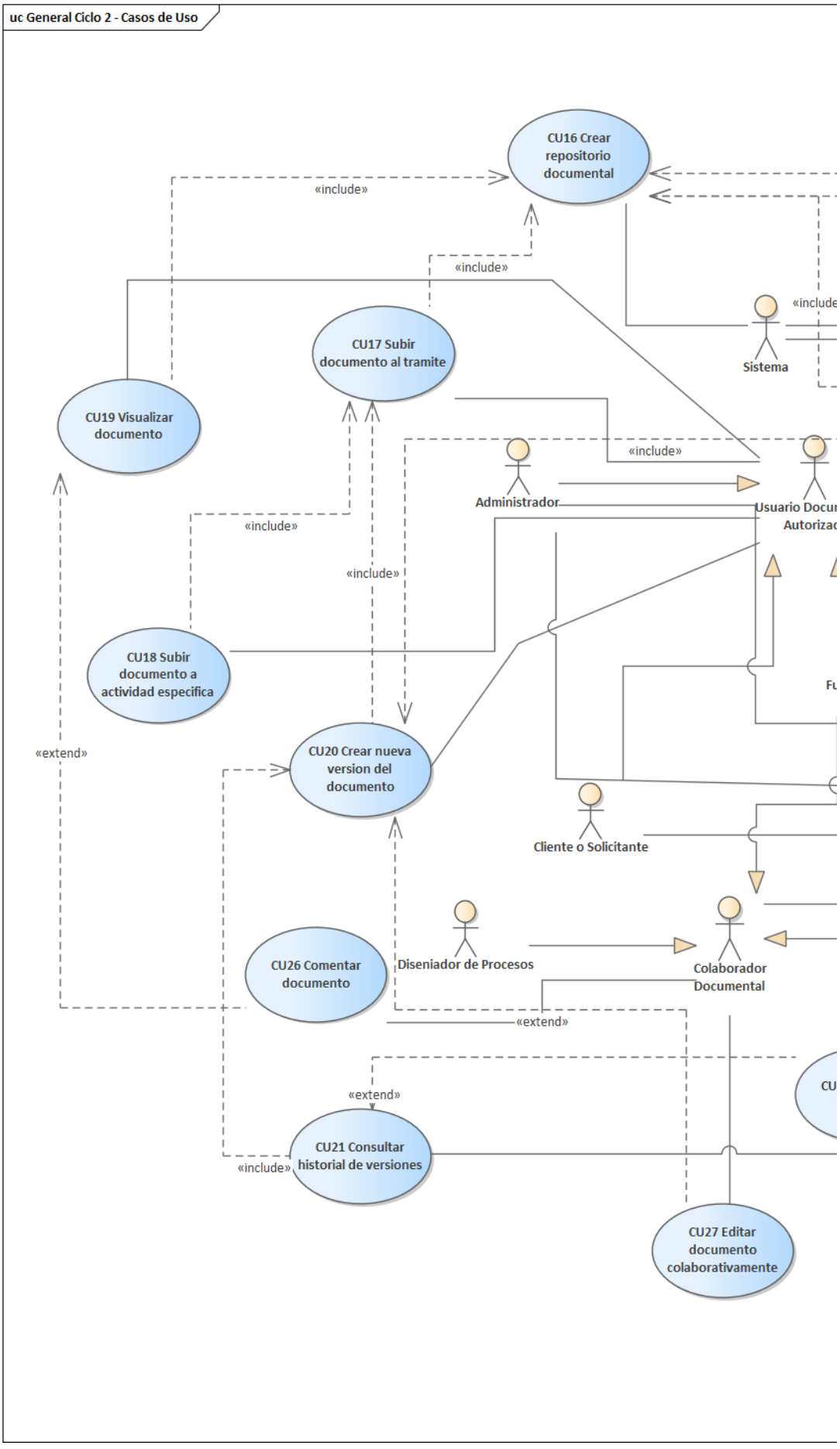
2.1.4 Estructurar Modelos De Casos De Uso

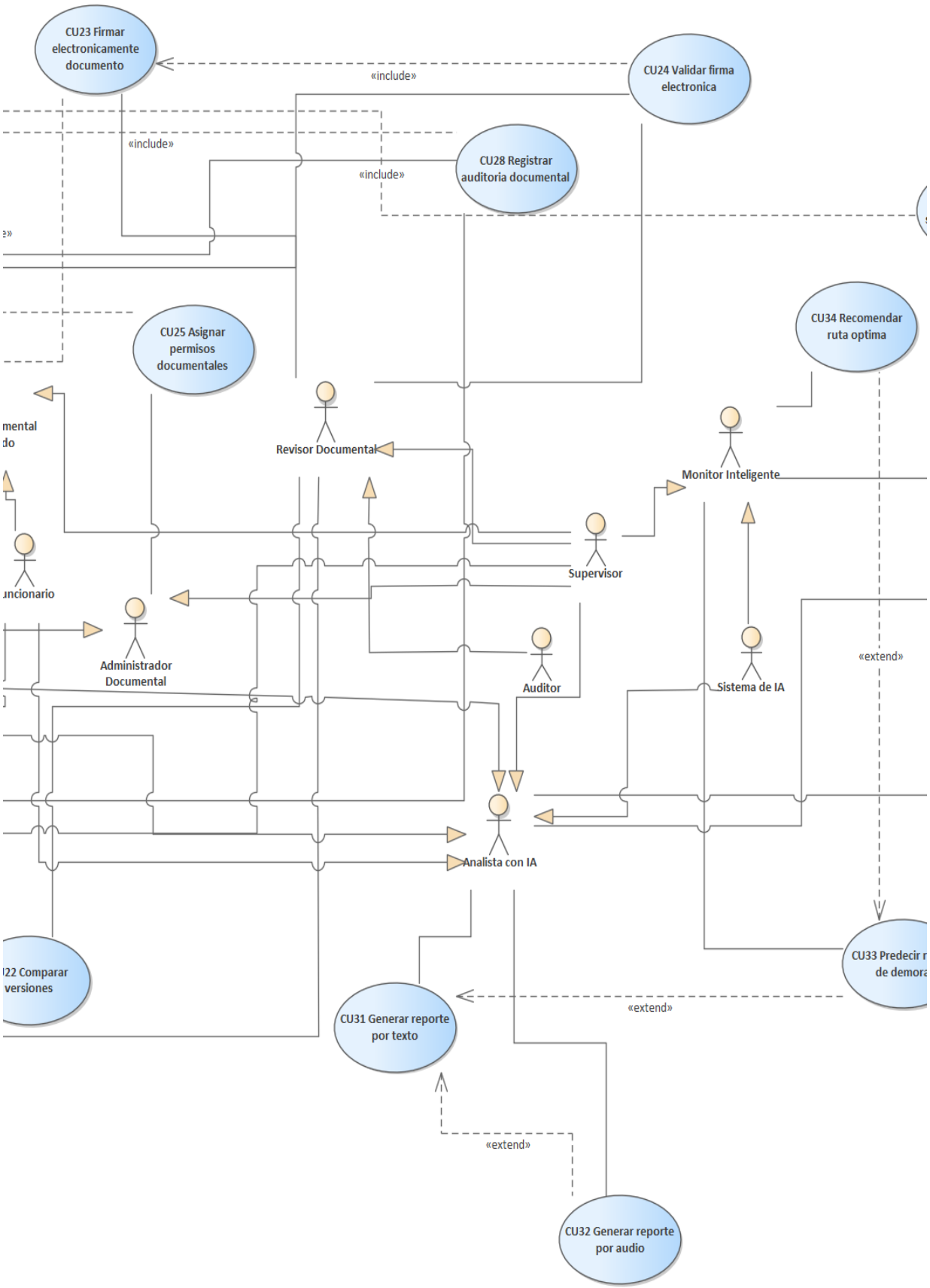
Ciclo #1

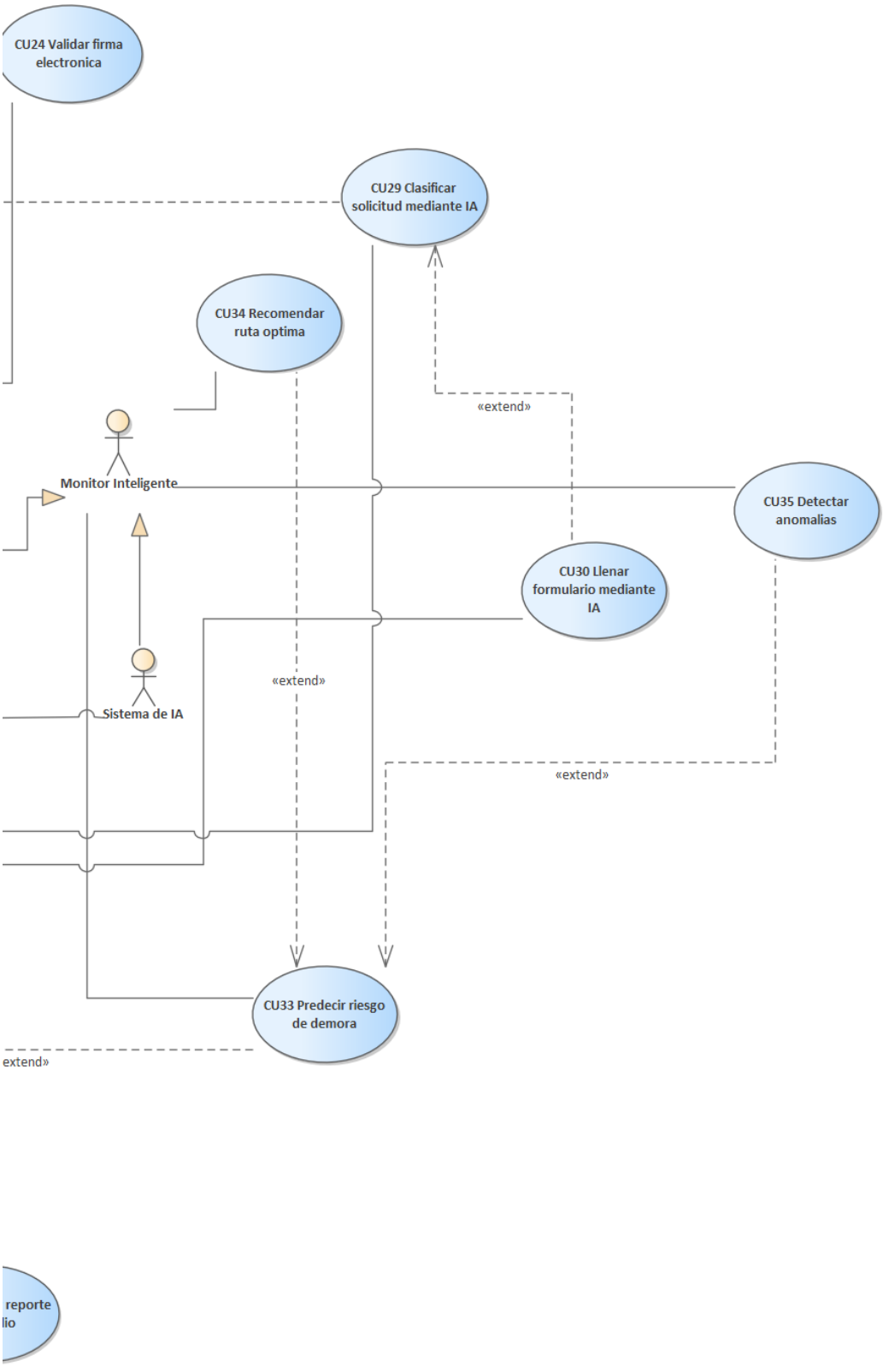



CU7 Diseniar diagrama de actividades
«include»
CU3 Gestionar usuarios
Auditor
«include»
«include»
CU8 Gestionar formularios dinamicos
CU2 Cerrar sesion
Supervisor
«include»
CU1 Iniciar sesion
«include»
CU10 Visualizar bandeja de tareas
«include»
CU15 Generar KPI basico
«extend»
CU9 Crear tramite
«include»
«include»
CU14 Consultar estado del tramite

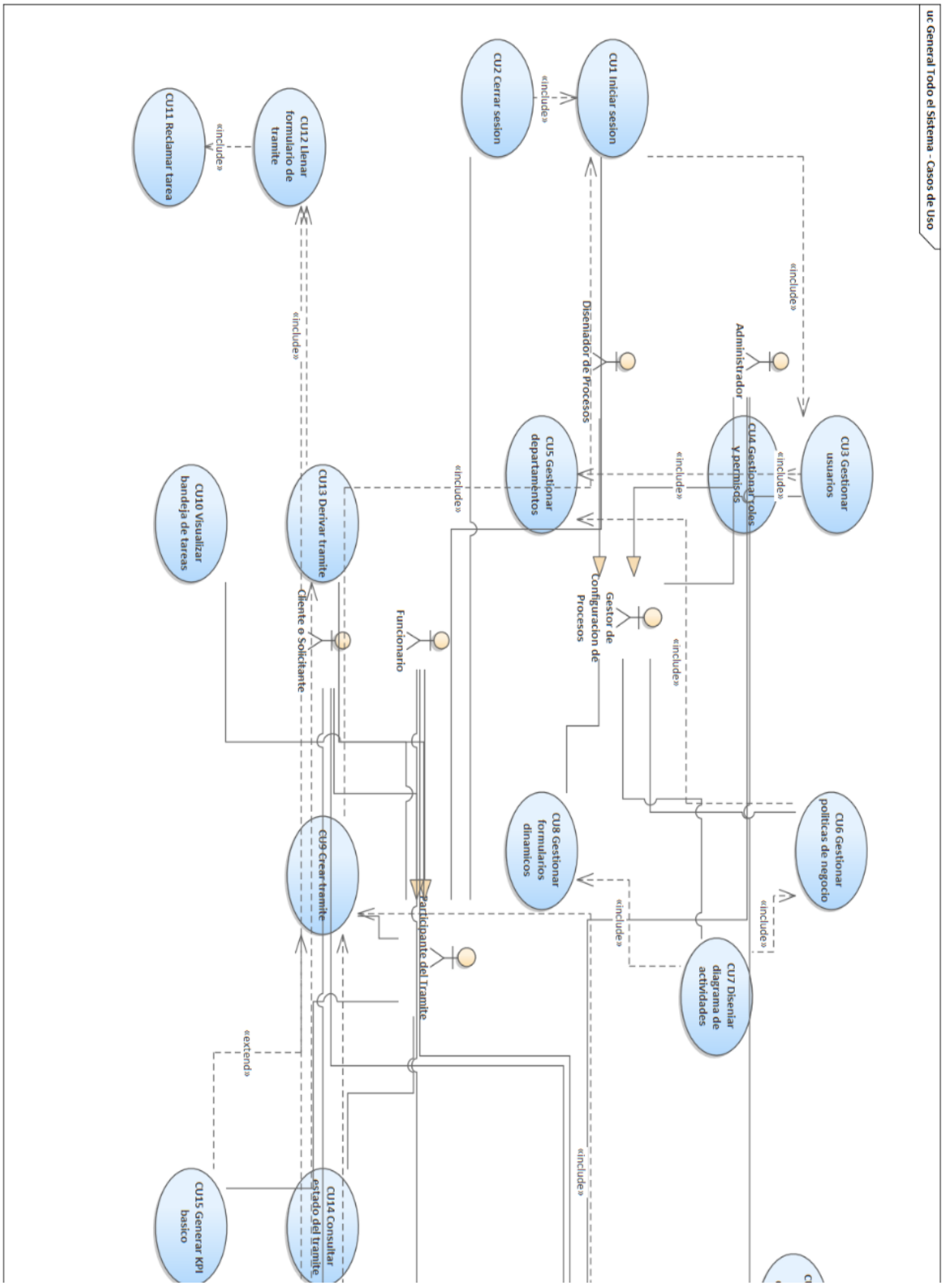
Ciclo #2

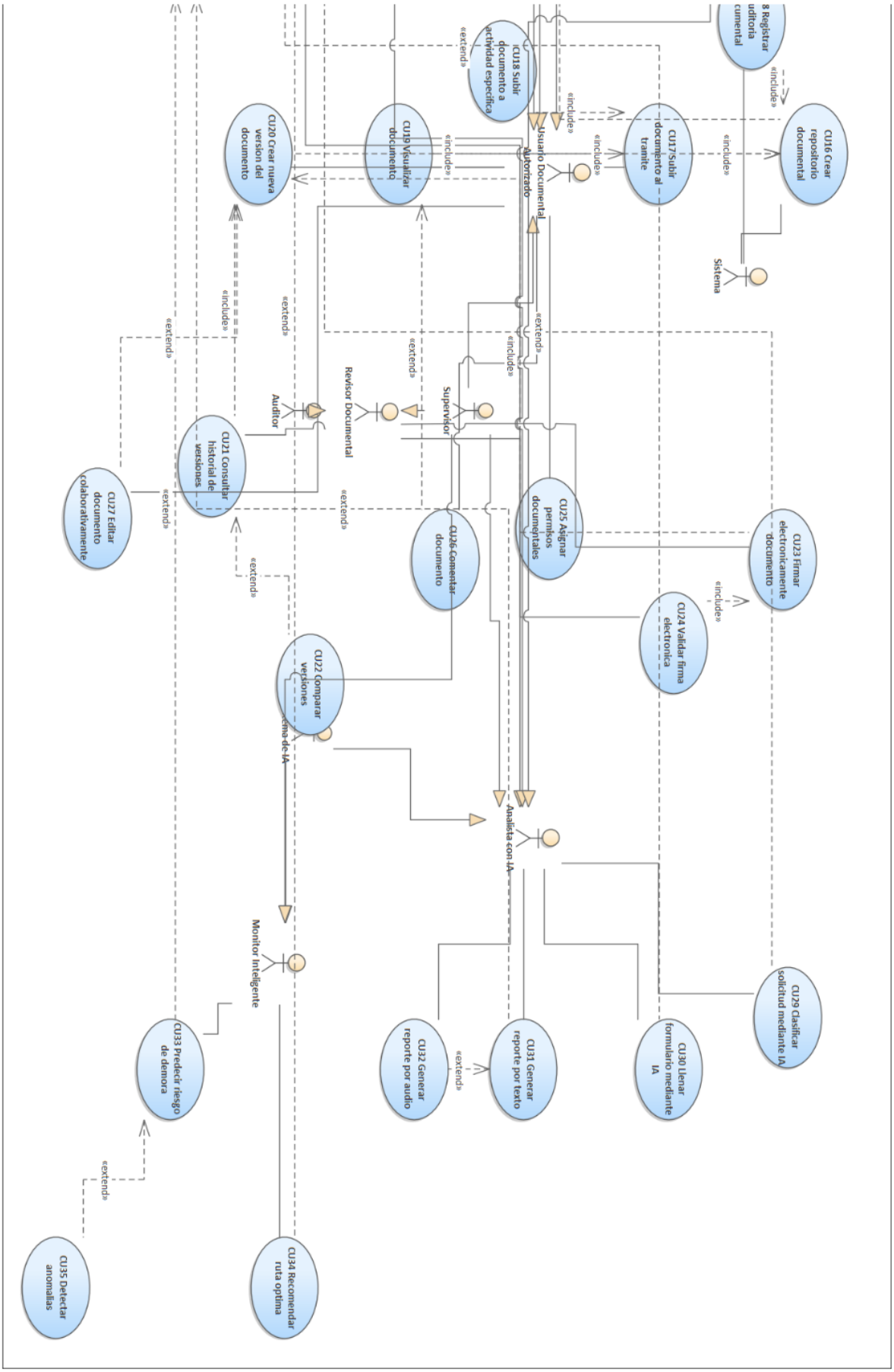

uc General Ciclo 2 - Casos de Uso
CU16 Crear repositorio documental
«include»
«include»
CU17 Subir documento al tramite
Sistema
CU19 Visualizar documento
«include»
Administrador
«include»
«include»
«include»
CU18 Subir documento a actividad especifica
«extend»
CU20 Crear nueva version del documento
Cliente o Solicitante
Diseniador de Procesos
Colaborador Documental
CU26 Comentar documento
«extend»
«extend»
CU21 Consultar historial de versiones
«include»
CU27 Editar documento colaborativamente


CU23 Firmar electronicamente documento
«include»
CU24 Validar firma electronica
«include»
«include»
CU28 Registrar auditoria documental
CU25 Asignar permisos documentales
CU34 Recomendar ruta optima
Revisor Documental
Monitor Inteligente
Supervisor
Administrador Documental
Auditor
Sistema de IA
«extend»
Analista con IA
CU31 Generar reporte por texto
«extend»
«extend»
CU32 Generar reporte por audio


CU24 Validar firma electronica
CU29 Clasificar solicitud mediante IA
CU34 Recomendar ruta optima
«extend»
Monitor Inteligente
CU35 Detectar anomalias
CU30 Llenar formulario mediante IA
«extend»
Sistema de IA
«extend»
CU33 Predecir riesgo de demora
extend»
reporte
lio

General

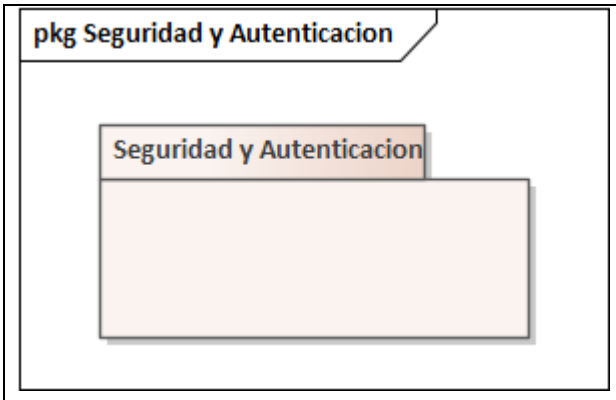
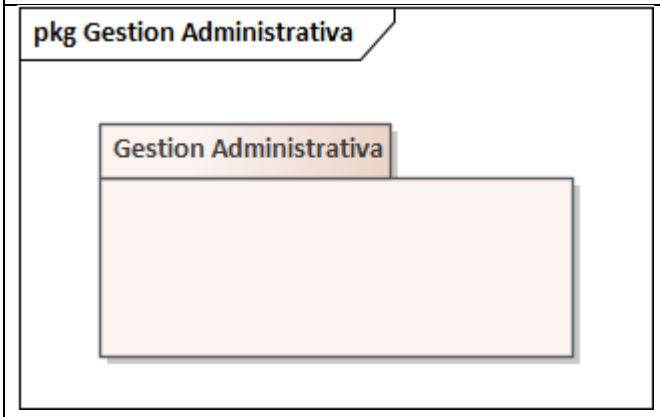

uc General Todo el Sistema - Casos de Uso
CU1 Iniciar sesion
CU2 Cerrar sesion
CU3 Gestionar usuarios
CU4 Gestionar roles y permisos
CU5 Gestionar departamentos
CU6 Gestionar politicas de negocio
CU7 Diseniar diagrama de actividades
CU8 Gestionar formularios dinamicos
CU9 Crear tramite
CU10 Visualizar bandeja de tareas
CU11 Reclamar tarea
CU12 Llenar formulario de tramite
CU13 Derivar tramite
CU14 Consultar estado del tramite
CU15 Generar KPI basico
Administrador
Diseñador de Procesos
Gestor de Configuracion de Procesos
Funcionario
Cliente o Solicitante
Participante del Tramite
«include»
«extend»

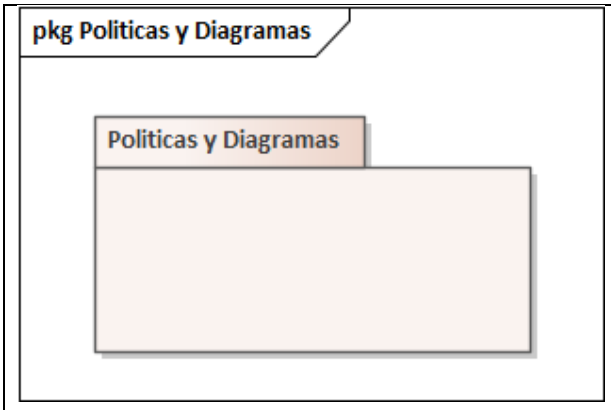
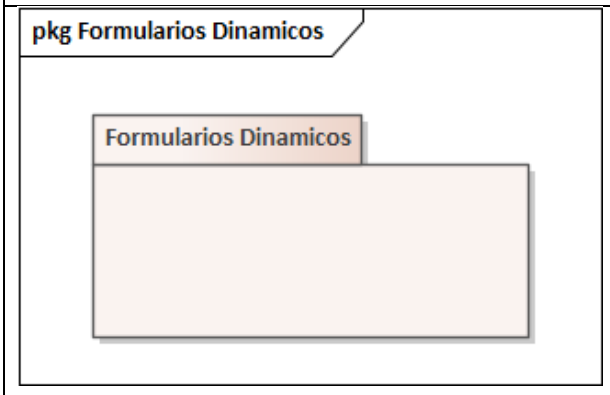

CU16 Crear repositorio documental
CU17Subir documento al tramite
CU18 Subir documento a actividad especifica
CU19 Visualizar documento
CU20 Crear nueva version del documento
CU21 Consultar historial de versiones
CU22 Comparar versiones
CU23 Firmar electronicamente documento
CU24 Validar firma electronica
CU25 Asignar permisos documentales
CU26 Comentar documento
CU27 Editar documento colaborativamente
CU29 Clasificar solicitud mediante IA
CU30 Llenar formulario mediante IA
CU31 Generar reporte por texto
CU32 Generar reporte por audio
CU33 Predecir riesgo de demora
CU34 Recomendar ruta optima
CU35 Detectar anomalias
Usuario Documental Autorizado
Sistema
Auditor
Revisor Documental
Supervisor
Analista con IA
Monitor Inteligente
«include»
«extend»

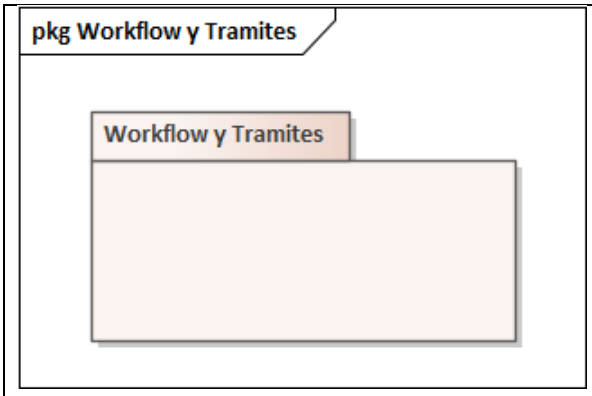
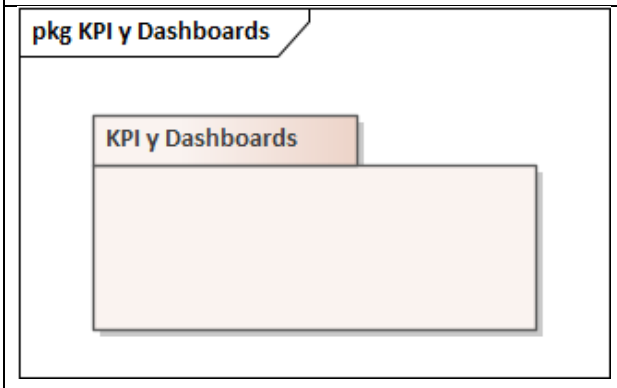
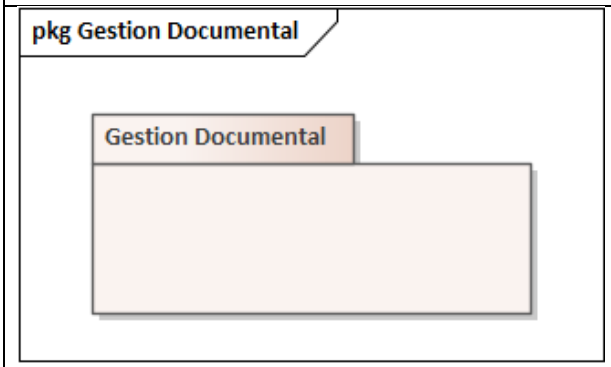
2.2 FLUJO DE TRABAJO – ANÁLISIS

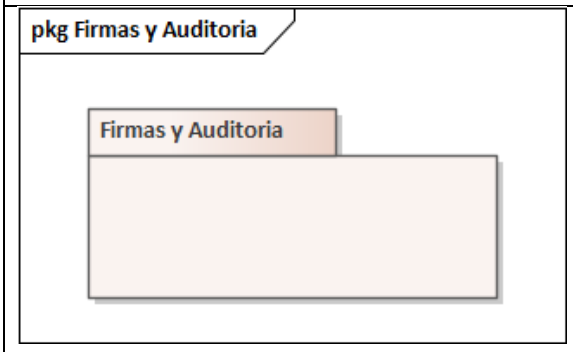
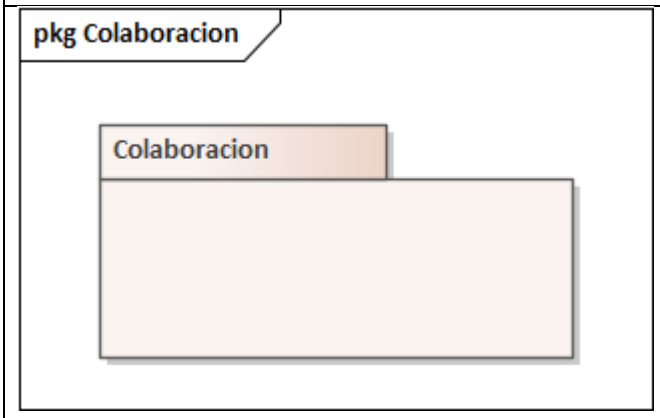
2.2.1 Análisis de Arquitectura

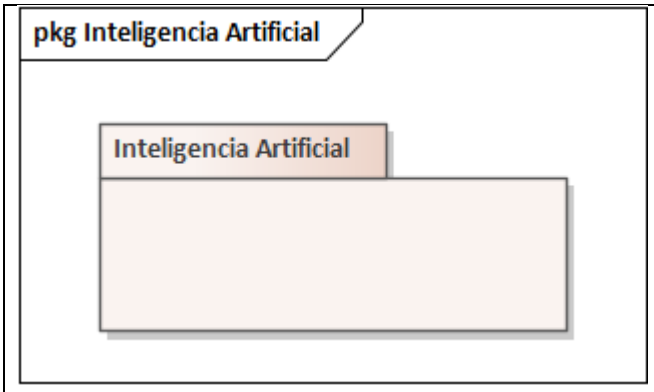
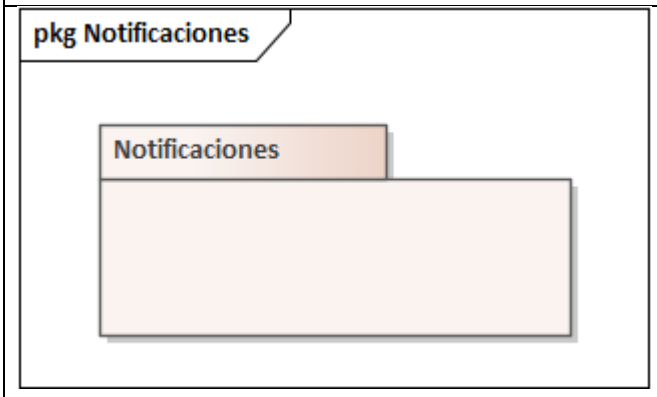
2.2.1.1 Identificar Paquetes

	Encargado del control de acceso al sistema. Gestiona el inicio de sesión, cierre de sesión, generación y validación de tokens JWT. Garantiza que solo usuarios autenticados y con los permisos adecuados accedan a las funcionalidades.
	Responsable de la administración de la estructura organizacional y los usuarios del sistema. Incluye el CRUD de usuarios, roles, permisos y departamentos. Define qué puede hacer cada rol y a qué departamento pertenece cada funcionario.

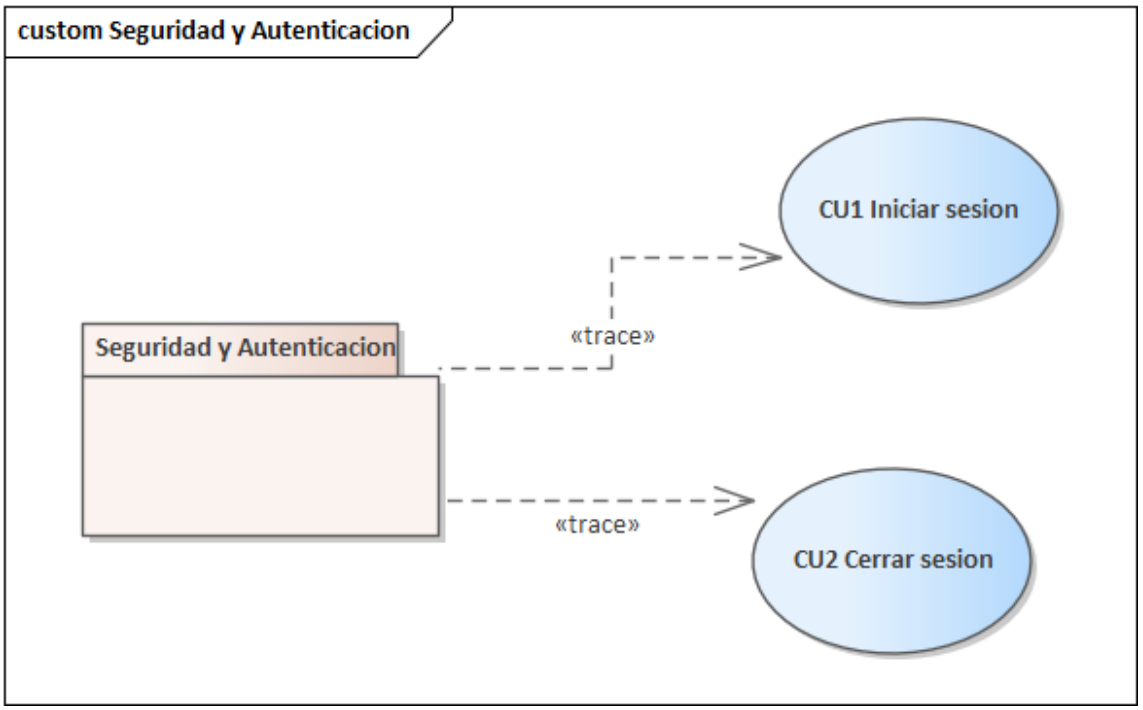
	Gestiona el ciclo de vida completo de las políticas de negocio: creación, edición, versionado, publicación y archivado. Contiene el editor visual de diagramas de actividades UML, donde se modelan los flujos de trabajo, las condiciones de guarda, las asignaciones de responsables (swimlanes) y la asociación con formularios.
	Administra las definiciones de formularios configurables. Permite crear, versionar y eliminar formularios compuestos por campos de diversos tipos (texto, número, fecha, selector, archivo) con reglas de validación. Estos formularios se asocian posteriormente a las actividades del workflow.

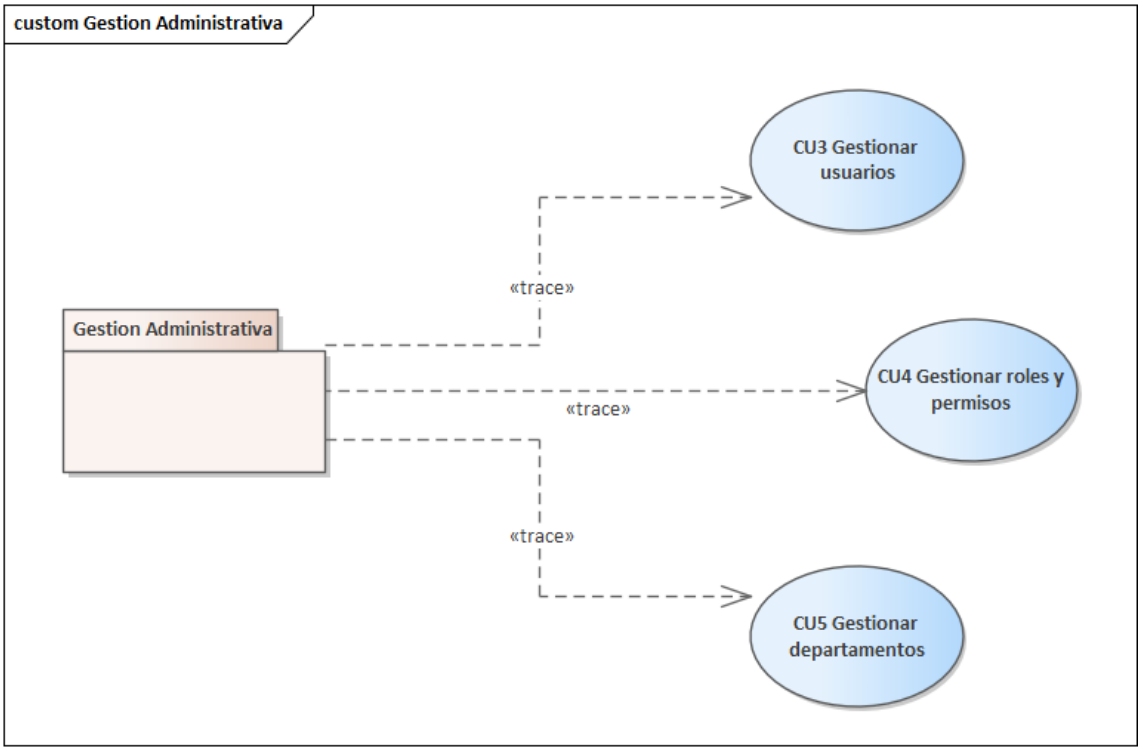
	Contiene el núcleo ejecutivo del sistema. Incluye la creación de trámites, la gestión de la bandeja de tareas (worklist), la reclamación y ejecución de tareas, el llenado de formularios y la derivación de trámites mediante el motor de reglas. También abarca la consulta del estado de los trámites.
	Proporciona indicadores clave de desempeño para supervisores y administradores. Calcula y muestra métricas como totales de trámites, tiempos promedio, distribución por departamento y funcionario, permitiendo evaluar la eficiencia de los procesos.
	Administra los repositorios documentales asociados a cada trámite. Gestiona la subida, visualización, versionado, comparación y eliminación de documentos. Controla los permisos de acceso a los

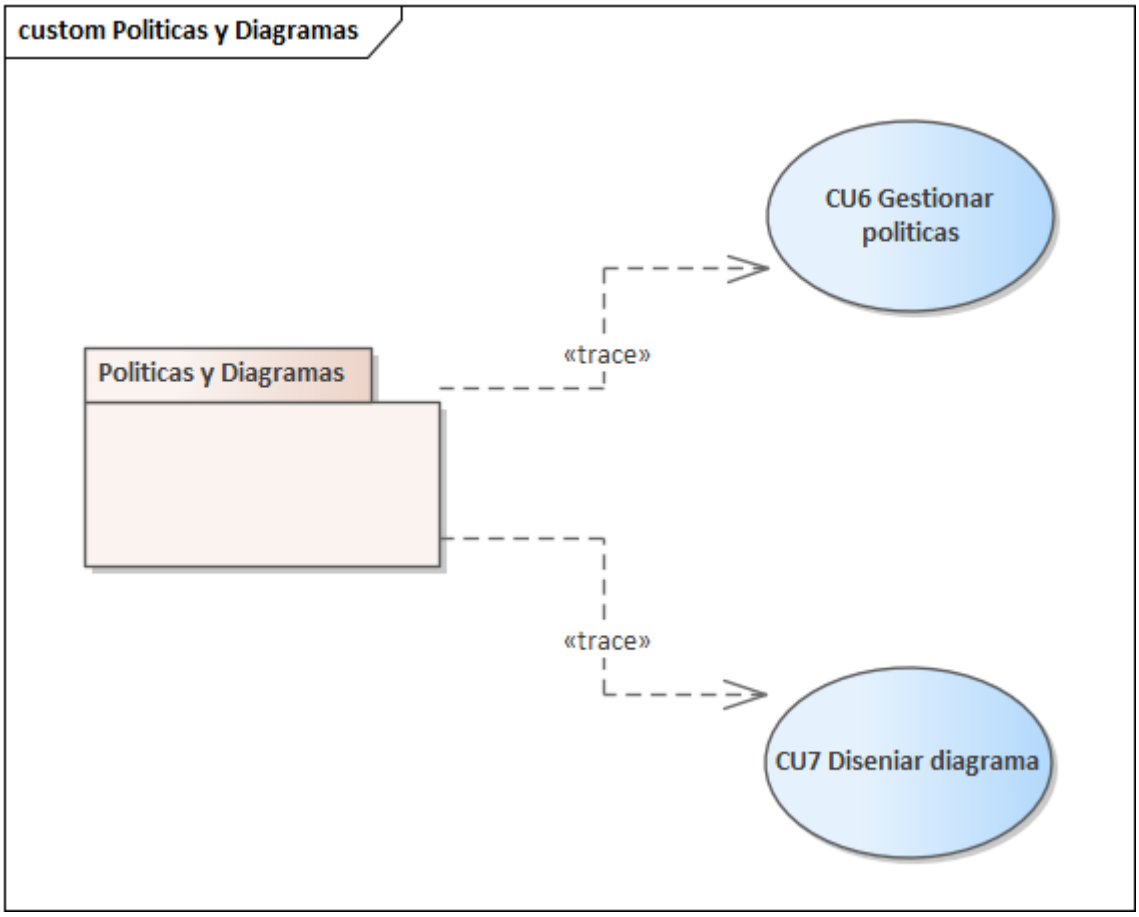
	documentos y permite comentarios sobre los mismos. Los archivos físicos se almacenan en Amazon S3.
	Responsable de la validación documental mediante firmas electrónicas y del registro de trazabilidad. Incluye la firma de documentos, la validación de firmas existentes y el registro automático de todas las acciones relevantes (creación, modificación, eliminación, firma, visualización) en el sistema.
	Proporciona las capacidades de trabajo simultáneo sobre diagramas y documentos. Utiliza WebSockets para la sincronización en tiempo real entre múltiples usuarios, permitiendo edición concurrente, visualización de cursores y resolución de conflictos.

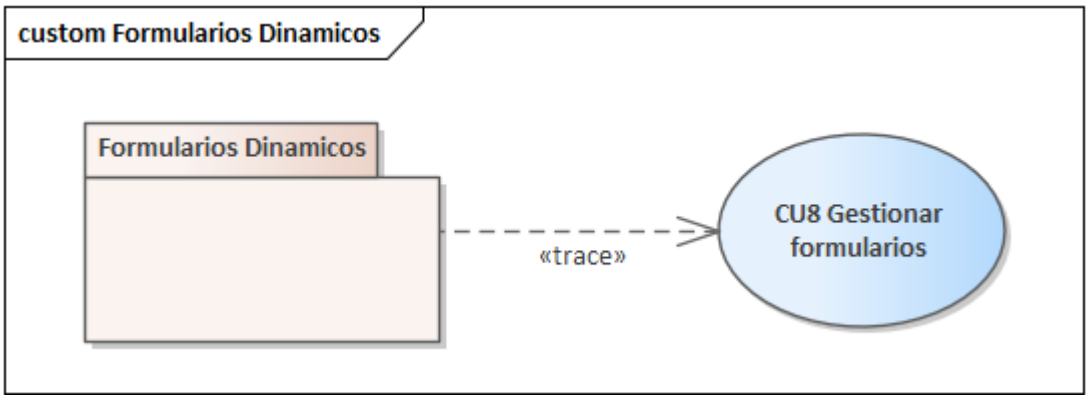
	Agrupa los servicios inteligentes del sistema: clasificación automática de solicitudes por texto o audio (PLN/ASR), llenado automático de formularios, generación de reportes mediante lenguaje natural, predicción de riesgos de demora, recomendación de rutas óptimas y detección de anomalías. Se apoya en modelos de TensorFlow.
	Gestiona el envío de alertas y notificaciones a los usuarios. Utiliza WebSockets para actualizar en tiempo real la bandeja de tareas de los funcionarios y Firebase Cloud Messaging para enviar notificaciones push a los clientes móviles cuando un trámite cambia de estado.

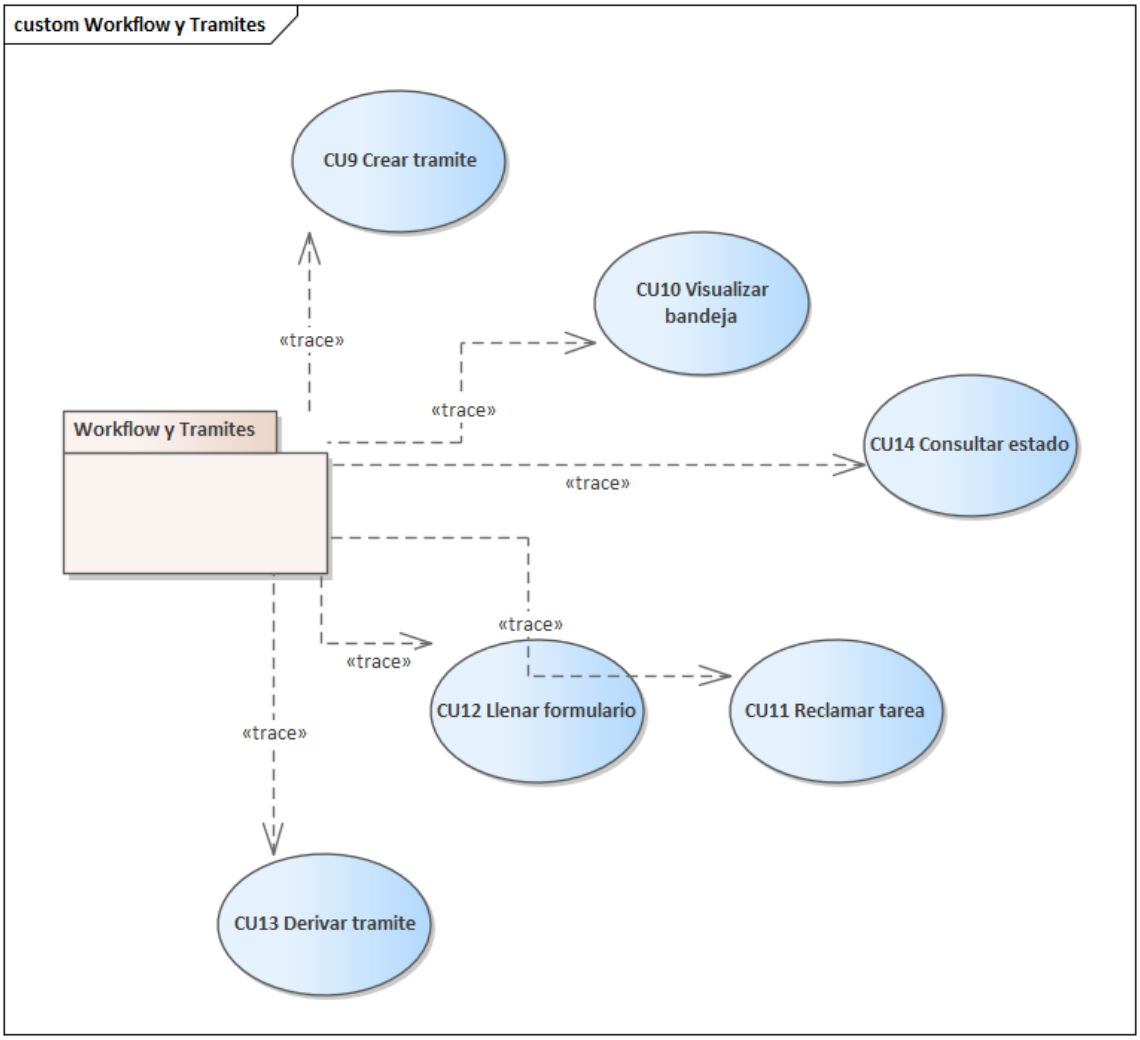
2.2.1.2 Relacionar Paquetes y Casos de Uso

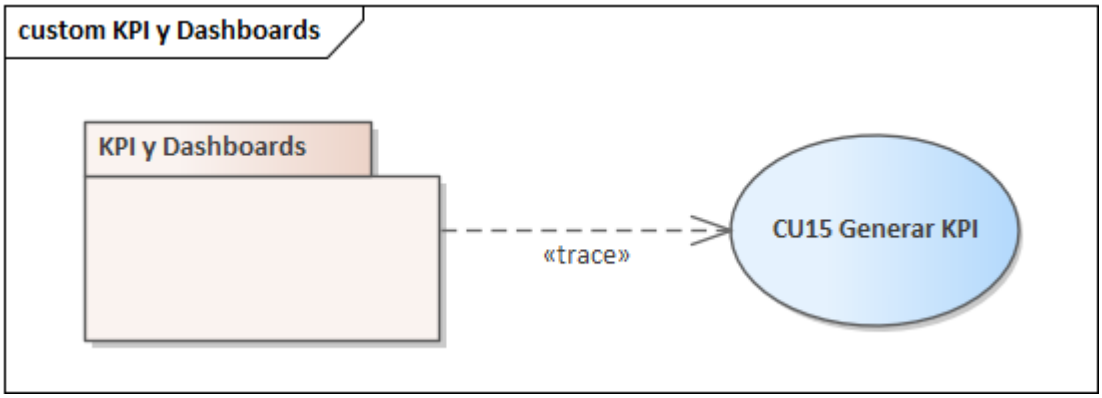


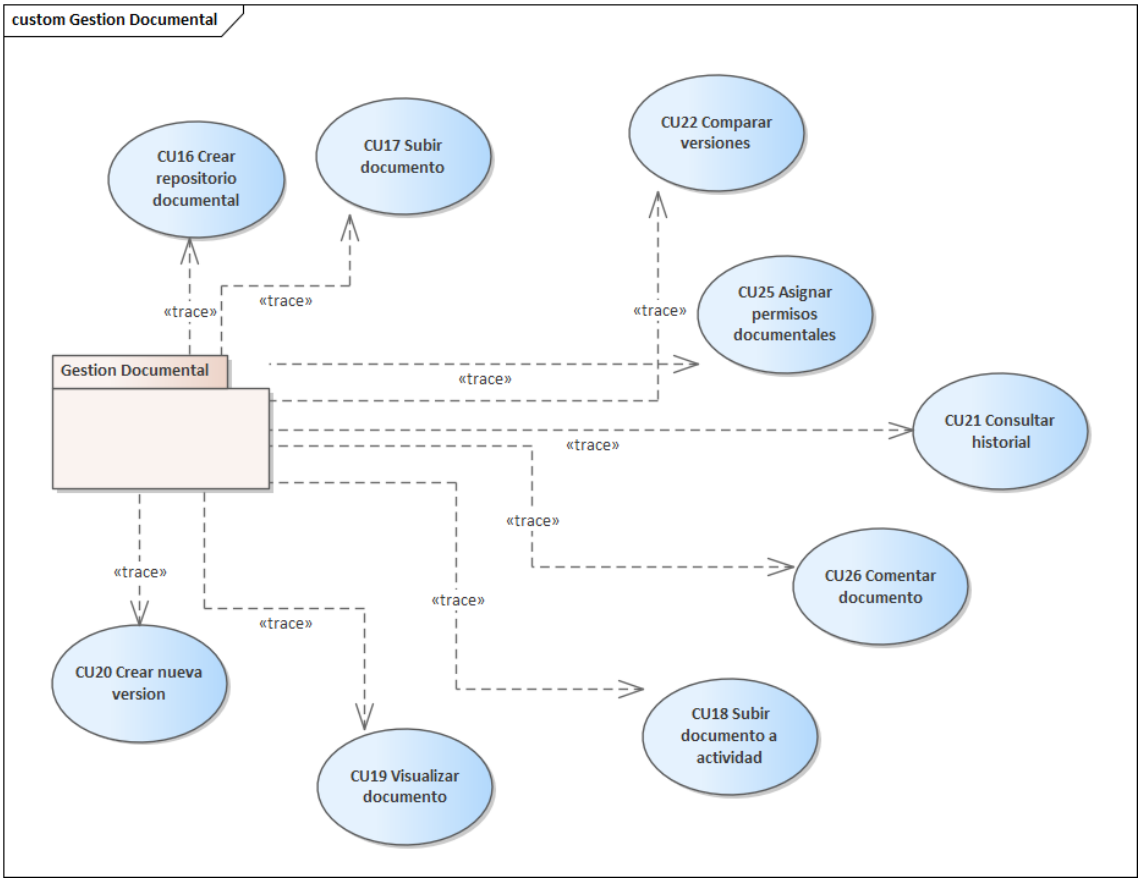


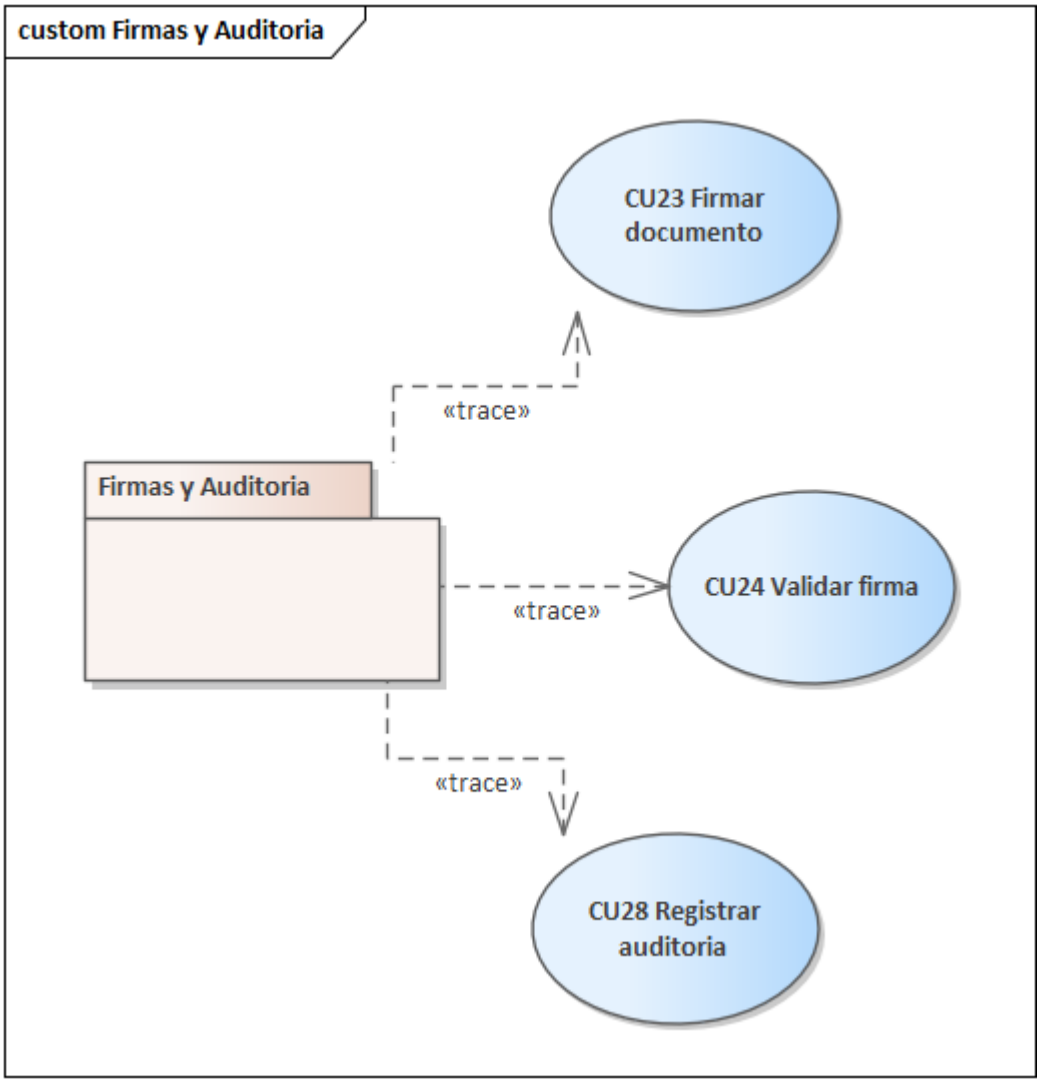

custom Politicas y Diagramas
CU6 Gestionar politicas
«trace»
Politicas y Diagramas
«trace»
CU7 Diseniar diagrama

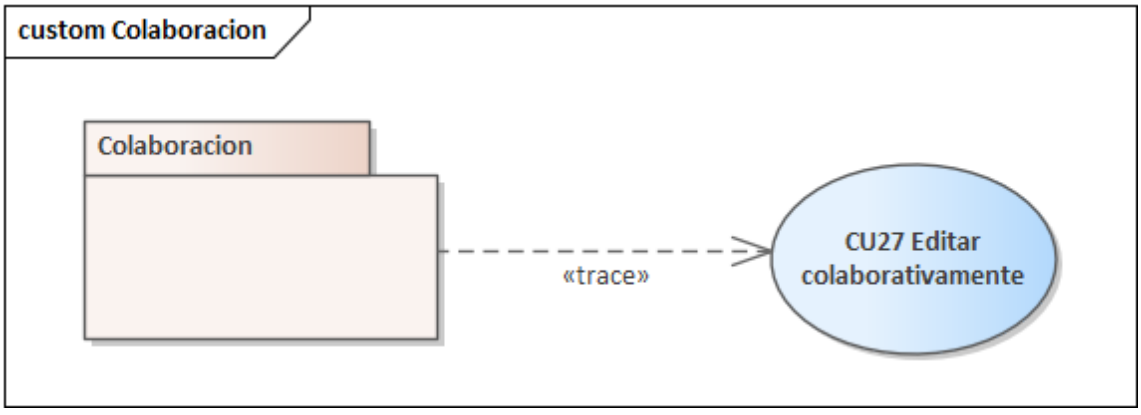

custom Formularios Dinamicos
Formularios Dinamicos
«trace»
CU8 Gestionar formularios

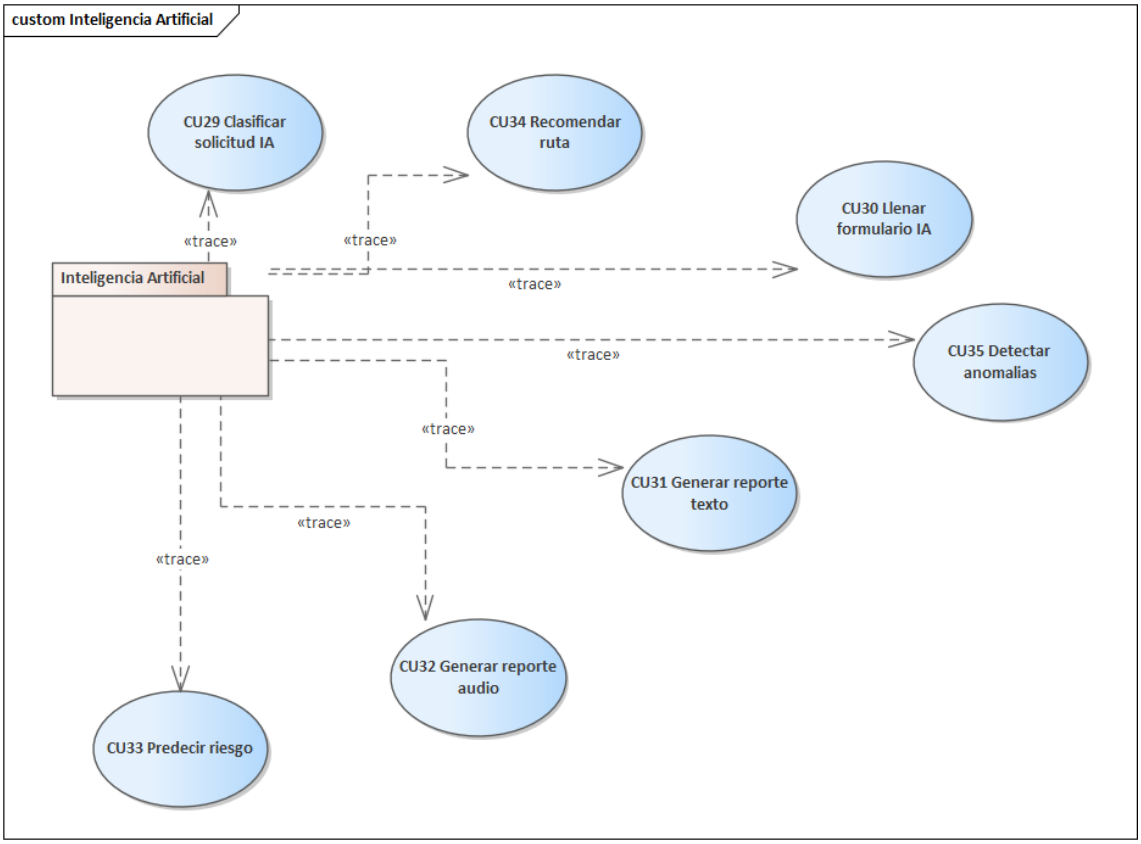

custom Workflow y Tramites
CU9 Crear tramite
CU10 Visualizar bandeja
«trace»
«trace»
Workflow y Tramites
CU14 Consultar estado
«trace»
«trace»
«trace»
CU12 Llenar formulario
CU11 Reclamar tarea
«trace»
CU13 Derivar tramite

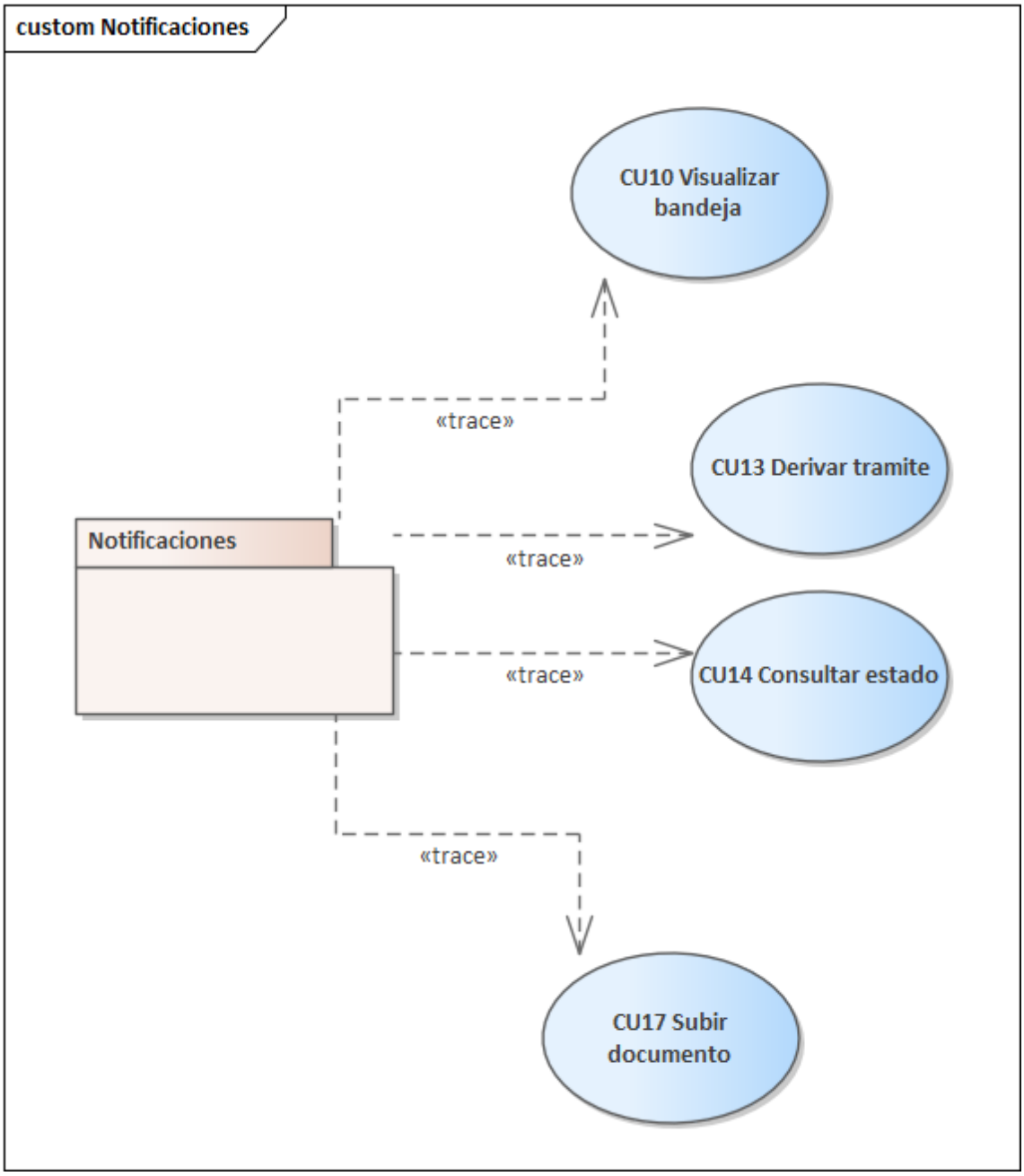

custom KPI y Dashboards
KPI y Dashboards
«trace»
CU15 Generar KPI


custom Gestion Documental
CU16 Crear repositorio documental
CU17 Subir documento
CU22 Comparar versiones
CU25 Asignar permisos documentales
Gestion Documental
CU21 Consultar historial
CU26 Comentar documento
CU20 Crear nueva version
CU19 Visualizar documento
CU18 Subir documento a actividad
«trace»


custom Firmas y Auditoria
CU23 Firmar documento
«trace»
Firmas y Auditoria
CU24 Validar firma
«trace»
«trace»
CU28 Registrar auditoria

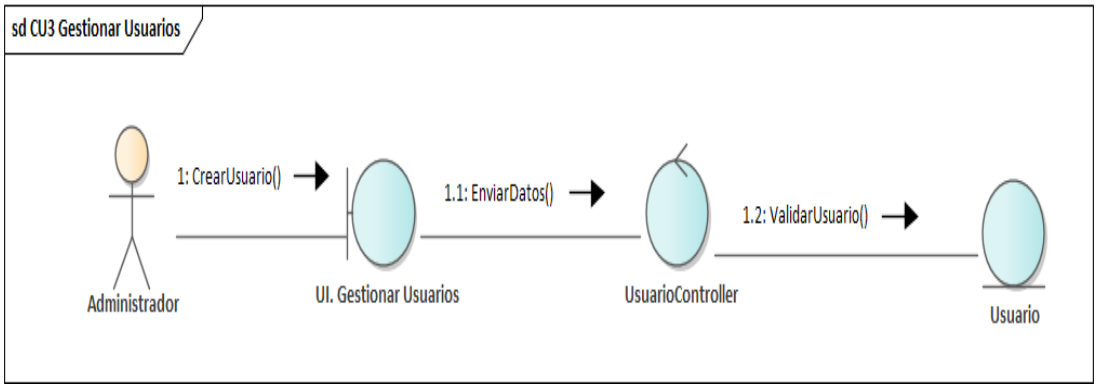

custom Colaboracion
Colaboracion
«trace»
CU27 Editar colaborativamente

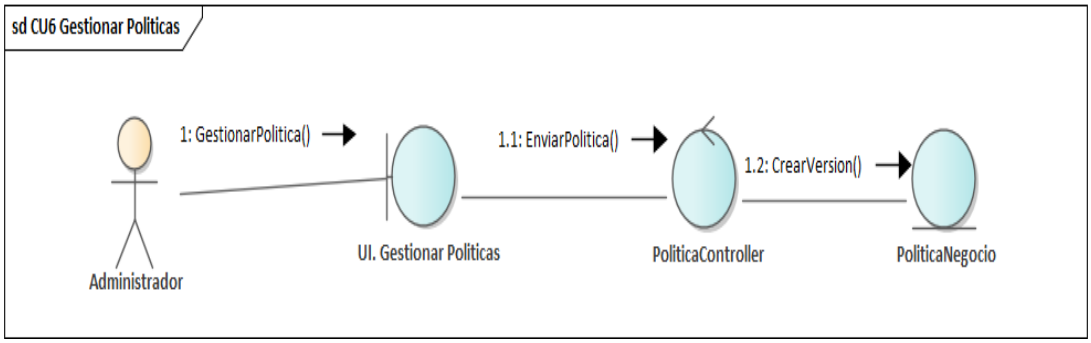

custom Inteligencia Artificial
CU29 Clasificar solicitud IA
CU34 Recomendar ruta
CU30 Llenar formulario IA
Inteligencia Artificial
CU35 Detectar anomalias
CU31 Generar reporte texto
CU32 Generar reporte audio
CU33 Predecir riesgo
«trace»
«trace»
«trace»
«trace»
«trace»
«trace»
«trace»

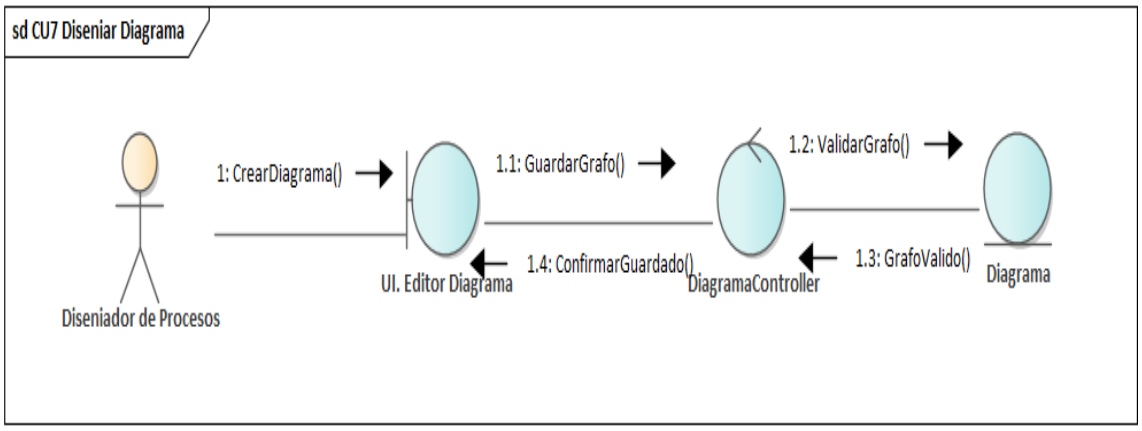

custom Notificaciones
CU10 Visualizar bandeja
«trace»
CU13 Derivar tramite
Notificaciones
«trace»
«trace»
CU14 Consultar estado
«trace»
CU17 Subir documento

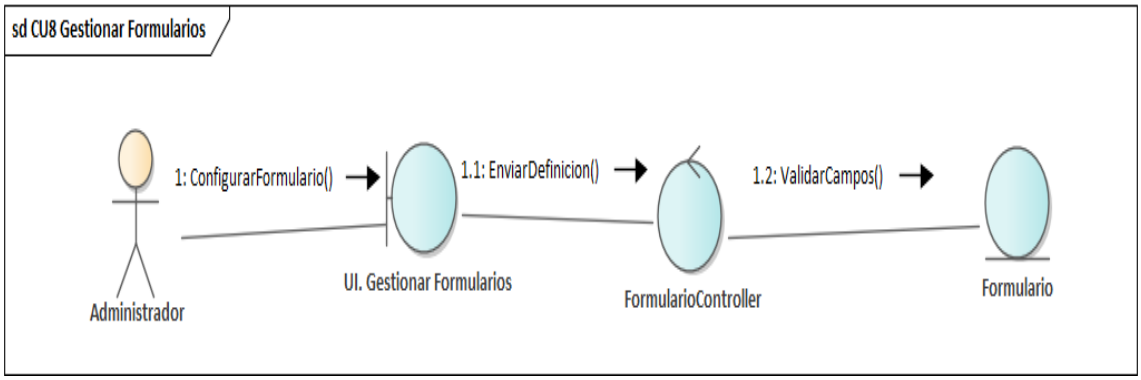
2.2.2 Análisis de Casos de Uso

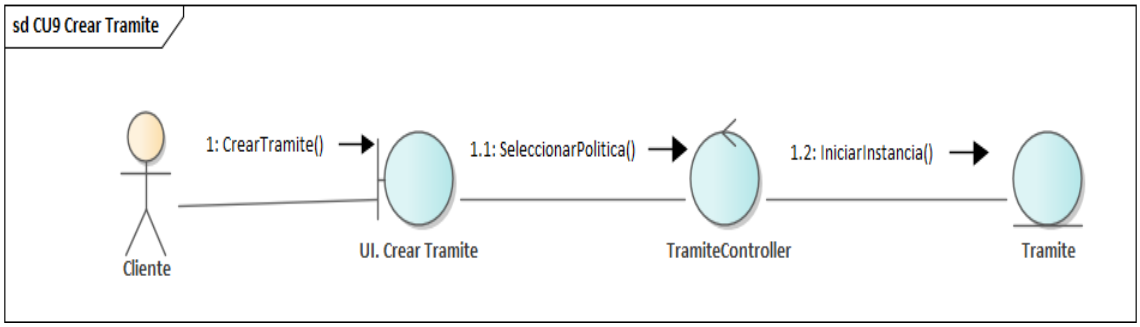
2.2.2.1 Diagrama de Comunicación

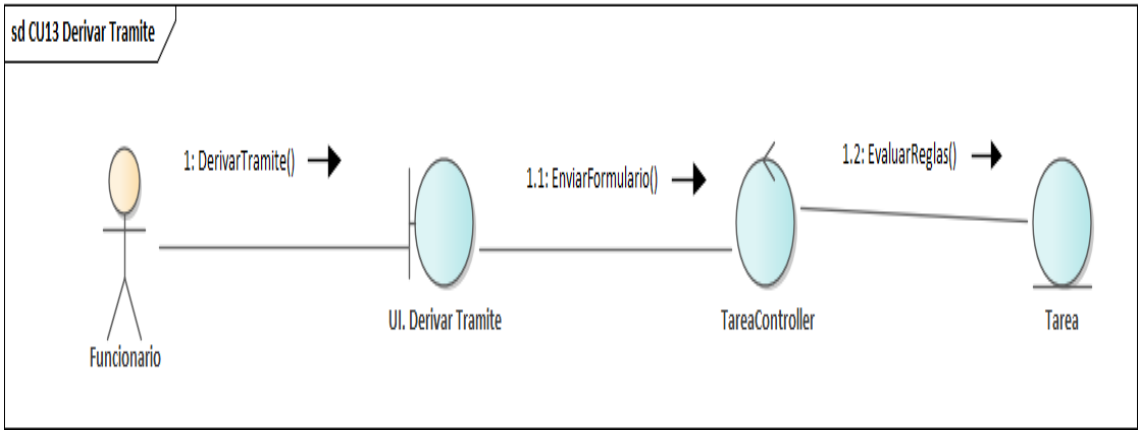


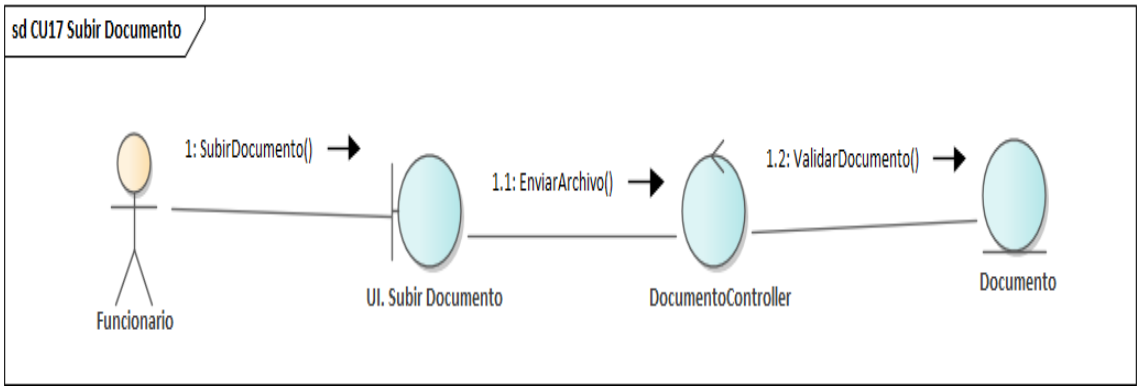


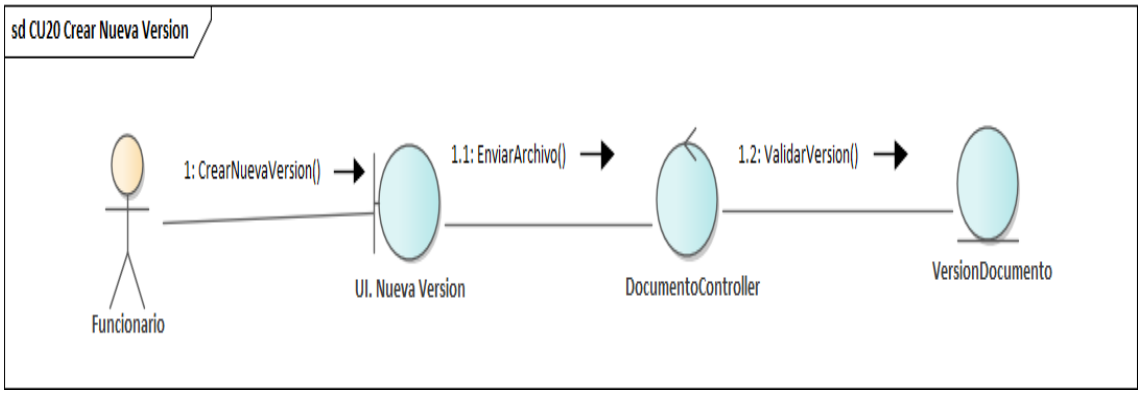


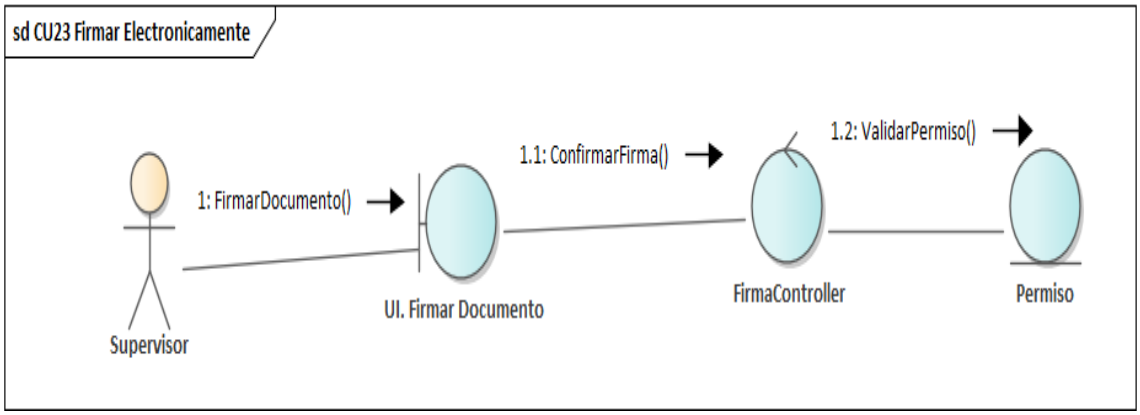

sd CU8 Gestionar Formularios
1: ConfigurarFormulario()
1.1: EnviarDefinicion()
1.2: ValidarCampos()
Administrador
UI. Gestionar Formularios
FormularioController
Formulario

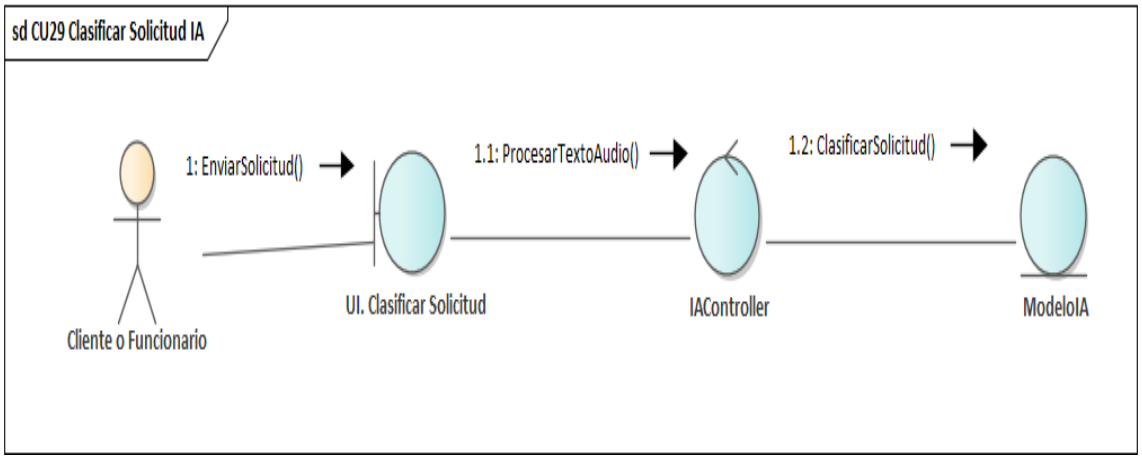

sd CU9 Crear Tramite
1: CrearTramite()
1.1: SeleccionarPolitica()
1.2: IniciarInstancia()
Cliente
UI. Crear Tramite
TramiteController
Tramite

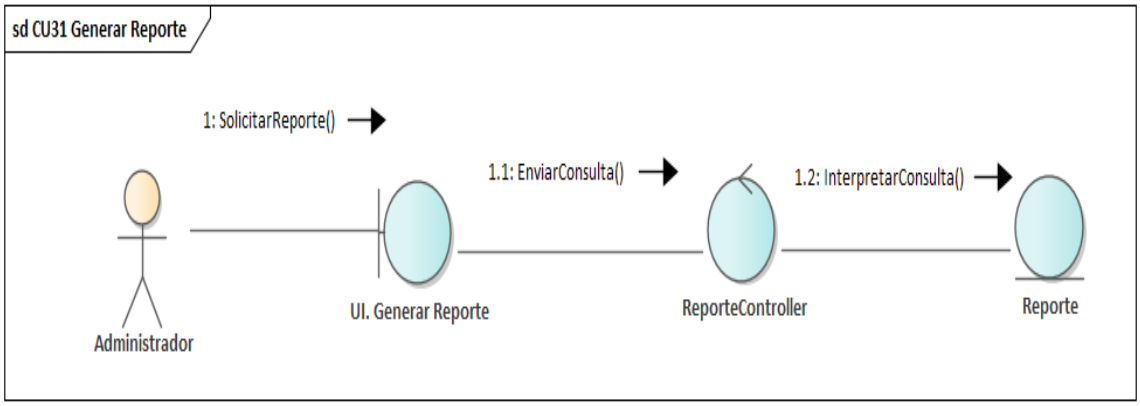

sd CU13 Derivar Tramite
1: DerivarTramite()
1.1: EnviarFormulario()
1.2: EvaluarReglas()
Funcionario
UI. Derivar Tramite
TareaController
Tarea

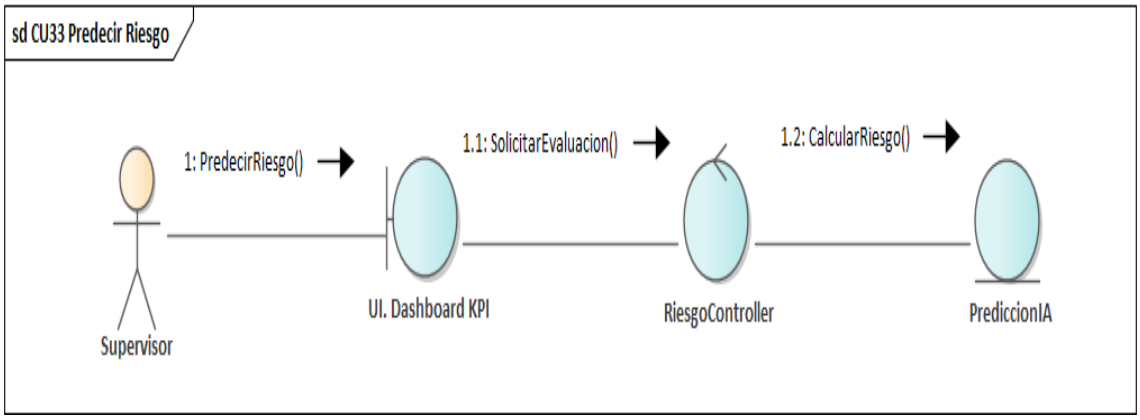

sd CU17 Subir Documento
1: SubirDocumento()
1.1: EnviarArchivo()
1.2: ValidarDocumento()
Funcionario
UI. Subir Documento
DocumentoController
Documento


sd CU20 Crear Nueva Version
1: CrearNuevaVersion()
1.1: EnviarArchivo()
1.2: ValidarVersion()
Funcionario
UI. Nueva Version
DocumentoController
VersionDocumento

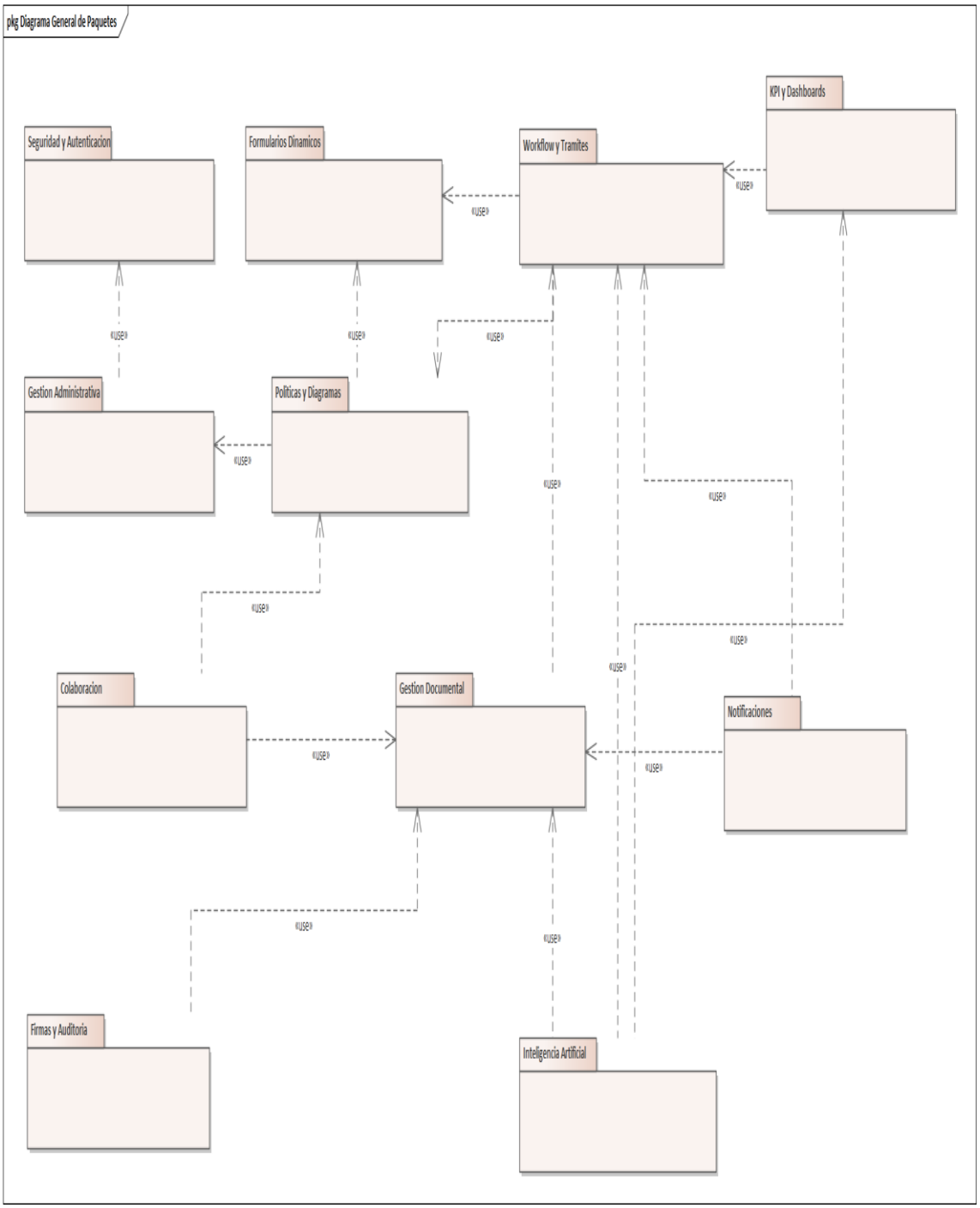

sd CU23 Firmar Electronicamente
1: FirmarDocumento()
1.1: ConfirmarFirma()
1.2: ValidarPermiso()
Supervisor
UI. Firmar Documento
FirmaController
Permiso


sd CU29 Clasificar Solicitud IA
1: EnviarSolicitud()
1.1: ProcesarTextoAudio()
1.2: ClasificarSolicitud()
Cliente o Funcionario
UI. Clasificar Solicitud
IAController
ModeloIA


sd CU31 Generar Reporte
1: SolicitarReporte()
1.1: EnviarConsulta()
1.2: InterpretarConsulta()
Administrador
UI. Generar Reporte
ReporteController
Reporte


sd CU33 Predecir Riesgo
1: PredecirRiesgo()
1.1: SolicitarEvaluacion()
1.2: CalcularRiesgo()
Supervisor
UI. Dashboard KPI
RiesgoController
PrediccionIA

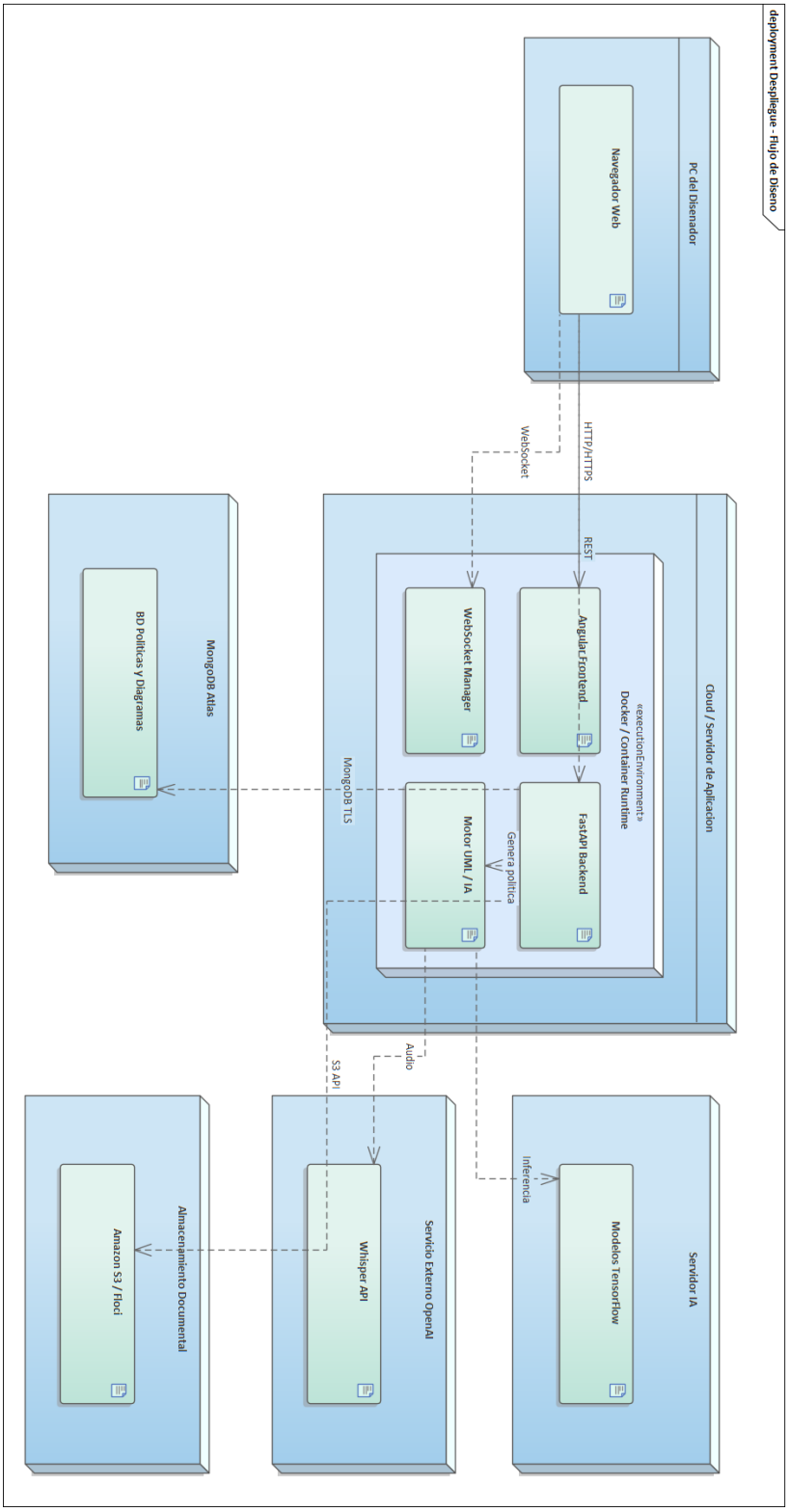
2.2.2.2 Análisis de Paquete



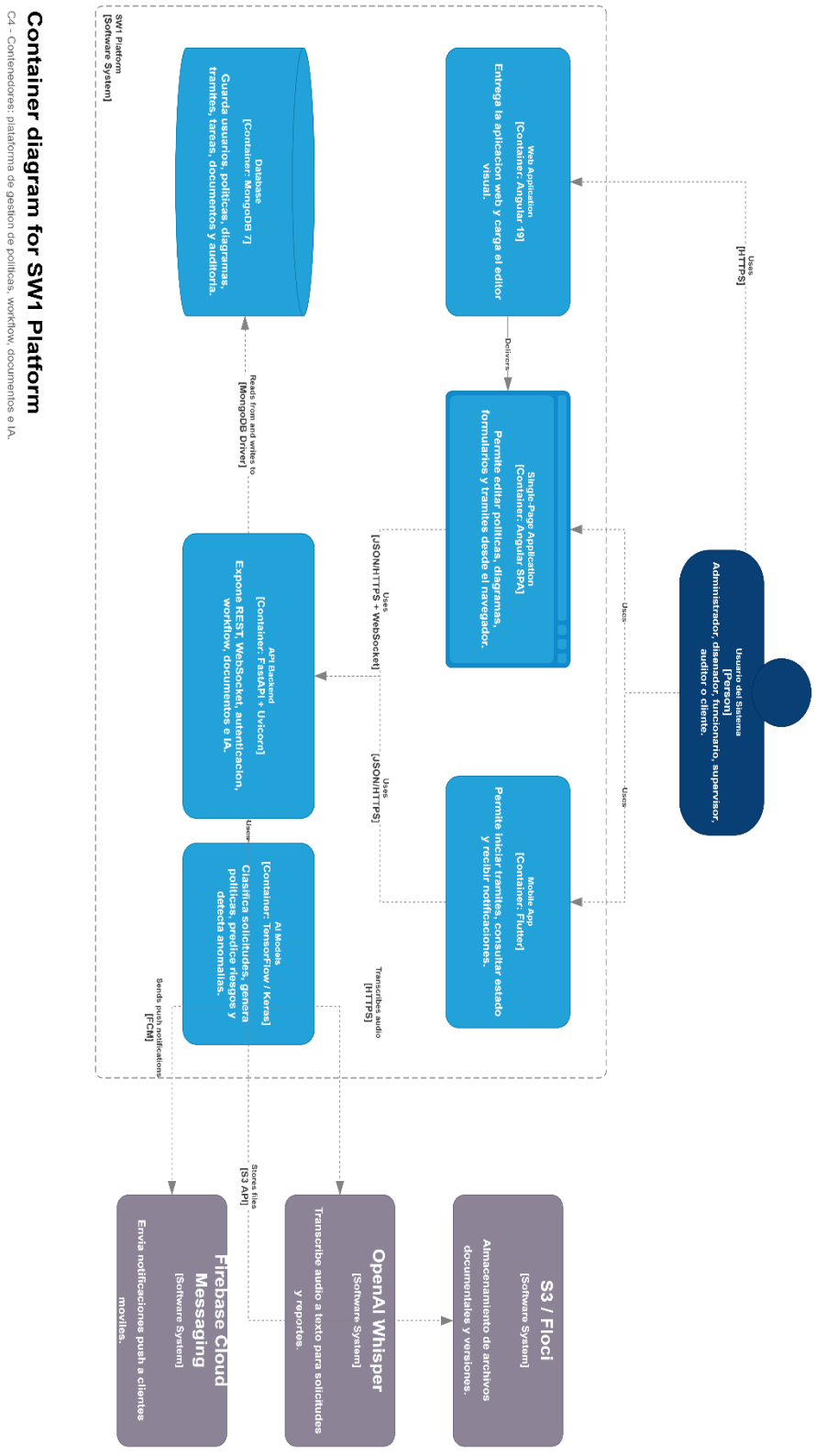
2.3 FLUJO DE TRABAJO – DISEÑO

2.3.1 Diseño de Arquitectura

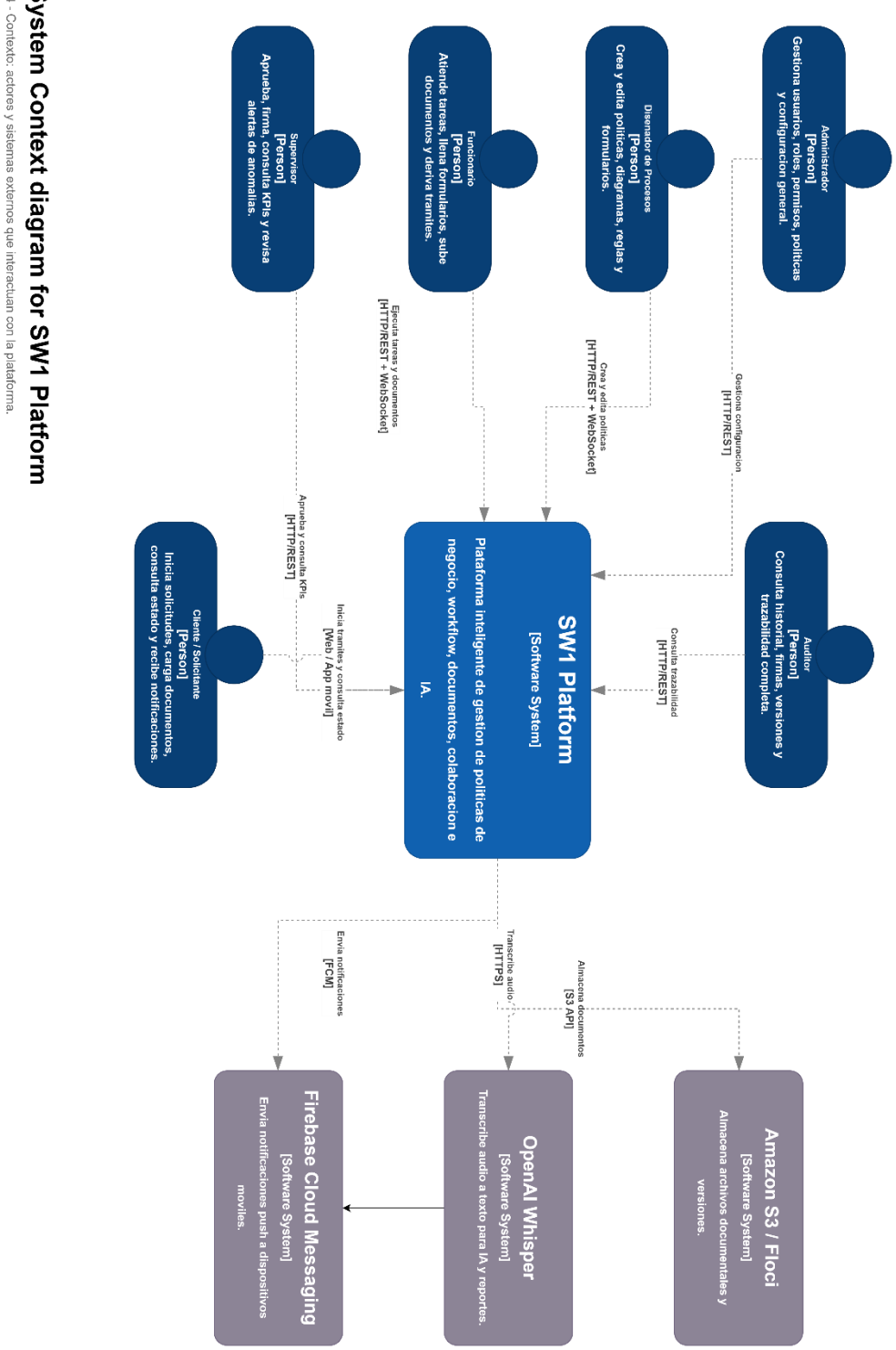
2.3.1.1 Diseño Físico (Diagrama de despliegue)



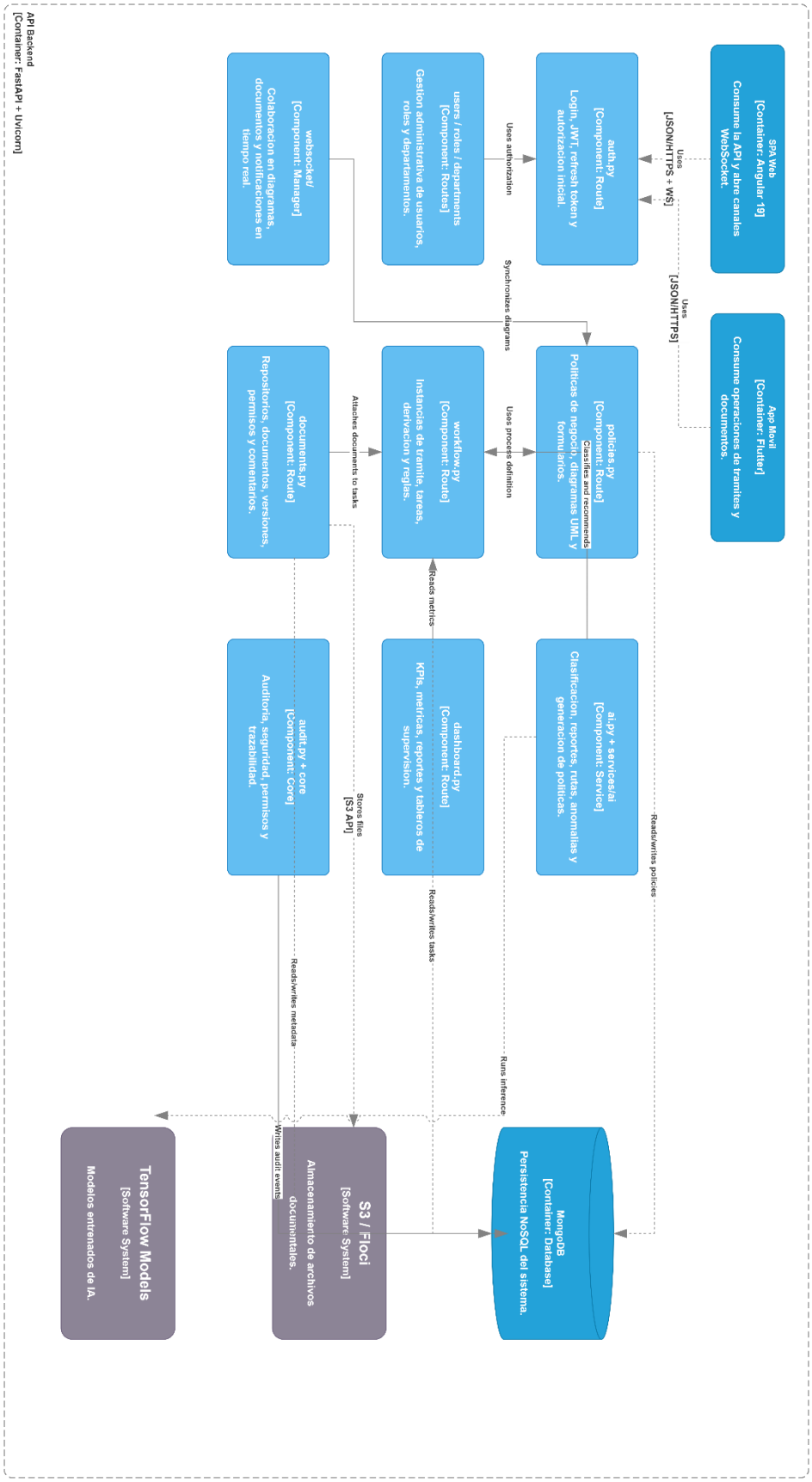
2.3.1.2 Diagrama C4 - Contexto



2.3.1.3 Diagrama C4 – Contenedores



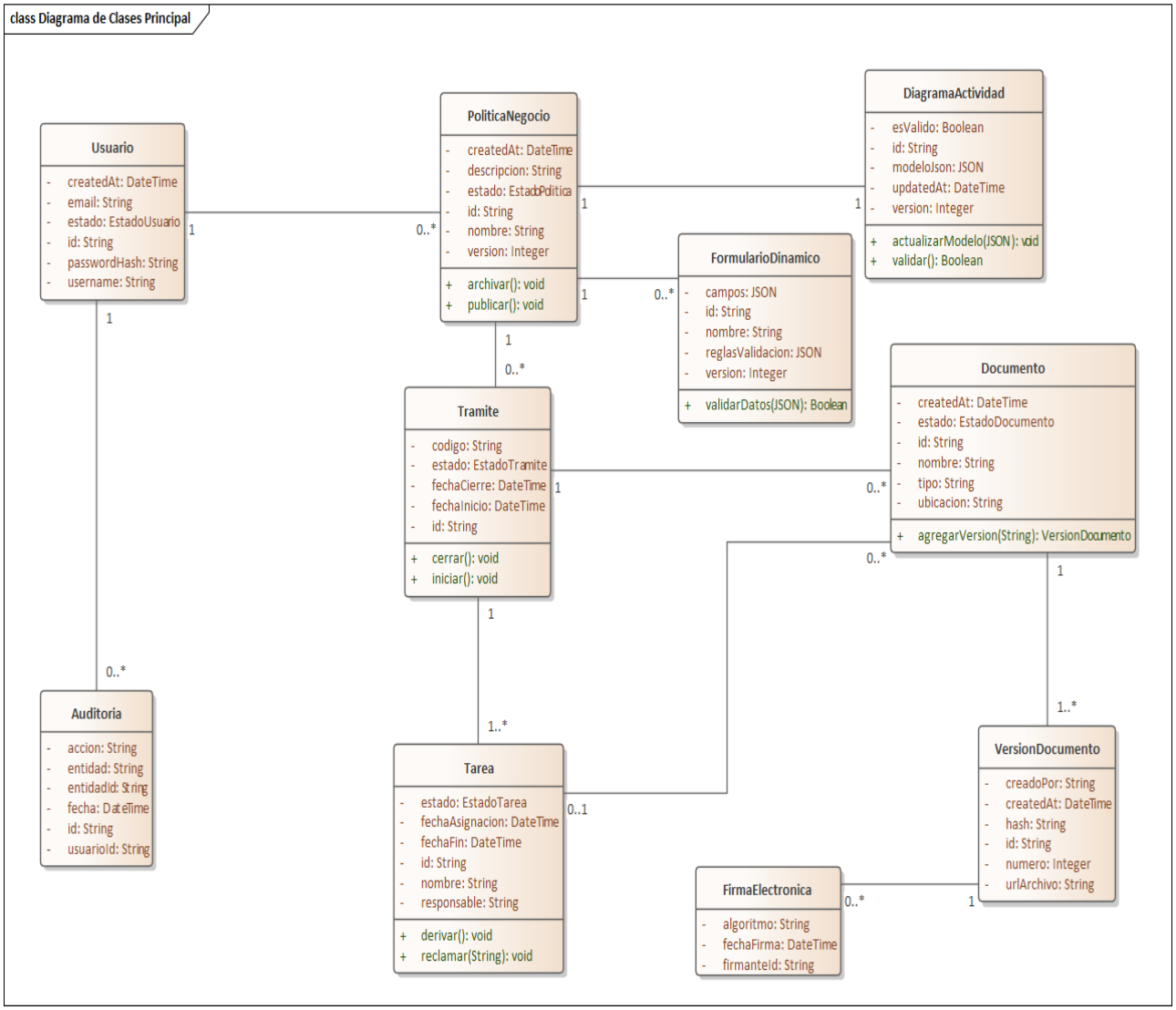
2.3.1.4 Diagrama C4 - Componentes



2.3.2 Diseño de Datos

2.3.2.1 Diseño de Datos Lógicos

2.3.2.1.1 Diagrama de Clases



2.3.2.1.2 Mapeo

Clase	Colección	Tipo de documento	Campos (tipo BSON)	Referencias / Notas
Usuario	usuarios	Documento	_id (ObjectId), username (string), email (string), passwordHash (string), estado (string enum), createdAt (date)	email único, username único

Auditoria	auditorias	Documento	_id, accion (string), entidad (string), entidadId (string), fecha (date), usuarioId (ObjectId)	usuarioId referencia a usuarios._id
Trama	tramas	Documento	_id, código (string), estado (string enum), fechaInicio (date), fechaCierre (date)	código único
PolíticaNegocio	politicasNegocio	Documento	_id, nombre (string), descripcion (string), version (int), estado (string enum), createdAt (date)	
FormularioDinamico	formulariosDinamicos	Documento	_id, nombre (string), descripcion (string), campos (object), estado (string enum), createdAt (date), updatedAt (date)	campos guarda la definición dinámica (JSON)
Tramite	tramites	Documento	_id, código (string), estado (string enum), fechaInicio (date), fechaCierre (date), formularioId (ObjectId, opc.)	código único, formularioId referencia a formulariosDinamicos
Tarea	tareas	Documento	_id, nombre (string), estado (string enum), fechaAsignacion (date), fechaFin (date), responsable (ObjectId), tramiteId (ObjectId, opc.), diagramaActividadId (ObjectId, opc.)	responsable referencia a usuarios._id
FirmaElectronica	firmasElectronicas	Documento	_id, algoritmo (string), fechaFirma (date), firmanteId (ObjectId), documentoId (ObjectId, opc.), tramiteId (ObjectId, opc.)	firmanteId referencia a usuarios._id
DiagramaActividad	diagramasActividad	Documento	_id, esValido (bool), modelo (object), version (int), updatedAt (date)	
Documento	documentos	Documento con array embebido	_id, nombre (string), tipo (string), ubicacion (string), estado (string enum), createdAt (date), versiones (array de subdocumentos)	Cada versión tiene: _id, numero (int), creadoPor (ObjectId), createdAt (date), hash (string), urlArchivo (string)

2.3.2.1.3 Normalización

Las Clases ya se encuentran completamente normalizadas

2.3.2.2 Diseño de Datos Físico

2.3.2.2.1 Script

```
// 1. Seleccionar o crear la base de datos
use sistemaGestion;

// 2. Eliminar colecciones existentes (opcional, solo para
limpiar pruebas)
db.usuarios.drop();
db.auditorias.drop();
db.tramas.drop();
db.politicasNegocio.drop();
db.formulariosDinamicos.drop();
db.tramites.drop();
db.tareas.drop();
db.firmasElectronicas.drop();
db.diagramasActividad.drop();
db.documentos.drop();

// 3. Crear colección 'usuarios' con validación
db.createCollection("usuarios", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["username", "email", "passwordHash",
"estado", "createdAt"],
         properties: {
            username: { bsonType: "string", description:
"Debe ser único" },
            email: { bsonType: "string", pattern:
"^.+@.+$", description: "Email válido único" },
            passwordHash: { bsonType: "string" },
            estado: { enum: ["ACTIVO", "INACTIVO",
"PENDIENTE"], description: "Estado del usuario" },
            createdAt: { bsonType: "date" }
         }
      }
   }
});
db.usuarios.createIndex({ username: 1 }, { unique: true
});
db.usuarios.createIndex({ email: 1 }, { unique: true });
```

```
      // 4. Colección 'auditorias'
      db.createCollection("auditorias", {
         validator: {
            $jsonSchema: {
               bsonType: "object",
               required: ["accion", "entidad", "entidadId",
"fecha", "usuarioId"],
               properties: {
                  accion: { bsonType: "string" },
                  entidad: { bsonType: "string" },
                  entidadId: { bsonType: "string" },
                  fecha: { bsonType: "date" },
                  usuarioId: { bsonType: "objectId" }
               }
            }
         }
      });
      db.auditorias.createIndex({ usuarioId: 1 });
      db.auditorias.createIndex({ entidad: 1, entidadId: 1 });

      // 5. Colección 'tramas'
      db.createCollection("tramas", {
         validator: {
            $jsonSchema: {
               bsonType: "object",
               required: ["código", "estado", "fechaInicio"],
               properties: {
                  código: { bsonType: "string" },
                  estado: { enum: ["ABIERTA", "EN_PROCESO",
"CERRADA"] },
                  fechaInicio: { bsonType: "date" },
                  fechaCierre: { bsonType: "date" }
               }
            }
         }
      });
      db.tramas.createIndex({ código: 1 }, { unique: true });

      // 6. Colección 'politicasNegocio'
      db.createCollection("politicasNegocio", {
         validator: {
            $jsonSchema: {
               bsonType: "object",
```

```
               required: ["nombre", "version", "estado",
"createdAt"],
               properties: {
                  nombre: { bsonType: "string" },
                  descripcion: { bsonType: "string" },
                  version: { bsonType: "int", minimum: 1 },
                  estado: { enum: ["BORRADOR", "PUBLICADA",
"ARCHIVADA"] },
                  createdAt: { bsonType: "date" }
               }
            }
         }
      });
      db.politicasNegocio.createIndex({ nombre: 1, version: 1 },
{ unique: true });

      // 7. Colección 'formulariosDinamicos'
      db.createCollection("formulariosDinamicos", {
         validator: {
            $jsonSchema: {
               bsonType: "object",
               required: ["nombre", "estado", "createdAt"],
               properties: {
                  nombre: { bsonType: "string" },
                  descripcion: { bsonType: "string" },
                  campos: { bsonType: "object" }, // definición
JSON del formulario
                  estado: { enum: ["BORRADOR", "PUBLICADO",
"ARCHIVADO"] },
                  createdAt: { bsonType: "date" },
                  updatedAt: { bsonType: "date" }
               }
            }
         }
      });

      // 8. Colección 'tramites'
      db.createCollection("tramites", {
         validator: {
            $jsonSchema: {
               bsonType: "object",
               required: ["código", "estado", "fechaInicio"],
               properties: {
```

```
                código: { bsonType: "string" },
                estado: { enum: ["INICIADO", "EN_CURSO",
"CERRADO"] },
                fechaInicio: { bsonType: "date" },
                fechaCierre: { bsonType: "date" },
                formularioId: { bsonType: "objectId" }
             }
          }
       }
    });
    db.tramites.createIndex({ código: 1 }, { unique: true });

    // 9. Colección 'tareas'
    db.createCollection("tareas", {
       validator: {
          $jsonSchema: {
             bsonType: "object",
             required: ["nombre", "estado", "fechaAsignacion",
"responsable"],
             properties: {
                nombre: { bsonType: "string" },
                estado: { enum: ["PENDIENTE", "EN_PROGRESO",
"COMPLETADA", "CANCELADA"] },
                fechaAsignacion: { bsonType: "date" },
                fechaFin: { bsonType: "date" },
                responsable: { bsonType: "objectId" },
                tramiteId: { bsonType: "objectId" },
                diagramaActividadId: { bsonType: "objectId" }
             }
          }
       }
    });
    db.tareas.createIndex({ responsable: 1 });
    db.tareas.createIndex({ tramiteId: 1 });

    // 10. Colección 'firmasElectronicas'
    db.createCollection("firmasElectronicas", {
       validator: {
          $jsonSchema: {
             bsonType: "object",
             required: ["algoritmo", "fechaFirma",
"firmanteId"],
             properties: {
```

```
                algoritmo: { bsonType: "string" },
                fechaFirma: { bsonType: "date" },
                firmanteId: { bsonType: "objectId" },
                documentoId: { bsonType: "objectId" },
                tramiteId: { bsonType: "objectId" }
             }
          }
       }
    });
    db.firmasElectronicas.createIndex({ firmanteId: 1 });
    db.firmasElectronicas.createIndex({ documentoId: 1 });
    db.firmasElectronicas.createIndex({ tramiteId: 1 });

    // 11. Colección 'diagramasActividad'
    db.createCollection("diagramasActividad", {
       validator: {
          $jsonSchema: {
             bsonType: "object",
             required: ["esValido", "modelo", "version",
"updatedAt"],
             properties: {
                esValido: { bsonType: "bool" },
                modelo: { bsonType: "object" },
                version: { bsonType: "int" },
                updatedAt: { bsonType: "date" }
             }
          }
       }
    });

    // 12. Colección 'documentos' con array embebido de
versiones
    db.createCollection("documentos", {
       validator: {
          $jsonSchema: {
             bsonType: "object",
             required: ["nombre", "tipo", "ubicacion",
"estado", "createdAt"],
             properties: {
                nombre: { bsonType: "string" },
                tipo: { bsonType: "string" },
                ubicacion: { bsonType: "string" },
```

```
                  estado: { enum: ["BORRADOR", "PUBLICADO",
"ARCHIVADO"] },
                  createdAt: { bsonType: "date" },
                  versiones: {
                     bsonType: "array",
                     items: {
                        bsonType: "object",
                        required: ["numero", "creadoPor",
"createdAt", "hash", "urlArchivo"],
                        properties: {
                           _id: { bsonType: "objectId" },
                           numero: { bsonType: "int", minimum: 1
},
                           creadoPor: { bsonType: "objectId" },
                           createdAt: { bsonType: "date" },
                           hash: { bsonType: "string" },
                           urlArchivo: { bsonType: "string" }
                        }
                     }
                  }
               }
            }
         }
      });
      db.documentos.createIndex({ "versiones.numero": 1 });

      // 13. Insertar datos de ejemplo
      // Usuarios
      const usuario1 = {
         username: "jperez",
         email: "jperez@example.com",
         passwordHash: "hash_de_contraseña",
         estado: "ACTIVO",
         createdAt: new Date()
      };
      const usuario2 = {
         username: "mgomez",
         email: "mgomez@example.com",
         passwordHash: "hash_otro",
         estado: "ACTIVO",
         createdAt: new Date()
      };
```

```
      const resUsuarios = db.usuarios.insertMany([usuario1,
usuario2]);
      const idUser1 = resUsuarios.insertedIds[0];
      const idUser2 = resUsuarios.insertedIds[1];

      // Trama
      const trama = {
         código: "TR-001",
         estado: "ABIERTA",
         fechaInicio: new Date(),
         fechaCierre: null
      };
      db.tramas.insertOne(trama);

      // Política de negocio
      db.politicasNegocio.insertOne({
         nombre: "Política de Plazos",
         descripcion: "Los trámites no pueden exceder 30 días",
         version: 1,
         estado: "PUBLICADA",
         createdAt: new Date()
      });

      // Formulario dinámico
      db.formulariosDinamicos.insertOne({
         nombre: "Solicitud básica",
         descripcion: "Formulario para recolección de datos
personales",
         campos: { campo1: "texto", campo2: "fecha" },
         estado: "PUBLICADO",
         createdAt: new Date(),
         updatedAt: new Date()
      });

      // Trámite
      const tramite = {
         código: "TM-100",
         estado: "INICIADO",
         fechaInicio: new Date(),
         formularioId: null
      };
      db.tramites.insertOne(tramite);
```

```
      // Tarea
      db.tareas.insertOne({
         nombre: "Revisar documentación",
         estado: "PENDIENTE",
         fechaAsignacion: new Date(),
         responsable: idUser1,
         tramiteId: null,
         diagramaActividadId: null
      });

      // Firma electrónica
      db.firmasElectronicas.insertOne({
         algoritmo: "RSA-SHA256",
         fechaFirma: new Date(),
         firmanteId: idUser2,
         documentoId: null
      });

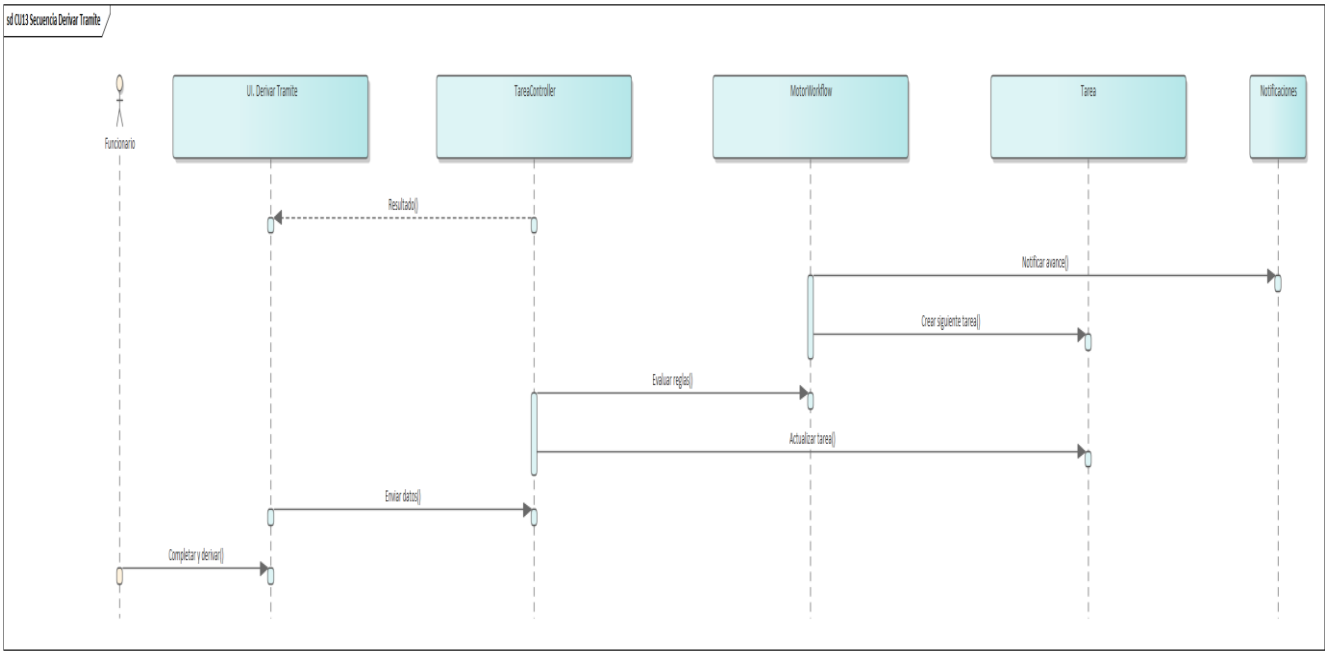
      // Diagrama actividad
      db.diagramasActividad.insertOne({
         esValido: true,
         modelo: { nodos: ["inicio", "tarea1", "fin"], aristas:
[] },
         version: 1,
         updatedAt: new Date()
      });

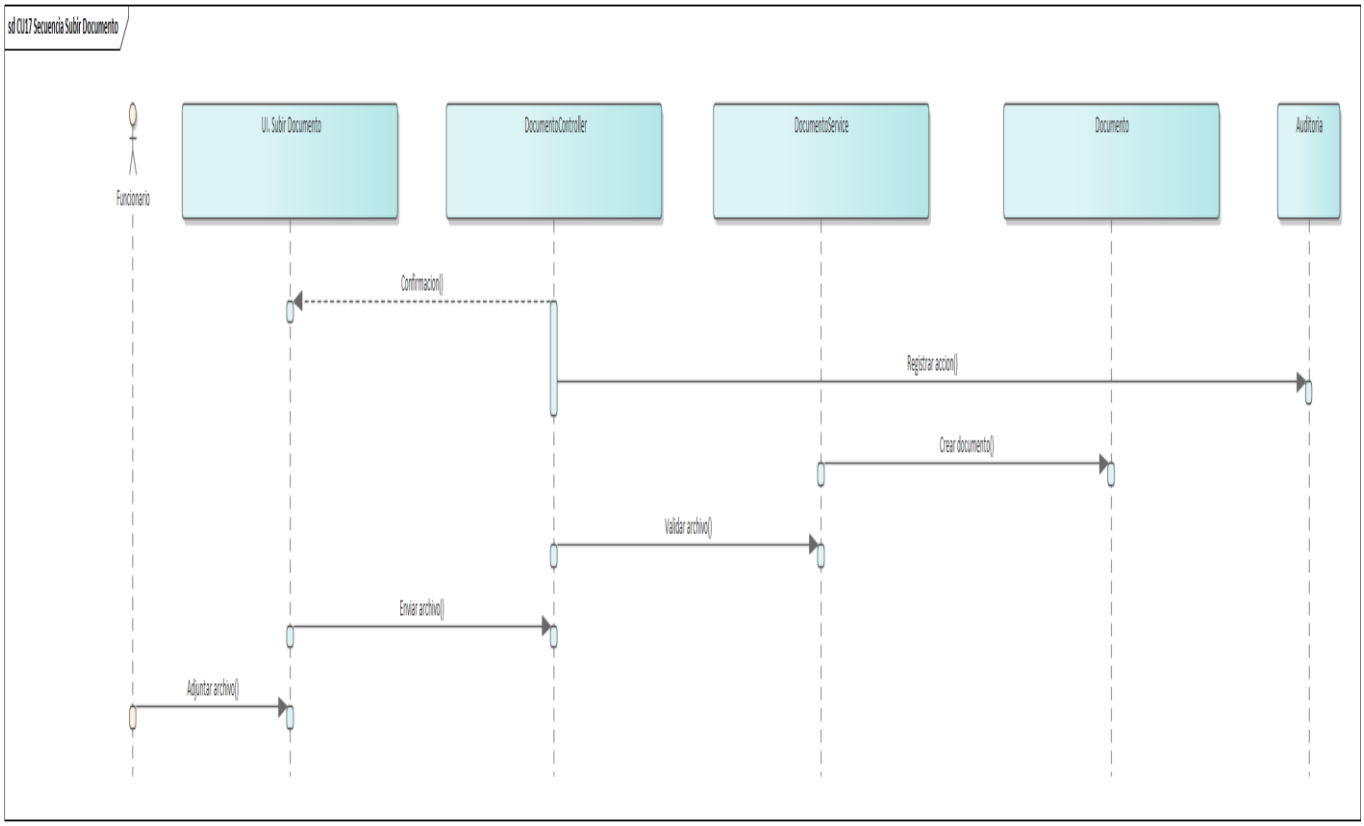
      // Documento con versión embebida
      const docId = new ObjectId();
      db.documentos.insertOne({
         _id: docId,
         nombre: "Contrato.pdf",
         tipo: "PDF",
         ubicacion: "/documentos/contrato.pdf",
         estado: "PUBLICADO",
         createdAt: new Date(),
         versiones: [
            {
               _id: new ObjectId(),
               numero: 1,
               creadoPor: idUser1,
               createdAt: new Date(),
               hash: "a1b2c3d4e5",
```

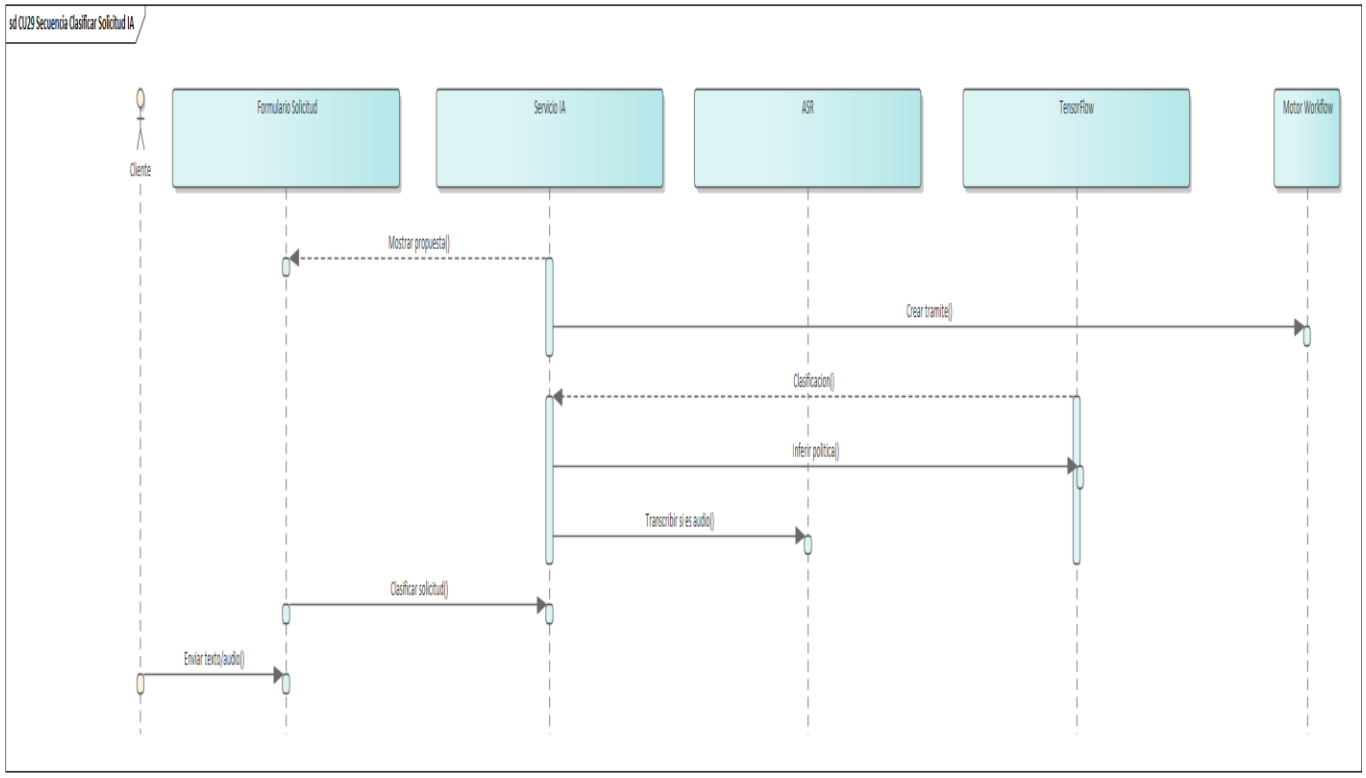
```
            urlArchivo: "/versiones/contrato_v1.pdf"
        }
    ]
});

// Auditoría (ejemplo)
db.auditorias.insertOne({
    accion: "CREAR_USUARIO",
    entidad: "Usuario",
    entidadId: idUser1.toString(),
    fecha: new Date(),
    usuarioId: idUser1
});
```

2.3.3 Diagramas de secuencia




sd CU17 Secuencia Subir Documento
Funcionario
UI. Subir Documento
DocumentoController
DocumentoService
Documento
Auditoria
Confirmacion()
Registrar accion()
Crear documento()
Validar archivo()
Enviar archivo()
Adjuntar archivo()


sd CU29 Secuencia Clasificar Solicitud IA
Cliente
Formulario Solicitud
Servicio IA
ASR
TensorFlow
Motor Workflow
Mostrar propuesta()
Crear tramite()
Clasificacion()
Inferir politica()
Transcribir si es audio()
Clasificar solicitud()
Enviar texto/audio()

2.4 FLUJO DE TRABAJO – IMPLEMENTACIÓN

2.4.1 Elección de Plataforma de Desarrollo de Software

La plataforma seleccionada para el desarrollo del sistema se compone de **Angular 19** para el frontend web, **FastAPI** para el backend, **Flutter** para la aplicación móvil, **MongoDB** como base de datos principal y **Amazon S3** para el almacenamiento de archivos. Esta combinación ofrece un ecosistema moderno, asíncrono y altamente escalable, alineado con los requisitos del proyecto: editor UML interactivo, motor de workflow, gestión documental con versionado, colaboración en tiempo real mediante WebSockets e integración de inteligencia artificial con TensorFlow.

2.4.1.1 Lenguajes de Programación

El proyecto utiliza **TypeScript** para el frontend web, **Python** para el backend y **Dart** para la aplicación móvil. Esta decisión se fundamenta en las fortalezas específicas de cada lenguaje para el dominio donde se aplica.

TypeScript (Frontend – Angular 19)

TypeScript es un superconjunto tipado de JavaScript que compila a JavaScript estándar. Se eligió para el desarrollo del frontend web por las siguientes razones:

1. **Confiabilidad:** El tipado estático, los genéricos, los enumerados, los union types y el type narrowing reducen significativamente los errores en tiempo de ejecución. Los contratos entre componentes, servicios y la API REST se definen mediante interfaces claras, garantizando consistencia en toda la capa de presentación.
2. **Mantenibilidad:** Los refactores son seguros gracias al análisis estático del compilador. Las API públicas de los servicios y componentes están bien definidas, facilitando la evolución del código a largo plazo.

3. **Productividad:** El ecosistema de desarrollo ofrece una experiencia superior: autocompletado inteligente (IntelliSense), linters (ESLint), formateadores (Prettier) y builders rápidos (esbuild/Vite). Las pruebas unitarias y de integración se realizan con Jasmine y Karma.
4. **Integración nativa con Angular 19:** Angular está construido sobre TypeScript, aprovechando al máximo sus decoradores, metadatos y sistema de tipos. La versión 19 introduce componentes standalone como patrón predeterminado, eliminando la necesidad de NgModules y simplificando la arquitectura del proyecto.
5. **Ecosistema:** Excelente integración con RxJS (programación reactiva), Angular Material (componentes UI), Socket.IO-client (WebSockets) y bibliotecas de visualización de diagramas como maxGraph.

Python (Backend – FastAPI)

Python se eligió para el desarrollo del backend por las siguientes razones:

1. **Rendimiento asíncrono:** FastAPI, construido sobre Starlette y ASGI, permite operaciones de entrada/salida no bloqueantes de forma nativa mediante async/await. Esto es crítico para manejar múltiples solicitudes concurrentes de workflow, carga de documentos a S3 e invocaciones a modelos de IA.
2. **Productividad y legibilidad:** La sintaxis clara y concisa de Python acelera el desarrollo de la lógica de negocio. La validación automática de datos mediante Pydantic elimina código repetitivo y reduce errores.
3. **Ecosistema de IA y Ciencia de Datos:** Python es el lenguaje dominante en inteligencia artificial y machine learning. La integración con TensorFlow, bibliotecas de NLP y herramientas de OCR es directa y natural.

4. **Documentación automática:** FastAPI genera automáticamente documentación OpenAPI (Swagger UI y ReDoc) a partir de los modelos Pydantic y las anotaciones de tipo, facilitando la integración con el frontend y las pruebas.
5. **Integración con AWS:** El SDK de AWS para Python (Boto3) proporciona una interfaz completa y bien documentada para interactuar con S3 y otros servicios de la nube.

Dart (Móvil – Flutter)

Dart es el lenguaje de programación utilizado por el framework Flutter. Se eligió para el desarrollo de la aplicación móvil por las siguientes razones:

1. **Rendimiento nativo:** Dart compila a código nativo para iOS y Android, ofreciendo un rendimiento cercano al de las aplicaciones nativas tradicionales.
2. **Hot Reload:** Permite iterar rápidamente durante el desarrollo, visualizando los cambios en tiempo real sin perder el estado de la aplicación.
3. **Ecosistema unificado:** Flutter proporciona un conjunto completo de widgets Material Design, bibliotecas de gestión de estado (BLoC) y paquetes para comunicación HTTP (dio), WebSockets y notificaciones push (Firebase).
4. **Offline-first:** La combinación de Hive para almacenamiento local y una cola de sincronización permite que la aplicación funcione sin conexión, sincronizando los datos cuando se restablece la conectividad.

2.4.1.2 Sistema de Gestión de Bases de Datos (DBMS)

Para el presente proyecto se ha seleccionado **MongoDB** como sistema de gestión de bases de datos. La elección responde a la necesidad de contar con una plataforma flexible, escalable y de alto rendimiento, que permita manejar eficientemente tanto los datos estructurados de usuarios, roles y departamentos, como las estructuras

dinámicas y semiestructuradas propias de los diagramas de actividades, formularios configurables, contextos de instancias de workflow y metadatos documentales.

MongoDB es una base de datos NoSQL orientada a documentos de código abierto, ampliamente reconocida en la industria por su capacidad para manejar grandes volúmenes de datos con esquemas flexibles. Una de las principales razones de esta elección es su modelo de datos basado en documentos BSON (Binary JSON), que permite almacenar estructuras jerárquicas complejas —como los grafos de workflow con nodos y transiciones— de forma natural y sin necesidad de costosas operaciones de unión.

El principio de "schema on read" de MongoDB es particularmente valioso en este proyecto, donde las definiciones de formularios y las políticas de negocio son creadas dinámicamente por los administradores. Cada documento de una colección puede tener una estructura diferente, adaptándose a las necesidades específicas de cada tipo de trámite sin requerir migraciones de esquema que bloqueen el sistema.

La integración con FastAPI se realiza mediante **Motor**, un driver asíncrono para MongoDB que aprovecha el loop de eventos de Python para ejecutar operaciones de base de datos de manera no bloqueante, maximizando el rendimiento del servidor. La capa de persistencia se complementa con índices predefinidos y seeds de datos iniciales gestionados desde el módulo app/db/mongodb.py.

2.4.1.3 Sistemas Operativos

El sistema operativo seleccionado para la implementación y despliegue del proyecto es **Linux**, debido a su estabilidad, seguridad y amplia adopción en entornos de servidores y cloud computing. Linux es un sistema operativo de código abierto que ofrece un ecosistema sólido y maduro, con soporte para arquitecturas distribuidas, redes de alta concurrencia y administración de recursos eficiente.

El proyecto se apoya en **Docker** como capa de virtualización ligera para empaquetar y distribuir las aplicaciones. Docker permite aislar cada servicio en contenedores, garantizando portabilidad entre distintos entornos (desarrollo, pruebas y producción) y reduciendo problemas de configuración o dependencias. El archivo docker-compose.yml en la raíz del monorepo orquesta los servicios de backend, frontend y sus dependencias.

El uso combinado de Linux y Docker proporciona un entorno altamente escalable y reproducible, donde es posible desplegar múltiples instancias de los servicios del sistema de manera independiente. Esto favorece una arquitectura basada en microservicios y facilita la integración con orquestadores como Kubernetes, garantizando un despliegue confiable y resiliente en entornos de producción.

2.4.1.4 Despliegue en la Nube (AWS)

Para el despliegue del sistema se utilizarán los siguientes servicios de **Amazon Web Services (AWS)**:

1. Amazon EC2 (Elastic Compute Cloud)

- **Propósito:** Servidores virtuales escalables para ejecutar el backend FastAPI y el frontend Angular.
- **Configuración:** Instancias EC2 con Amazon Linux 2, ejecutando Docker para orquestar los contenedores de la aplicación.
- **Escalado:** Auto Scaling Groups para ajustar automáticamente el número de instancias según la demanda.
- **Balanceo de Carga:** Application Load Balancer (ALB) para distribuir el tráfico entrante.

2. Amazon S3 (Simple Storage Service)

- **Propósito:** Almacenamiento de objetos para los archivos físicos de los documentos (PDF, Word, imágenes) y versiones documentales.
- **Características aplicadas:** Versionado nativo, URLs prefirmadas, cifrado en reposo (SSE-S3) y en tránsito (TLS), políticas de ciclo de vida para archivar documentos antiguos a S3 Glacier.
- **Integración:** Boto3 (SDK de AWS para Python) desde el servicio app/services/s3.py.

3. Amazon ECR (Elastic Container Registry)

- **Propósito:** Repositorio privado para almacenar las imágenes Docker del backend y frontend.
- **Flujo:** Las imágenes se construyen y publican en ECR mediante GitHub Actions como parte del pipeline CI/CD.

4. Amazon CloudWatch

- **Propósito:** Monitorización, logging y alertas para las instancias EC2 y los contenedores.

5. AWS IAM (Identity and Access Management)

- **Propósito:** Control de acceso granular a los recursos de AWS mediante roles y políticas.

2.4.2 Estructura del Proyecto

El proyecto se organiza como un **monorepo** con tres aplicaciones ejecutables independientes: backend, frontend y mobile. A continuación se detalla la organización de carpetas, puntos de entrada y responsabilidades de cada capa.

2.4.2.1 Vista General del Monorepo

text

codigo/

├── backend/ # API REST + WebSocket + IA/ML (Python 3.12, FastAPI, MongoDB)

├── frontend/ # Panel web administrativo y portal de workflow (Angular 19)

├── mobile/ # Cliente móvil para solicitantes/funcionarios (Flutter)

├── diagramas/ # Scripts VBScript para Enterprise Architect y artefactos C4

├── docker-compose.yml

└── ArquitecturaArchivos.md

App	Stack principal	Puerto dev	Punto de entrada
Backend	FastAPI, Motor (MongoDB), TensorFlow	8000/8001	backend/app/main.py
Frontend	Angular 19, standalone components	4200	frontend/src/main.ts
Mobile	Flutter 3.x, Hive, Firebase Messaging	—	mobile/lib/main.dart

2.4.2.2 Estructura del Backend (backend/)

text

backend/

├── app/ # Código de la aplicación

│ ├── main.py # FastAPI: lifespan, CORS, routers, /health

│ ├── api/

│ │ ├── router.py # Agrega todos los módulos REST bajo /api/v1

│ │ ├── dependencies.py # JWT, usuario actual, require_permission()

│ │ └── routes/ # Controladores HTTP por dominio

│ ├── core/ # Config, seguridad, seeds, auditoría, tenancy

│ ├── db/

│ │ └── mongodb.py # Cliente Motor, índices, seed inicial

│ ├── schemas/ # DTOs Pydantic (contratos de API)

```
│   ├── services/              # Lógica de negocio e integraciones
│   │   └── ai/                # Pipeline de IA/ML (reportes, políticas, routing…)
│   ├── utils/
│   │   └── mongo.py           # Helpers ObjectId, serialización, utcnow
│   └── websocket/
│       ├── router.py          # Canales en tiempo real (/ws/…)
│       └── manager.py         # Salas y broadcast por dominio
│
├── scripts/                   # Entrenamiento ML, datasets, seeds, migraciones
├── models/                    # Artefactos .keras y metadatos JSON
├── datasets/                  # JSON de entrenamiento para scripts ML
├── tests/                     # pytest (mongomock-motor, TF mockeado)
├── storage/documents/         # Almacenamiento local de archivos (si no S3)
│
├── requirements.txt
├── Dockerfile
├── .env / .env.example
└── README.md
```

Patrón arquitectónico del backend:

Capas: **routes → services → MongoDB / S3**. No existe una capa ORM ni directorio app/models/ de dominio; la persistencia es documental en MongoDB y los contratos viven en app/schemas/.

text

Cliente HTTP/WS

→ app/main.py

→ api/router.py → routes/*.py → services/*

→ websocket/router.py → manager.py

→ db/mongodb.py → MongoDB

→ services/storage.py → local / S3

→ services/ai/* → models/*.keras

Puntos de entrada principales:

Archivo	Rol
app/main.py	Crea la app FastAPI, conecta Mongo en el lifespan, monta CORS, incluye api_router (/api/v1) y websocket_router (/ws), expone GET /health
app/db/mongodb.py	Al iniciar: connect_to_mongo(), ensure_indexes(), seed_initial_data()

API REST (app/api/routes/**)**

Prefijo global: /api/v1 (configurable en app/core/config.py).

Módulo	Prefijo	Responsabilidad
auth.py	/auth	Login, refresh, logout, revocación de token
users.py	/users	CRUD de usuarios
roles.py	/roles	Roles y permisos
departments.py	/departments	Departamentos
policies.py	/policies	Políticas, diagramas, formularios, publicación, colaboradores
documents.py	/documents	Subida, versiones, firmas, comentarios, colaboración
workflow.py	/workflow	Instancias, tareas, bandeja, KPIs, derivación
dashboard.py	/dashboard	Paneles operativos y de auditoría
audit.py	/audit	Consulta de logs de auditoría
notifications.py	/notifications	Notificaciones push/outbox
chat.py	/chat	Conversaciones y mensajes
ai.py	/ai	Reportes NL, clasificación, formularios IA, políticas IA, routing ML, asistente

Core (app/core/**)**

Módulo	Propósito
config.py	Settings: Mongo, JWT, S3, rutas ML, modos intérprete, CORS
security.py	Hash de contraseñas (pbkdf2_sha256), JWT encode/decode
permissions.py	Nombres de roles y permisos (reports:ai, documents:sign, etc.)
seeds.py	Índices Mongo + datos demo (roles, usuarios, políticas)
tenancy.py	Multi-empresa (company_id)
audit.py	Registro estructurado de eventos
policy_access.py	Reglas de acceso al editor de políticas

Servicios de negocio (app/services/**)**

Módulo	Propósito
workflow_engine.py	Motor BPMN: recorrido del grafo, guardas, tareas
workflow_form_normalizer.py	Normaliza respuestas de formularios en el contexto del trámite
policy_validator.py	Valida diagramas antes de publicar
document_repositories.py	Repositorios documentales por trámite
document_permissions.py	ACL por documento
storage.py	Orquestación de subida (local vs S3)
s3.py	Cliente boto3 para Amazon S3
notifications.py	Persistencia + Firebase push
email_service.py	Envío de correos

Servicios de IA (app/services/ai/**)**

Grupo	Módulos clave	Propósito
Reportes	report_orchestrator, report_query_parser, tf_report_interpreter, nlp_report_interpreter, report_spec_validator, report_executor, report_presenter, report_exporter	Consulta NL → spec → Mongo → presentación → PDF/Excel
Políticas	policy_classifier, policy_generator, diagram_composer, policy_prompt_parser	Clasificar solicitudes y generar políticas/diagramas
Formularios	form_suggester, deep_form_suggester	Auto-llenado de formularios
Routing ML	route_recommender, delay_risk_predictor, bottleneck_analyzer, anomaly_detector, task_prioritizer	Riesgo, cuellos de botella, rutas, anomalías
Asistente	assistant_service, assistant_rules, assistant_knowledge	Asistente in-app
Infra	tensorflow_service, audio_transcriber, text_normalizer	Carga de modelos, ASR, normalización de texto

WebSocket (app/websocket/**)**

Endpoint	Propósito
/ws/documents/{id}/collab	Colaboración en documentos (presencia + cambios)
/ws/policies/{id}/editor	Edición simultánea de diagramas de política
/ws/workflow/...	Actualizaciones de bandeja y tareas
/ws/chat/...	Mensajes de chat en tiempo real

2.4.2.3 Estructura del Frontend (frontend/)

text

```
frontend/
├── angular.json            # Config CLI (entry: src/main.ts)
├── package.json
├── tsconfig*.json
```

```
├── Dockerfile
├── public/                # Assets estáticos
└── src/
    ├── main.ts            # Bootstrap: maxGraph + bootstrapApplication
    ├── index.html
    ├── styles.scss
    ├── environments/
    │   ├── environment.ts             # Dev: localhost:8001
    │   └── environment.production.ts  # Prod: EC2
    └── app/
        ├── app.component.ts    # Root: <router-outlet>, carga usuario
        ├── app.config.ts       # Router, HTTP + authInterceptor, animaciones
        ├── app.routes.ts       # Tabla central de rutas
        ├── core/               # Servicios, guards, modelos, interceptors
        ├── features/           # Páginas por dominio (standalone components)
        ├── layout/             # Shells con sidebar
        └── shared/             # Componentes reutilizables y utilidades
```

Patrón arquitectónico del frontend:

Angular 19 con **componentes standalone** como patrón predeterminado (sin NgModules). La lógica de API reside en core/services/; las páginas en features/; la UI compartida en shared/.

text

```
main.ts → app.config.ts → app.routes.ts
  → AppComponent
    → LoginPage | AppShell (/panel) | FuncionarioShell (/workflow)
```

→ features/* (páginas lazy o eager)

→ shared/components/* (chat, asistente, documentos, dashboard)

Puntos de entrada principales:

Archivo	Rol
src/main.ts	Configura assets de maxGraph y arranca AppComponent
src/app/app.config.ts	Registra router, HttpClient + authInterceptor, animaciones
src/app/app.routes.ts	Árbol completo de rutas y guards
src/environments/environment.ts	apiBaseUrl, apiV1Prefix, wsUrl

Core (src/app/core/**)**

Subcarpeta	Contenido
config/	Configuración global (ej. maxgraph.config.ts)
guards/	authGuard, adminPanelGuard, workflowGuard, auditPanelGuard, guestG
interceptors/	auth.interceptor.ts — adjunta JWT
models/	Interfaces TS: auth, admin, ai, audit, dashboard, policy, workflow
services/	Capa API y dominio (api.service, auth.service, workflow.service, etc.)

Layout (src/app/layout/**)**

Componente	Ruta	Propósito
app-shell	/panel	Sidebar admin: usuarios, roles, políticas, auditoría, IA, documentos
funcionario-shell	/workflow	Portal funcionario: bandeja, trámites, catálogo, documentos

Features (src/app/features/**)**

Las páginas principales se organizan por dominio: login/, dashboard/, users/, roles/, departments/, policies/, audit/, reports/, anomalies/, bottlenecks/, documents/ y workflow/. El módulo de workflow incluye

subpáginas para la bandeja de tareas, ejecución de tareas, seguimiento de trámites, catálogo de políticas y repositorio documental.

Shared (src/app/shared/**)**

Componentes reutilizables: assistant/ (widget flotante del asistente IA), chat/ (chat en tiempo real), dashboard/ (gráficos, tarjetas de métricas), documents/ (editor rich-text, preview, comparar versiones, permisos).

2.4.2.4 Estructura de la Aplicación Móvil (mobile/)

text

```
mobile/
├── lib/                    # Código Dart
│   ├── main.dart           # Bootstrap: Hive, Firebase, Sw1App
│   ├── app.dart            # MaterialApp + tema
│   ├── core/
│   │   ├── config/app_config.dart
│   │   ├── models/         # DTOs: auth, workflow, ai, document, report, audit
│   │   ├── services/       # Clientes HTTP y plataforma
│   │   ├── connectivity/   # Estado de red
│   │   └── offline/        # Cache Hive, cola de sync
│   └── presentation/
│       ├── app_shell.dart  # Gate de sesión
│       ├── screens/        # login_screen, home_screen
│       └── widgets/        # offline_banner
│
├── test/                   # widget_test, sync_queue, permissions
```

├── android/, ios/, web/, linux/, macos/, windows/ # Runners por plataforma

├── pubspec.yaml

└── analysis_options.yaml

Patrón arquitectónico del móvil:

Arquitectura **simplificada en dos capas**: core/ + presentation/. La lógica de negocio está en core/services/ y la UI se concentra en home_screen.dart con un dashboard unificado que integra trámites, asistente IA, documentos, reportes y auditoría.

text

main.dart → Sw1App → AppShell

├── LoginScreen (sin sesión)

└── HomeScreen (autenticado: trámites, IA, documentos, reportes, auditoría)

Puntos de entrada principales:

Archivo	Rol
lib/main.dart	Inicializa OfflineStore (Hive) y PushNotificationService (Firebase)
lib/app.dart	MaterialApp con tema Material 3
lib/presentation/app_shell.dart	Restaura sesión, valida token, redirige a login o home
lib/core/config/app_config.dart	URL del API, WebSocket, TTL de cache offline

Canal	Uso
REST /api/v1	CRUD, workflow, documentos, IA, reportes
WebSocket /ws	Colaboración, bandeja en tiempo real, chat, editor de políticas
Firebase	Push notifications al móvil
S3 / storage/documents/	Binarios de documentos

2.4.3 Control de Versiones y CI/CD

Repositorio de Código Fuente: GitHub

El código fuente del proyecto se almacena en un repositorio Git alojado en GitHub, siguiendo un modelo de branching que separa las ramas main (producción), develop (integración) y feature/* (funcionalidades).

Pipeline CI/CD: GitHub Actions

Workflow de Backend (FastAPI):

1. **Trigger:** Push a las ramas develop o main, o Pull Request hacia main.
2. **Pasos:**
    - Checkout del código fuente.
    - Configuración de Python 3.12.
    - Instalación de dependencias (pip install -r requirements.txt).
    - Ejecución de tests unitarios y de integración (pytest).
    - Verificación de estilo de código (flake8, black).
    - Construcción de la imagen Docker (docker build).
    - Publicación de la imagen en Amazon ECR.
    - (Solo en main) Despliegue en EC2: actualización del contenedor.

Workflow de Frontend (Angular):

1. **Trigger:** Push a las ramas develop o main, o Pull Request hacia main.
2. **Pasos:**
    - Checkout del código fuente.
    - Configuración de Node.js 20.
    - Instalación de dependencias (npm ci).
    - Ejecución de tests unitarios (ng test --watch=false).
    - Construcción del bundle de producción (ng build --configuration=production).
    - Construcción de la imagen Docker con Nginx.

- Publicación de la imagen en Amazon ECR.
- (Solo en main) Despliegue en EC2.

2.4.4 Configuración de Entornos

El sistema utiliza variables de entorno para gestionar las configuraciones específicas de cada entorno. En el backend, estas se definen en el archivo .env y se cargan mediante app/core/config.py. En el frontend, se utilizan los archivos src/environments/environment.ts (desarrollo) y environment.production.ts (producción). En el móvil, la configuración se centraliza en lib/core/config/app_config.dart.

2.4.5 Despliegue en AWS

Paso 1: Creación de la Instancia EC2

- Lanzar una instancia con Amazon Linux 2.
- Seleccionar un tipo de instancia apropiado (t3.medium para MVP).
- Configurar el Security Group para permitir tráfico HTTP (80), HTTPS (443) y SSH (22).
- Asignar un rol IAM con permisos para ECR y S3.

Paso 2: Instalación de Docker en EC2

bash

sudo yum update -y

sudo amazon-linux-extras install docker

sudo service docker start

sudo usermod -a -G docker ec2-user

sudo systemctl enable docker

Paso 3: Despliegue de los Contenedores

bash

```
# Autenticación en ECR
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com

# Descarga y ejecución de imágenes
docker pull 123456789012.dkr.ecr.us-east-1.amazonaws.com/backend:latest
docker pull 123456789012.dkr.ecr.us-east-1.amazonaws.com/frontend:latest

docker run -d --name backend --env-file .env -p 8000:8000 123456789012.dkr.ecr.us-east-1.amazonaws.com/backend:latest
docker run -d --name frontend -p 80:80 123456789012.dkr.ecr.us-east-1.amazonaws.com/frontend:latest
```

Paso 4: Configuración del Bucket S3

- Crear un bucket con versionado habilitado.
- Configurar políticas CORS para el dominio del frontend.
- Configurar políticas de ciclo de vida para S3 Glacier.

Paso 5: Configuración de MongoDB Atlas

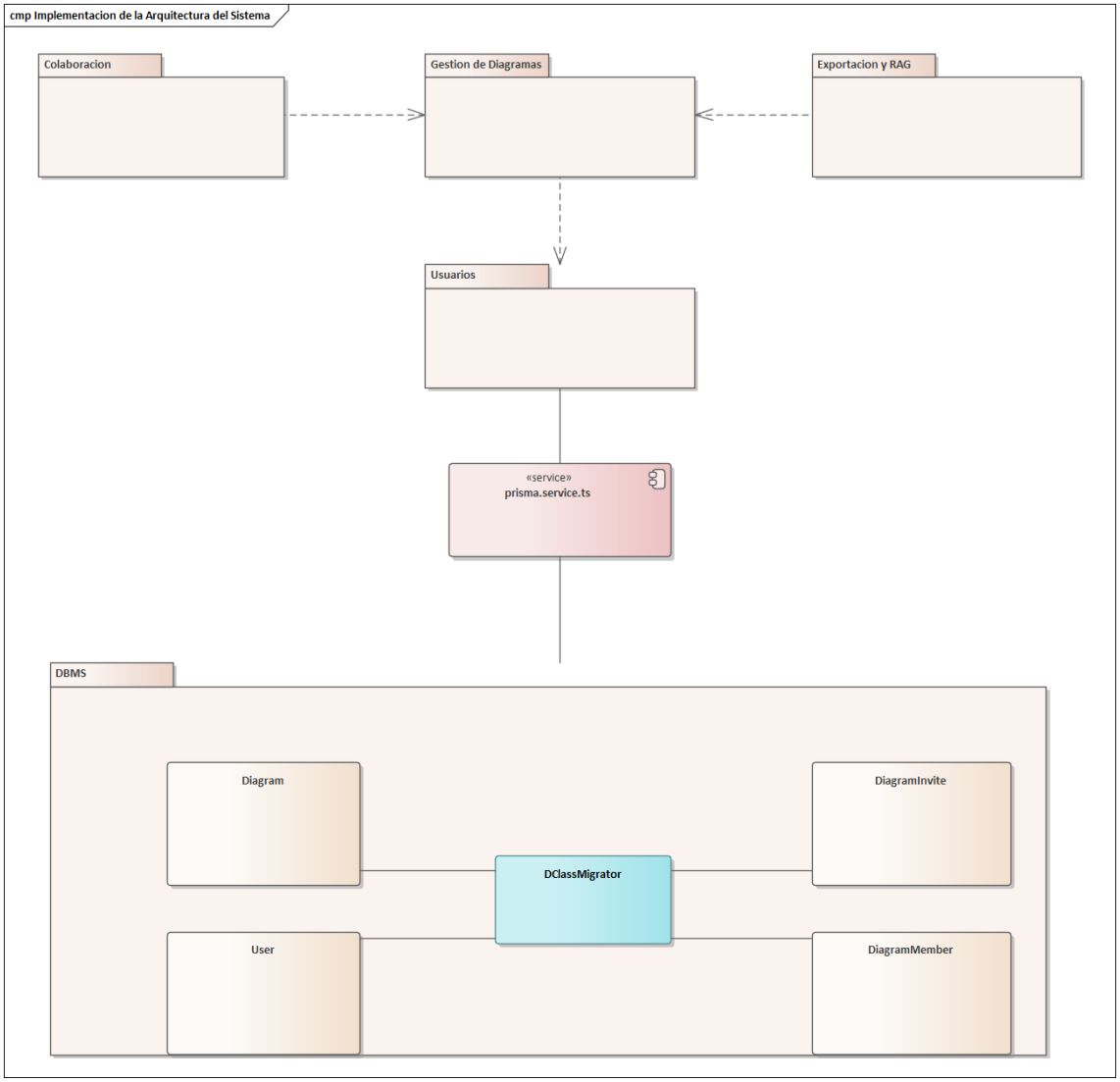
- Crear un cluster en MongoDB Atlas.
- Configurar Network Access para permitir conexiones desde la IP de la instancia EC2.
- Crear un usuario de base de datos y agregar la URI a las variables de entorno.

2.4.6 Convenciones Transversales

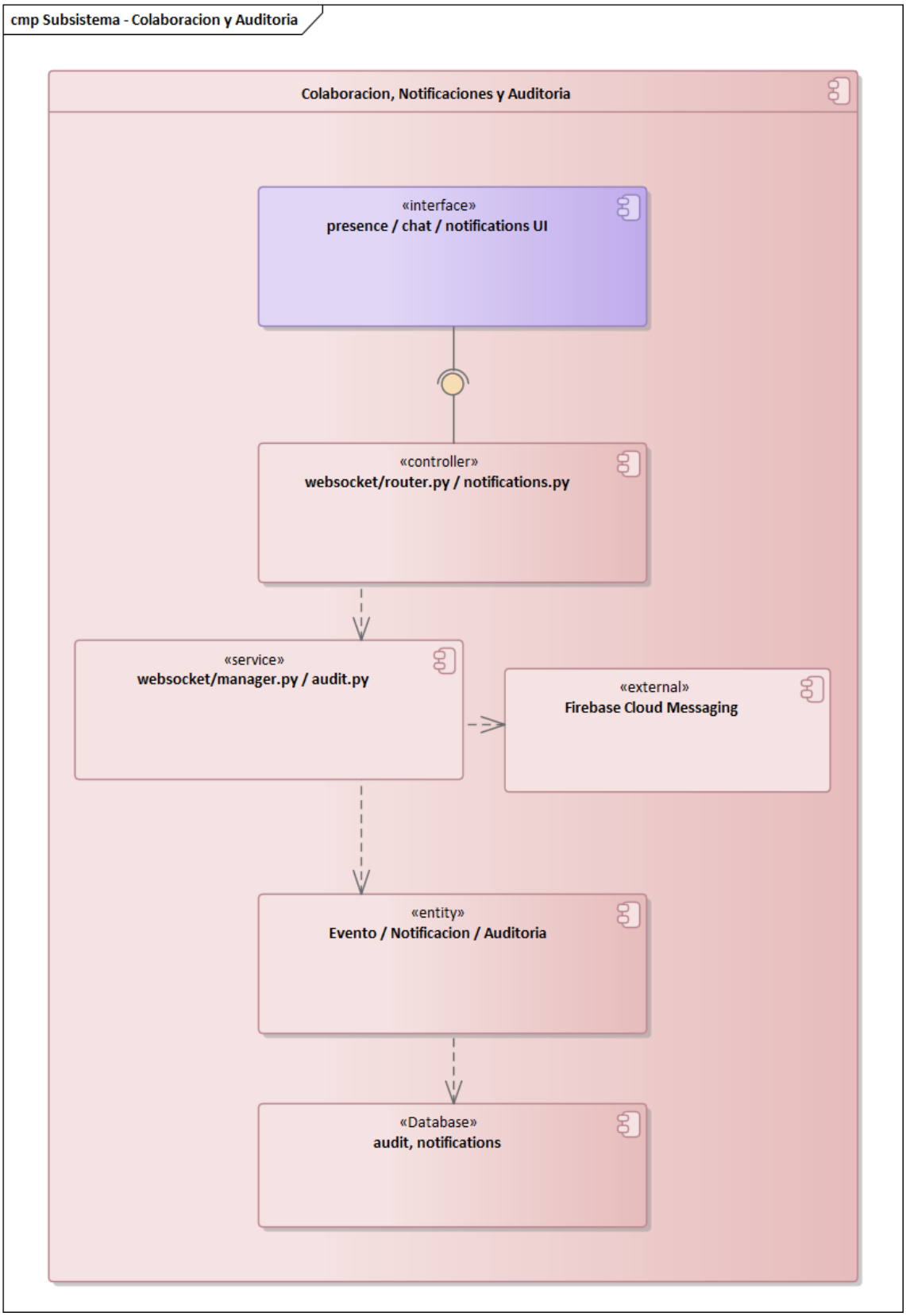
Tema	Backend	Frontend	Mobile
Autenticación	JWT Bearer en dependencies.py	authInterceptor + authGuard	SessionStorage + header en api_service
Permisos	require_permission("codigo")	Guards + user.permissions	user.hasPermission(...)

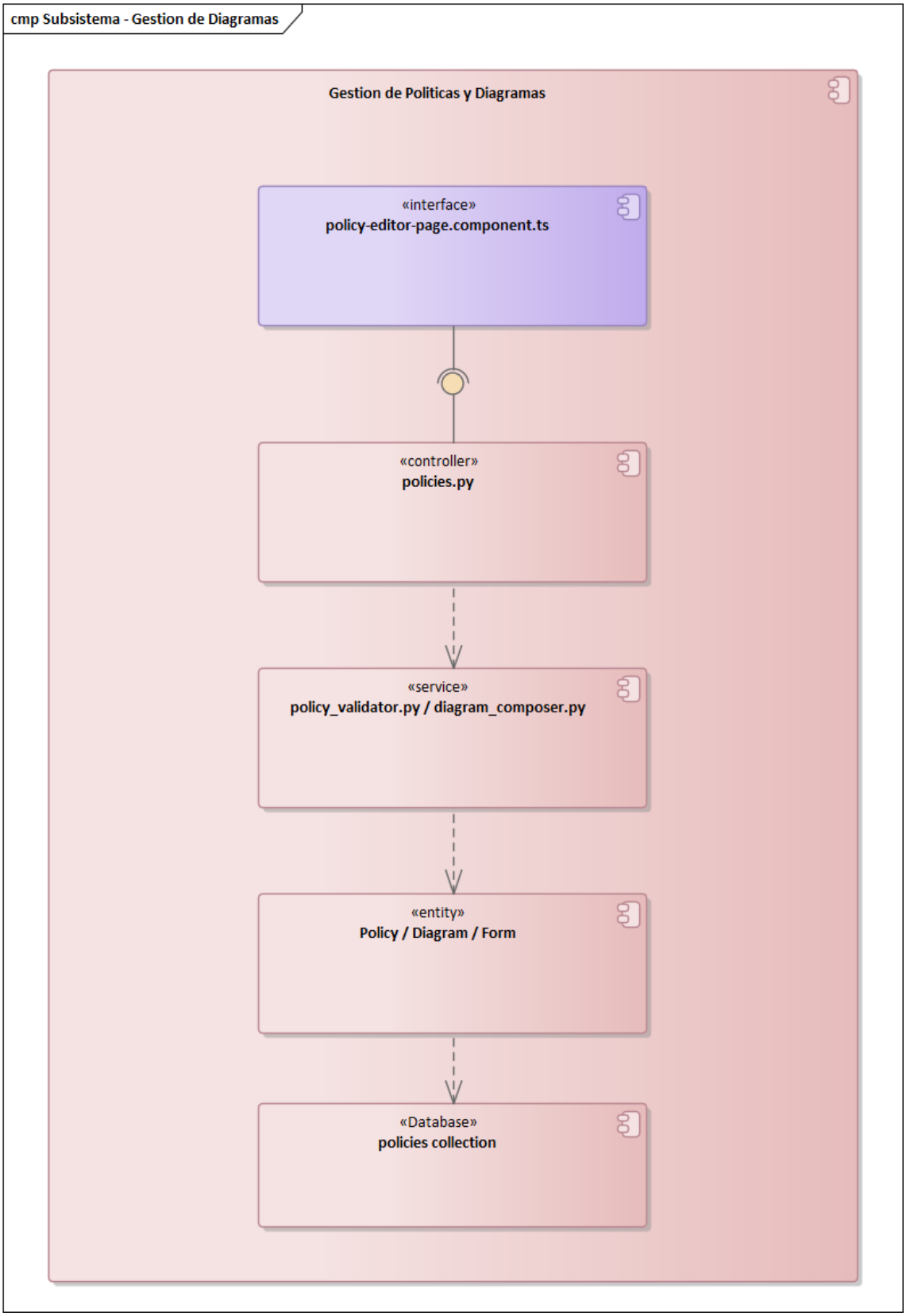
Config de API	Solo backend/.env	src/environments/*.ts	lib/core/config/app_config.dart
Tests	pytest en backend/tests/	Karma/Jasmine	flutter test
Demo user	Seed en core/seeds.py	admin@example.com / Admin123*	Mismas credenciales

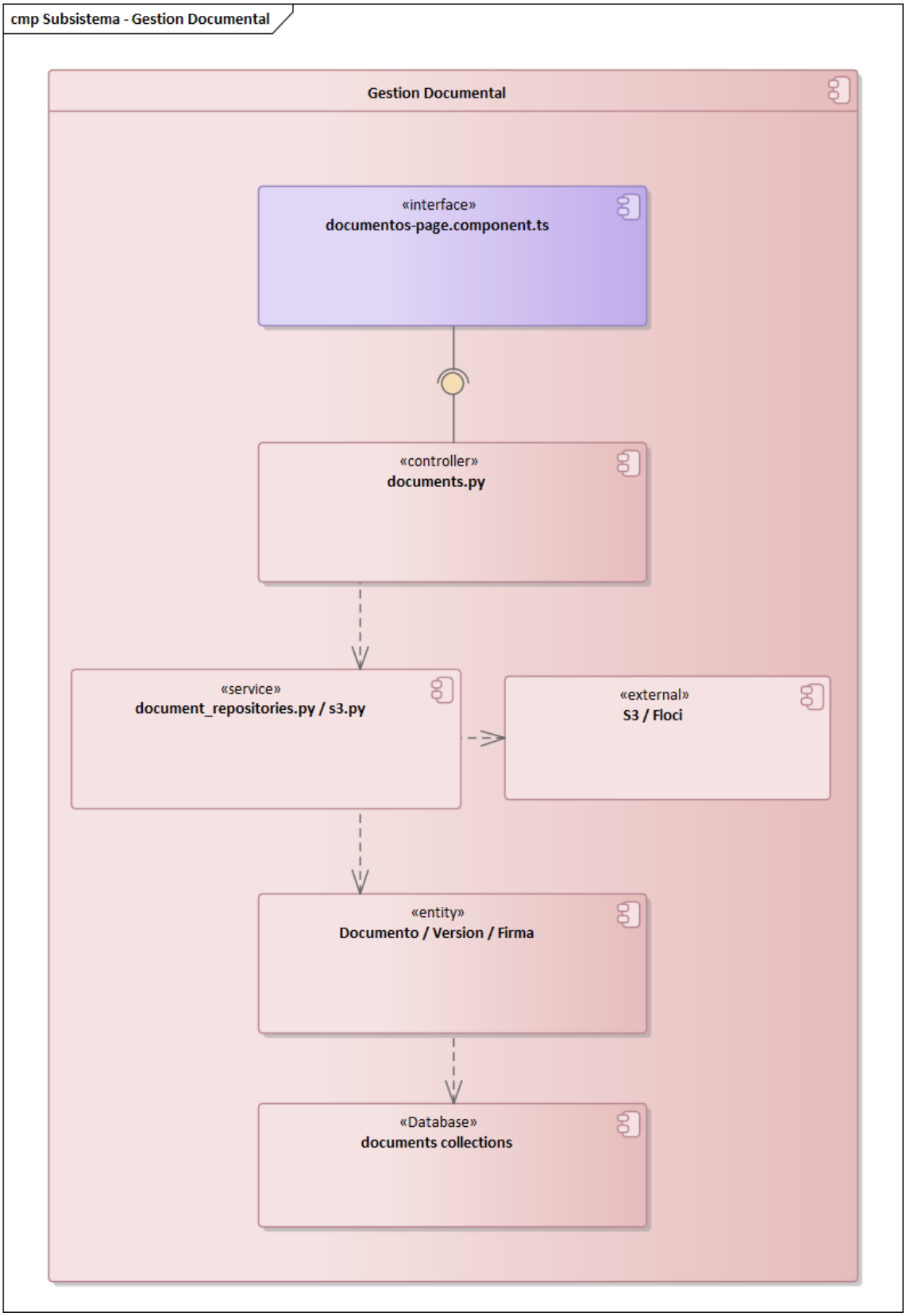
2.4.7 Implementación de la Arquitectura del Sistema

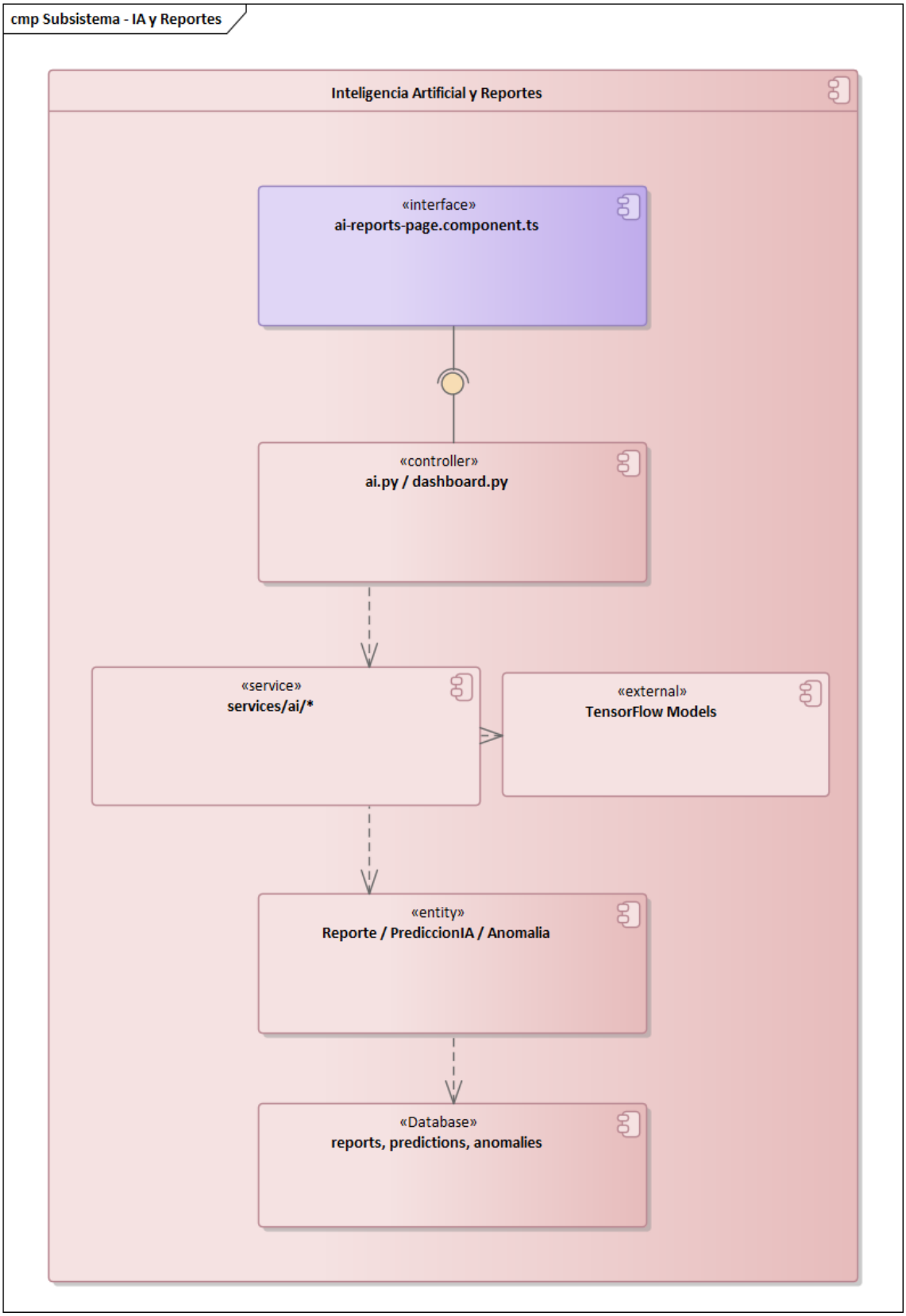


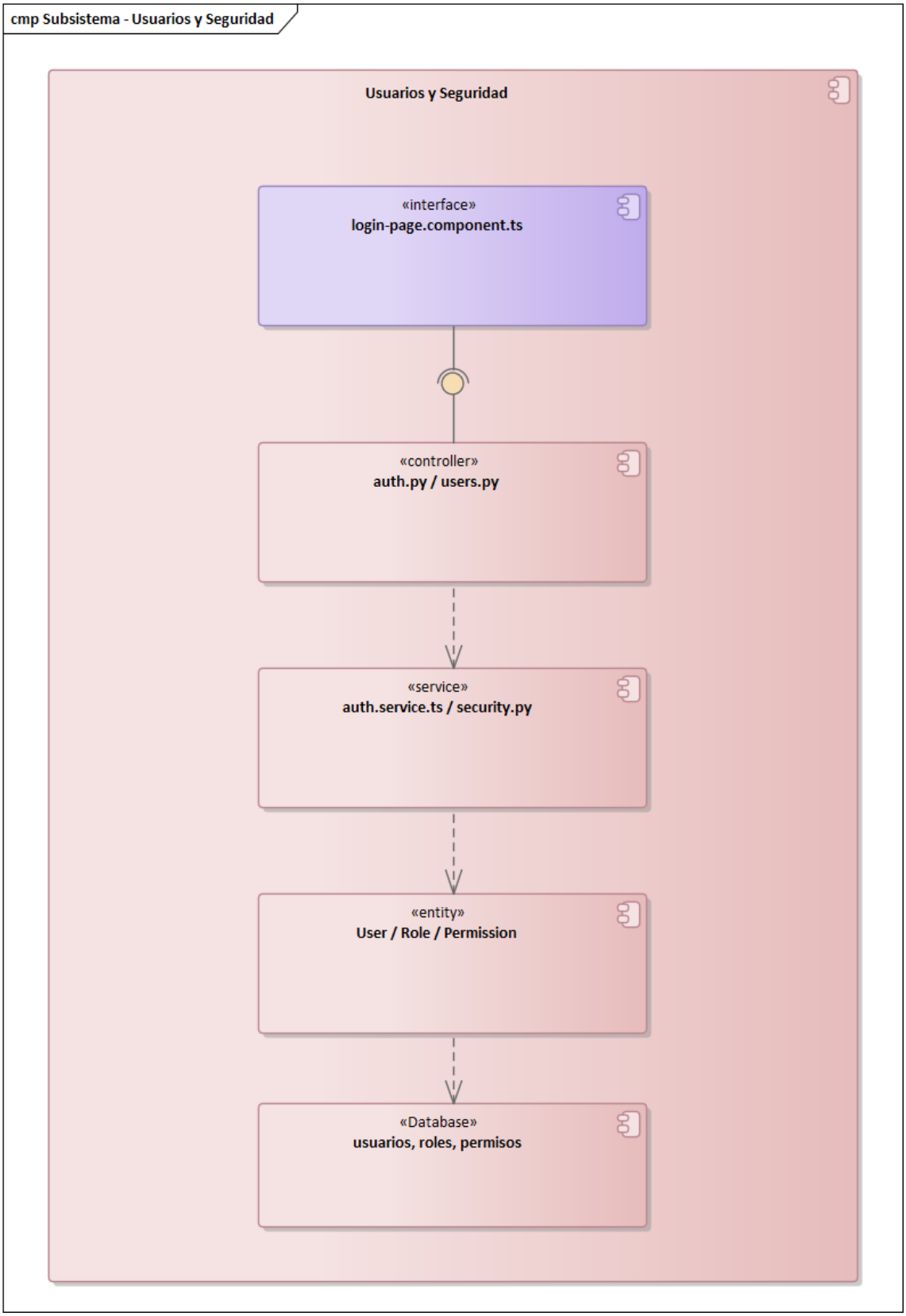
2.4.8 Implementación de la Arquitectura del Subsistema

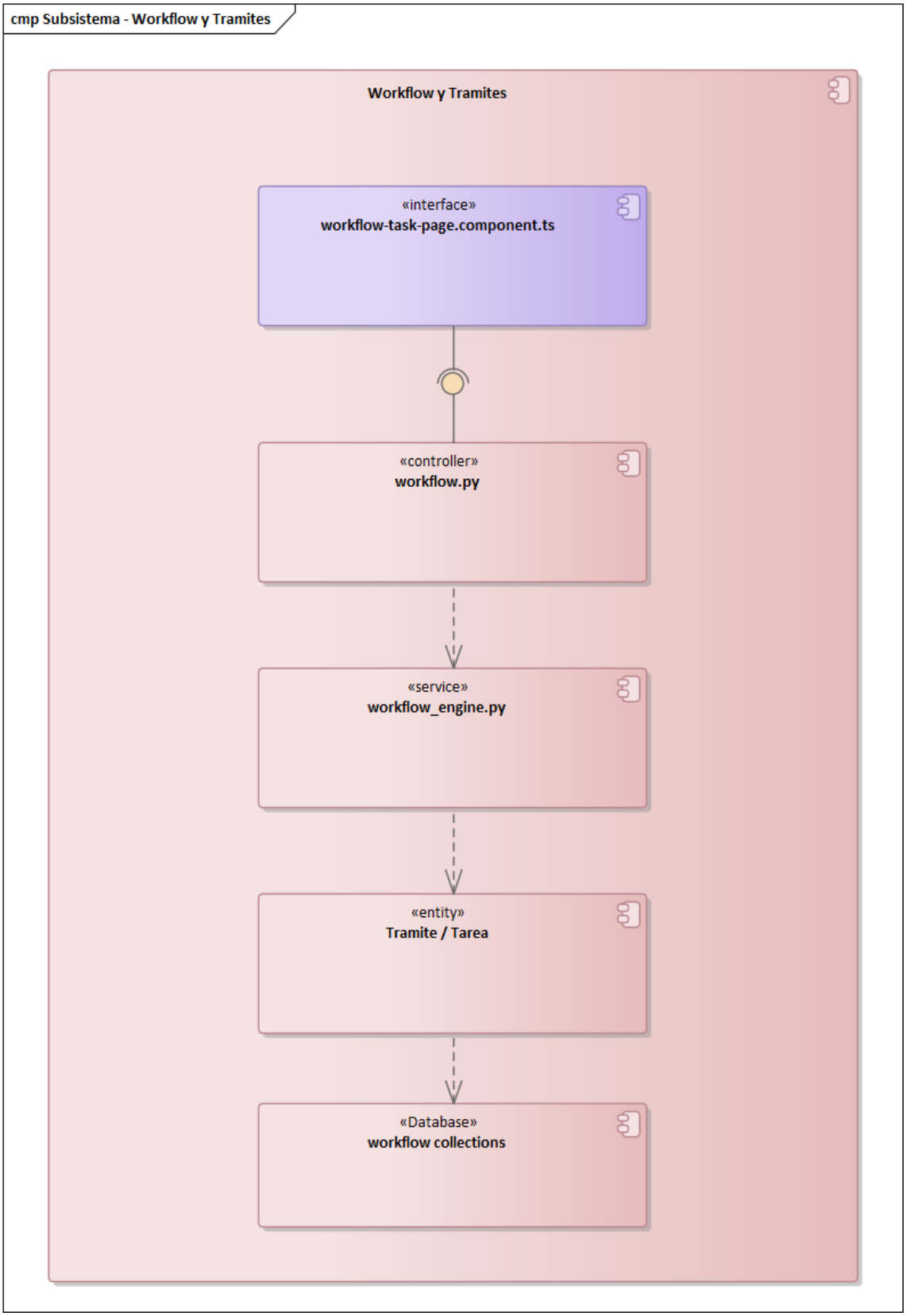



cmp Subsistema - Gestion de Diagramas
Gestion de Politicas y Diagramas
«interface»
policy-editor-page.component.ts
«controller»
policies.py
«service»
policy_validator.py / diagram_composer.py
«entity»
Policy / Diagram / Form
«Database»
policies collection


cmp Subsistema - Gestion Documental
Gestion Documental
«interface»
documentos-page.component.ts
«controller»
documents.py
«service»
document_repositories.py / s3.py
«external»
S3 / Floci
«entity»
Documento / Version / Firma
«Database»
documents collections


cmp Subsistema - IA y Reportes
Inteligencia Artificial y Reportes
«interface»
ai-reports-page.component.ts
«controller»
ai.py / dashboard.py
«service»
services/ai/*
«external»
TensorFlow Models
«entity»
Reporte / PrediccionIA / Anomalia
«Database»
reports, predictions, anomalies


cmp Subsistema - Usuarios y Seguridad
Usuarios y Seguridad
«interface»
login-page.component.ts
«controller»
auth.py / users.py
«service»
auth.service.ts / security.py
«entity»
User / Role / Permission
«Database»
usuarios, roles, permisos


cmp Subsistema - Workflow y Tramites
Workflow y Tramites
«interface»
workflow-task-page.component.ts
«controller»
workflow.py
«service»
workflow_engine.py
«entity»
Tramite / Tarea
«Database»
workflow collections

2.5 FLUJO DE TRABAJO – PRUEBAS

```
from __future__ import annotations


def login(client, email: str = "admin@example.com", password: str =
"Admin123*") -> dict:
    response = client.post("/api/v1/auth/login", json={"email": email,
"password": password})
    assert response.status_code == 200
    return response.json()


def auth_headers(token: str) -> dict[str, str]:
    return {"Authorization": f"Bearer {token}"}


def admin_headers(client) -> dict[str, str]:
    return auth_headers(login(client)["access_token"])


def role_id_by_name(client, headers: dict[str, str], role_name: str) ->
str:
    response = client.get("/api/v1/roles/", headers=headers)
    assert response.status_code == 200
    for role in response.json():
        if role["name"] == role_name:
            return role["id"]
    raise AssertionError(f"No existe el rol {role_name}")


def department_id(client, headers: dict[str, str]) -> str:
    response = client.get("/api/v1/departments/", headers=headers)
    assert response.status_code == 200
    assert response.json()
    return response.json()[0]["id"]


def valid_diagram_payload(department_id_value: str) -> dict:
    return {
        "notation": "uml_activity",
        "uml_version": "2.5",
        "lanes": [
            {
                "id": "lane-1",
                "department_id": department_id_value,
                "department_name": "Direccion General",
```

```
            }
        ],
        "nodes": [
            {
                "id": "start",
                "type": "initial",
                "label": "",
                "lane_id": None,
                "x": 40,
                "y": 80,
                "width": 32,
                "height": 32,
                "metadata": {},
            },
            {
                "id": "activity-1",
                "type": "activity",
                "label": "Registrar tramite",
                "lane_id": "lane-1",
                "x": 140,
                "y": 60,
                "width": 180,
                "height": 72,
                "metadata": {},
            },
            {
                "id": "end",
                "type": "final",
                "label": "",
                "lane_id": None,
                "x": 380,
                "y": 80,
                "width": 34,
                "height": 34,
                "metadata": {},
            },
        ],
        "edges": [
            {"id": "edge-1", "source_id": "start", "target_id":
"activity-1", "label": "", "metadata": {}},
            {"id": "edge-2", "source_id": "activity-1", "target_id":
"end", "label": "", "metadata": {}},
        ],
        "forms": [],
        "metadata": {"kind": "activity_swimlane", "orientation":
"horizontal"},
    }
```

2.5.1 Unit Testing para DiagramsService

```
class TestUnitTestingParaDiagramsService:
    def test_create_should_create_a_diagram_draft_successfully(self,
client):
        headers = admin_headers(client)

        response = client.post(
            "/api/v1/policies/drafts",
            headers=headers,
            json={
                "name": "Test Diagram",
                "description": "Test Description",
                "category": "Pruebas FT",
            },
        )

        assert response.status_code == 201
        body = response.json()
        assert body["name"] == "Test Diagram"
        assert body["description"] == "Test Description"
        assert body["status"] == "draft"
        assert body["diagram"]["notation"] == "uml_activity"
        assert body["diagram"]["uml_version"] == "2.5"

    def test_create_should_reject_invalid_diagram_draft_data(self,
client):
        headers = admin_headers(client)

        response = client.post(
            "/api/v1/policies/drafts",
            headers=headers,
            json={"name": "A", "description": "x", "category": ""},
        )

        assert response.status_code == 422

    def test_find_all_should_return_an_array_of_diagrams(self, client):
        headers = admin_headers(client)
        created = client.post(
            "/api/v1/policies/drafts",
            headers=headers,
            json={
                "name": "Listado Diagramas FT",
                "description": "Diagrama creado para validar findAll",
                "category": "Pruebas FT",
            },
        )
```

```
        assert created.status_code == 201

        response = client.get("/api/v1/policies/", headers=headers)

        assert response.status_code == 200
        assert isinstance(response.json(), list)
        assert any(item["id"] == created.json()["id"] for item in
response.json())

    def test_find_one_should_return_a_diagram_when_found(self, client):
        headers = admin_headers(client)
        created = client.post(
            "/api/v1/policies/drafts",
            headers=headers,
            json={
                "name": "Diagrama Individual FT",
                "description": "Diagrama creado para validar findOne",
                "category": "Pruebas FT",
            },
        )
        assert created.status_code == 201

        response = client.get(f"/api/v1/policies/{created.json()['id']}",
headers=headers)

        assert response.status_code == 200
        assert response.json()["id"] == created.json()["id"]
        assert response.json()["name"] == "Diagrama Individual FT"

    def
test_find_one_should_return_not_found_when_diagram_does_not_exist(self,
client):
        headers = admin_headers(client)

        response =
client.get("/api/v1/policies/64b000000000000000000001", headers=headers)

        assert response.status_code == 404
        assert "encontrada" in response.json()["detail"]

    def test_update_should_update_a_diagram_successfully(self, client):
        headers = admin_headers(client)
        created = client.post(
            "/api/v1/policies/drafts",
            headers=headers,
            json={
                "name": "Diagrama Actualizable FT",
                "description": "Diagrama creado para validar update",
                "category": "Pruebas FT",
```

```
            },
        )
        assert created.status_code == 201
        payload = valid_diagram_payload(department_id(client, headers))

        response =
client.put(f"/api/v1/policies/{created.json()['id']}/diagram",
headers=headers, json=payload)

        assert response.status_code == 200
        diagram = response.json()["diagram"]
        assert diagram["nodes"][1]["label"] == "Registrar tramite"
        assert diagram["edges"][0]["source_id"] == "start"

    def test_update_should_reject_invalid_diagram_structure(self,
client):
        headers = admin_headers(client)
        created = client.post(
            "/api/v1/policies/drafts",
            headers=headers,
            json={
                "name": "Diagrama Invalido FT",
                "description": "Diagrama creado para validar errores",
                "category": "Pruebas FT",
            },
        )
        assert created.status_code == 201
        payload = valid_diagram_payload(department_id(client, headers))
        payload["nodes"] = [node for node in payload["nodes"] if
node["type"] != "final"]

        response =
client.put(f"/api/v1/policies/{created.json()['id']}/diagram",
headers=headers, json=payload)

        assert response.status_code == 400
        assert response.json()["detail"] == "El diagrama debe tener al
menos un nodo final"
```

2.5.2 Unit Test para el UsersService

```
class TestUnitTestParaElUsersService:
    def test_create_should_create_a_user_successfully(self, client):
        headers = admin_headers(client)
        role_id = role_id_by_name(client, headers, "funcionario")
        dept_id = department_id(client, headers)

        response = client.post(
```

```
            "/api/v1/users/",
            headers=headers,
            json={
                "email": "test.ft@example.com",
                "full_name": "Usuario Test FT",
                "password": "Usuario123*",
                "role_id": role_id,
                "department_id": dept_id,
                "is_active": True,
            },
        )

        assert response.status_code == 201
        body = response.json()
        assert body["email"] == "test.ft@example.com"
        assert body["full_name"] == "Usuario Test FT"
        assert body["role_name"] == "funcionario"
        assert body["is_active"] is True

    def test_create_should_reject_duplicate_email(self, client):
        headers = admin_headers(client)
        role_id = role_id_by_name(client, headers, "funcionario")

        response = client.post(
            "/api/v1/users/",
            headers=headers,
            json={
                "email": "admin@example.com",
                "full_name": "Admin Duplicado",
                "password": "Usuario123*",
                "role_id": role_id,
                "department_id": None,
                "is_active": True,
            },
        )

        assert response.status_code == 409
        assert "correo" in response.json()["detail"]

    def test_find_all_should_return_an_array_of_users(self, client):
        headers = admin_headers(client)

        response = client.get("/api/v1/users/", headers=headers)

        assert response.status_code == 200
        users = response.json()
        assert isinstance(users, list)
        assert any(user["email"] == "admin@example.com" for user in
users)
```

```
    def test_find_one_should_return_a_user_when_found(self, client):
        headers = admin_headers(client)
        users = client.get("/api/v1/users/", headers=headers).json()
        admin_user = next(user for user in users if user["email"] ==
"admin@example.com")

        response = client.get(f"/api/v1/users/{admin_user['id']}",
headers=headers)

        assert response.status_code == 200
        assert response.json()["email"] == "admin@example.com"
        assert response.json()["role_name"] == "administrador"

    def
test_find_one_should_return_not_found_when_user_does_not_exist(self,
client):
        headers = admin_headers(client)

        response = client.get("/api/v1/users/64b00000000000000000001",
headers=headers)

        assert response.status_code == 404
        assert "Usuario" in response.json()["detail"]

    def test_update_should_update_a_user_successfully(self, client):
        headers = admin_headers(client)
        users = client.get("/api/v1/users/", headers=headers).json()
        user = next(item for item in users if item["email"] ==
"funcionario@example.com")

        response = client.patch(
            f"/api/v1/users/{user['id']}",
            headers=headers,
            json={"full_name": "Funcionario Actualizado FT"},
        )

        assert response.status_code == 200
        assert response.json()["full_name"] == "Funcionario Actualizado
FT"
        assert response.json()["email"] == "funcionario@example.com"

    def test_update_should_reject_duplicate_email(self, client):
        headers = admin_headers(client)
        users = client.get("/api/v1/users/", headers=headers).json()
        user = next(item for item in users if item["email"] ==
"funcionario@example.com")

        response = client.patch(
```

```
            f"/api/v1/users/{user['id']}",
            headers=headers,
            json={"email": "admin@example.com"},
        )

        assert response.status_code == 409
        assert "correo" in response.json()["detail"]

    def test_remove_should_deactivate_a_user_successfully(self, client):
        headers = admin_headers(client)
        users = client.get("/api/v1/users/", headers=headers).json()
        user = next(item for item in users if item["email"] ==
"funcionario@example.com")

        delete_response = client.delete(f"/api/v1/users/{user['id']}",
headers=headers)
        find_response = client.get(f"/api/v1/users/{user['id']}",
headers=headers)

        assert delete_response.status_code == 204
        assert find_response.status_code == 200
        assert find_response.json()["is_active"] is False

    def
test_remove_should_return_not_found_when_user_does_not_exist(self,
client):
        headers = admin_headers(client)

        response =
client.delete("/api/v1/users/64b000000000000000000001", headers=headers)

        assert response.status_code == 404
        assert "Usuario" in response.json()["detail"]

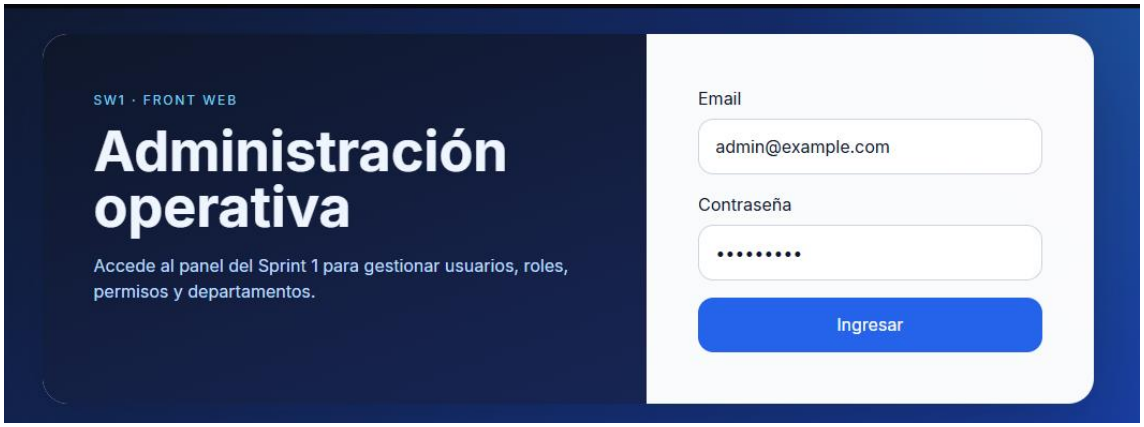
    def test_non_admin_should_not_access_user_crud(self, client):
        auth = login(client, email="funcionario@example.com",
password="Funcionario123*")
        headers = auth_headers(auth["access_token"])

        response = client.get("/api/v1/users/", headers=headers)

        assert response.status_code == 403
        assert "permisos" in response.json()["detail"]
```

3 DOCUMENTAFCION PARA EL USO DEL SOFTWARE

1. Iniciar Sesión



2. Crear Politica



O GENERA CON IA

Describe tu proceso de negocio

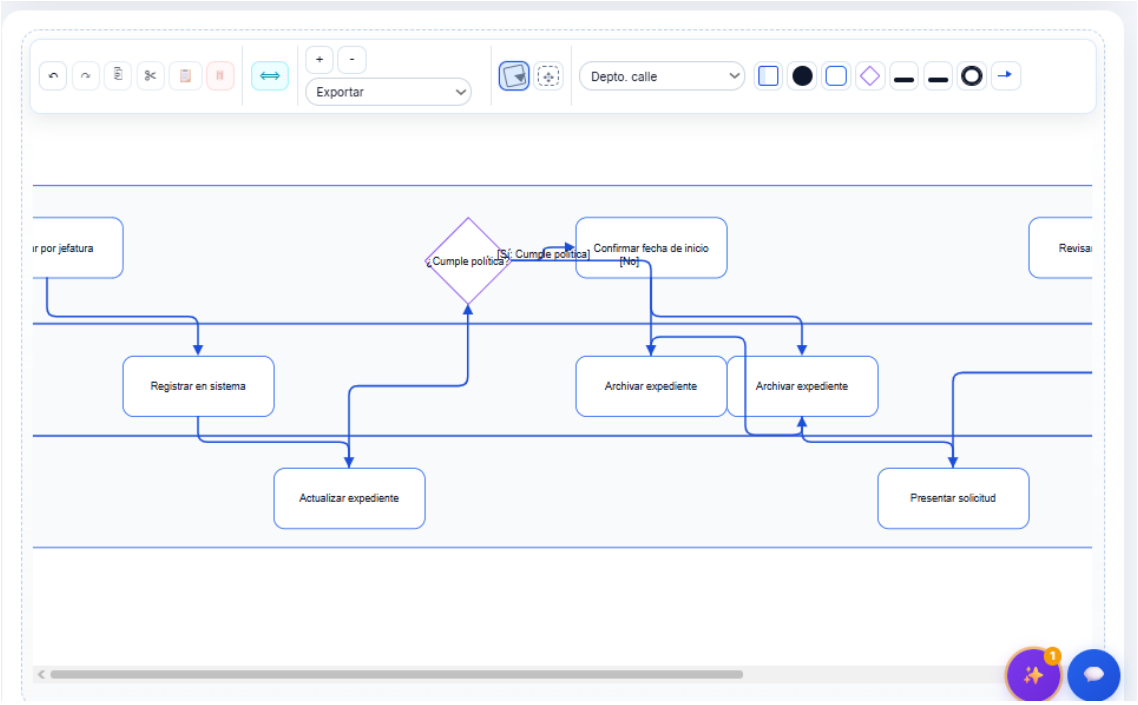
Necesito un proceso de vacaciones por 10 dias

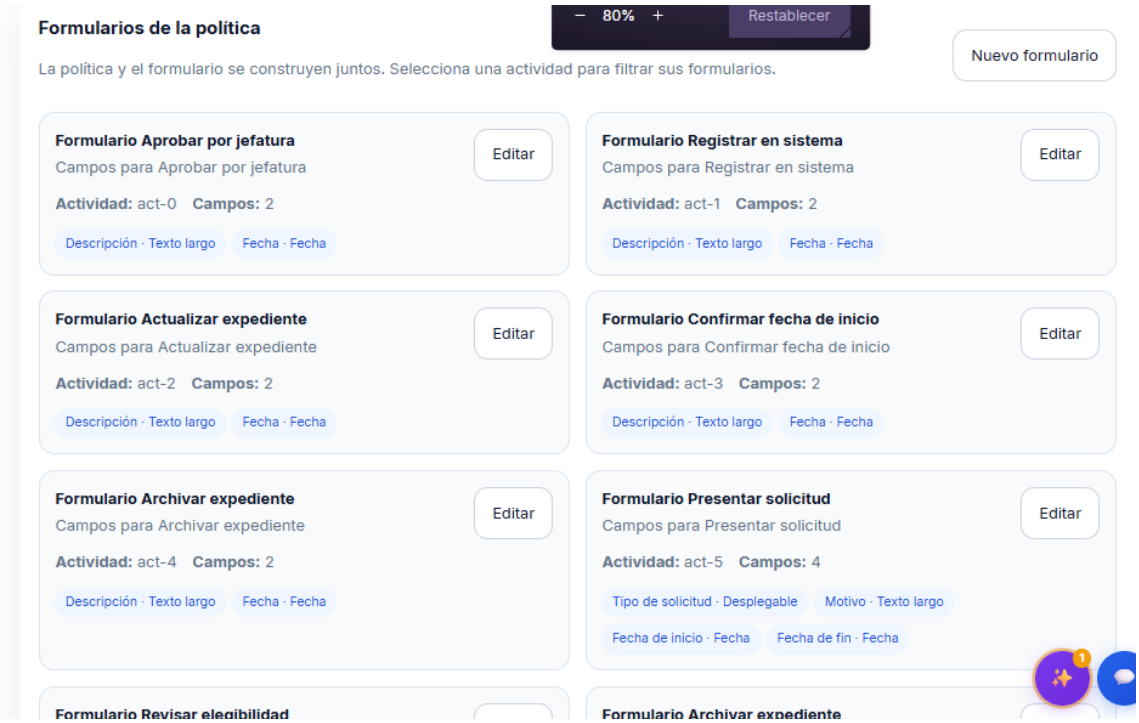
Generando... | Dictar proceso

La IA generará:

- Diagrama de actividades UML 2.5 con calles
- Nodos: inicio, actividades, decisiones, fork/join, final
- Carriles (swimlanes) por departamento
- Formularios con campos contextuales por actividad
- Categoría y nombre inferidos del prompt

El resultado se abre como borrador listo para editar.





4 Bibliografía

Metodología y Proceso de Desarrollo

- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The Unified Software Development Process*. Addison-Wesley.
- IBM. (2004). *Rational Unified Process Release Notes (v2003.06.00)*. https://public.dhe.ibm.com/software/xde/2003.06.12.01/install/DISK1/doc/rup_readme.html

Lenguaje Unificado de Modelado (UML)

- Object Management Group. (2017). *UML® Specification Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1
- Object Management Group. (2017). *UML 2.5.1 Machine Readable Files (XMI)*. https://www.omg.org/spec/UML/machine-readable

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.

Herramientas CASE

- Sparx Systems. (2024). *Enterprise Architect User Guide (v16.1)*. https://sparxsystems.com.au/enterprise_architect_user_guide/16.1/getting_started/availablehelpfileformats.html
- Sparx Systems. (2025). *Introducing SysMLv2 Modeling in Enterprise Architect Trechoro*. https://prolaborate.sparxsystems.com/resources/articles/sparx-sysmlv2-tool-enterprise-architect-trechoro

Patrones de Workflow y BPM

- Russell, N., ter Hofstede, A.H.M., van der Aalst, W.M.P., & Mulyar, N. (2006). *Workflow Control-Flow Patterns: A Revised View*. BPM Center Report BPM-06-22. http://www.workflowpatterns.com/patterns/control/
- Van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B., & Barros, A.P. (2003). Workflow Patterns. *Distributed and Parallel Databases, 14*(3), 5-51. http://www.workflowpatterns.com/patterns/control/

Backend y API (FastAPI + Python)

- FastAPI. (2024). *FastAPI – Modern, Fast (High-Performance), Web Framework for Building APIs with Python*. https://fastapi.tiangolo.com/
- Starlette. (2024). *Starlette – The Little ASGI Framework*. https://www.starlette.io/
- Pydantic. (2024). *Pydantic – Data Validation Using Python Type Hints*. https://docs.pydantic.dev/
- Python Software Foundation. (2024). *Python 3.12 Documentation*. https://docs.python.org/3/

Base de Datos (MongoDB)

- MongoDB. (2025). *PyMongo Tutorial: Building a CRUD Application with FastAPI and MongoDB*. https://www.mongodb.com/resources/languages/pymongo-tutorial
- MongoDB. (2025). *MongoDB Documentation*. https://www.mongodb.com/docs/
- Motor. (2024). *Motor: Asynchronous Python Driver for MongoDB*. https://motor.readthedocs.io/

Frontend Web (Angular)

- Google. (2024). *Angular – The Modern Web Developer's Platform*. https://angular.dev/
- TypeScript. (2024). *TypeScript Documentation*. https://www.typescriptlang.org/docs/

Aplicación Móvil (Flutter)

- Google. (2024). *Flutter Documentation*. https://docs.flutter.dev/
- Dart. (2024). *Dart Documentation*. https://dart.dev/guides

Infraestructura en la Nube (AWS)

- Amazon Web Services. (2024). *Amazon EC2 Documentation*. https://docs.aws.amazon.com/ec2/
- Amazon Web Services. (2024). *Amazon Simple Storage Service (S3) Developer Guide*. https://docs.aws.amazon.com/AmazonS3/latest/dev/
- Amazon Web Services. (2024). *Amazon Elastic Container Registry (ECR) Documentation*. https://docs.aws.amazon.com/ecr/
- Amazon Web Services. (2024). *AWS Identity and Access Management (IAM) Documentation*. https://docs.aws.amazon.com/iam/

- Amazon Web Services. (2024). *Amazon CloudWatch Documentation*. https://docs.aws.amazon.com/cloudwatch/
- Boto3. (2024). *Boto3 – The AWS SDK for Python*. https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

Contenedores y Orquestación

- Docker. (2024). *Docker Documentation*. https://docs.docker.com/
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239).

Inteligencia Artificial y Deep Learning

- TensorFlow. (2024). *TensorFlow Documentation*. https://www.tensorflow.org/api_docs
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). https://web.stanford.edu/~jurafsky/slp3/

Reconocimiento de Voz y OCR

- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. *Proceedings of the 40th International Conference on Machine Learning*. https://proceedings.mlr.press/v202/radford23a.html
- Tesseract OCR. (2024). *Tesseract Open Source OCR Engine*. https://github.com/tesseract-ocr/tesseract

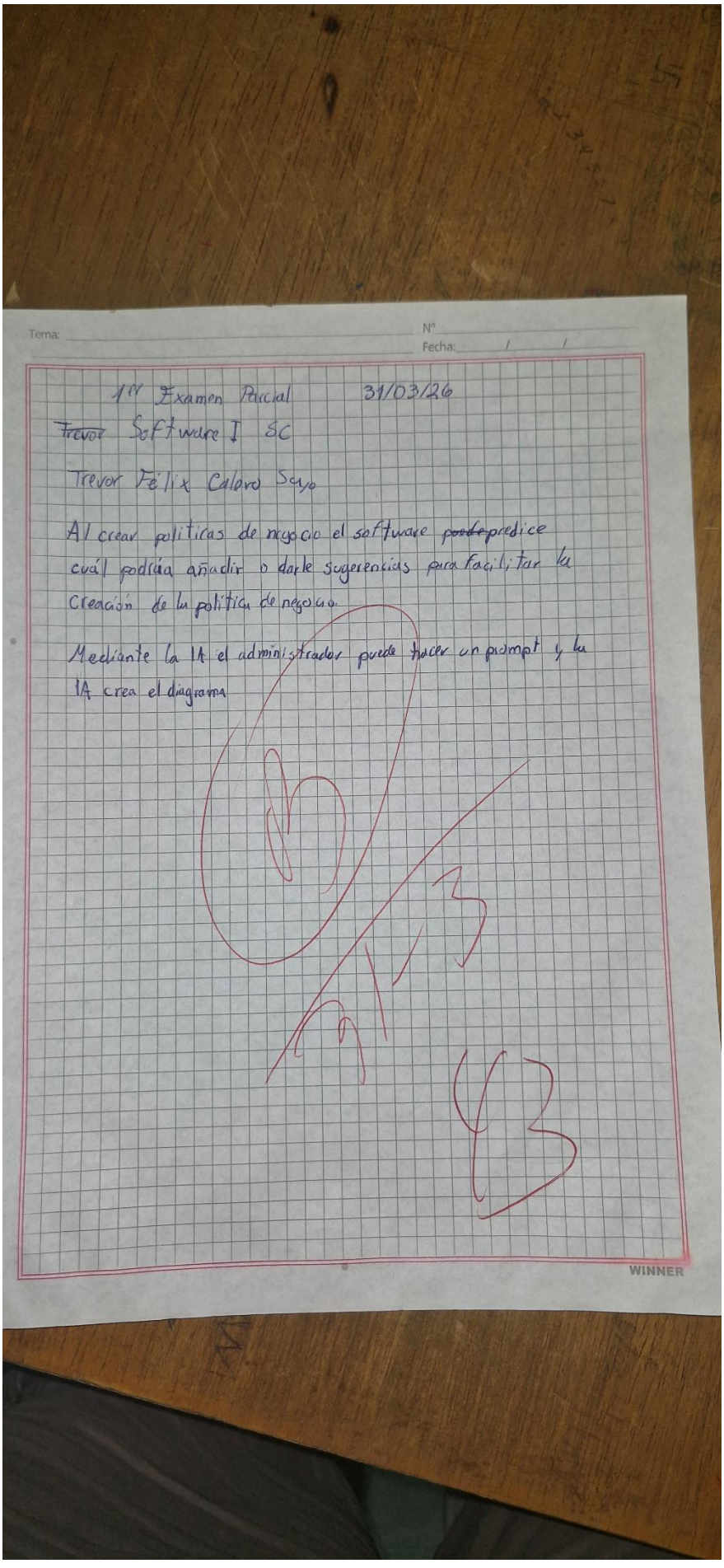
Ingeniería de Software

- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

- Schach, S. R. (2011). *Object-Oriented and Classical Software Engineering* (8th ed.). McGraw-Hill Education.

Estándares y Mejores Prácticas

- OpenAPI Initiative. (2024). *OpenAPI Specification v3.1.0*. https://spec.openapis.org/oas/v3.1.0
- JSON Web Token (JWT). (2015). *RFC 7519*. https://datatracker.ietf.org/doc/html/rfc7519
- Mozilla. (2024). *Security/Server Side TLS*. https://wiki.mozilla.org/Security/Server_Side_TLS



Tema: N°
Fecha: / /

1er Examen Parcial 31/03/26

~~Trevor~~ Software I SC

Trevor Félix Calero Sayo

Al crear políticas de negocio el software ~~podrá~~ predice cuál podría añadir o darle sugerencias para facilitar la creación de la política de negocio.

Mediante la IA el administrador puede hacer un prompt y la IA crea el diagrama

WINNER